2026 EDITION

Advanced Java, DSA and Job Interview Preparation

From Java fluency to data structures, algorithms, coding interviews and backend readiness

```java
// Java 21 LTS · data structures · algorithms · interviews
Deque<Integer> stack = new ArrayDeque<>();
PriorityQueue<int[]> pq =
        new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
Map<String, List<String>> groups = new HashMap<>();
```

Java 21 LTS · Collections · Data structures · Algorithms · Interviews · Backend basics

Version 0.1 (pilot)

Generated 15 October 2026

Contents

Preface iii
What this book assumes iii
What this book tries to do differently iii
Java version iii
A note on accuracy iii

How to Use This Book iv
How the book is organized iv
How each topic is taught iv
Reading the boxes iv
Reading the code v
A sample section: reversing an array in place vi
Exercises and solutions vii
The companion files OPTIONAL vii

Part I Java for Problem Solving

1 Modern Java Essentials for DSA 2
1.1 How Java runs your code 3
1.2 Java versions and LTS releases 4
1.3 Primitive and reference types 5
1.4 Numbers: conversions, overflow and precision 7
1.5 Equality: == versus equals 9
1.6 Strings, StringBuilder and StringBuffer 11
1.7 final, static, this and super 14
1.8 null and NullPointerException 15
1.9 Arrays 16
1.10 Summary 18
1.11 Exercises 18
1.12 Solutions 20

2 I/O, Standard-Library Tools and Lambda Essentials 27
2.1 Reading input: Scanner and BufferedReader 27
2.2 A fast reader 29
2.3 Writing output fast 31
2.4 Input patterns 32
2.5 Big numbers: BigInteger and BigDecimal 35
2.6 Arrays and Collections utilities 37
2.7 Lambda and method-reference essentials 38
2.8 Sorting with built-in APIs and comparators 40
2.9 A reusable template 42
2.10 Summary 44
2.11 Exercises 45
2.12 Solutions 46

Part III Data Structures and Algorithms

10 Stacks, Queues and Deques 54
10.1 Stacks: last in, first out 54
10.2 Queues: first in, first out 59
10.3 Deques and Java's ArrayDeque 61
10.4 Balanced brackets 64
10.5 Evaluating expressions with stacks 66
10.6 Monotonic stacks 70
10.7 Monotonic queues: the sliding-window maximum 73
10.8 Queues in breadth-first search (a preview) 75
10.9 Building one structure from another 76
10.10 Choosing the right structure 78
10.11 Summary 78
10.12 Exercises 79
10.13 Solutions 80

Glossary 91

References 94

Java language, JVM and tools . . . 94
Java Enhancement Proposals (JEPs) . . . 94
Java language specification chapters used . . . 94
Java API documentation (Java SE 21) . . . 94
OpenJDK source code . . . 96
Algorithms and data structures . . . 96
Java practice and style . . . 96
Online judges . . . 96
Tools used to build this book . . . 96

Index . . . **97**

Preface

This book is for a computer science student who already writes Java and wants to use it well: to solve data structure and algorithm (DSA) problems, to pass online assessments and technical interviews, and to start building real backend software. It was written with the Bangladesh job market of 2026 in mind, but almost everything in it applies to software interviews anywhere.

What this book assumes

You already know the fundamentals of Java: variables and types, operators, `if` and `switch`, loops, methods, arrays, `String`, classes and objects, constructors, inheritance, encapsulation, polymorphism, abstraction, interfaces, and basic exception handling. The book does not teach these again. When an early chapter returns to a familiar feature, such as == or `String`, it does so to show the traps, the internal behavior, and the way the feature is used in problem solving.

What this book tries to do differently

Most interview material shows you a finished solution. This book tries to show you how a solution is *found*. For every important algorithm you will see:

- the problem it solves and why the obvious approach is too slow;
- the idea that leads to the faster approach, and why that idea is correct;
- a complete Java program that was compiled and run while this book was built;
- a hand trace (a *dry run*) that shows every intermediate state;
- the time and space cost, with the reasoning behind it;
- the mistakes people usually make, and how to avoid them;
- the questions an interviewer is likely to ask next.

The goal is that you can write such solutions yourself, debug them when they fail, and explain your decisions clearly to another engineer.

Java version

The book uses **Java 21**, a long-term support (LTS) release, as its baseline. Every program compiles with `javac --release 21`. **Java 25**, the next LTS release, is mentioned where it adds something worth knowing, and such material is always labelled. Online judges sometimes run older versions of Java, so the competitive-programming templates avoid newer features and state the version they need.

A note on accuracy

Technical claims are checked against official documentation wherever possible, and the sources are listed in the References at the end of the book. The book separates what the Java specification *guarantees* from what the current OpenJDK implementation *happens to do*, because interviewers sometimes ask about the difference.

The chapters about jobs in Bangladesh separate verified observations (with a source and the date it was checked) from general advice. Nothing in this book can promise you a job, and no fixed schedule guarantees mastery. What the book can do is help you prepare well and measure your own progress honestly.

How to Use This Book

How the book is organized

The book has seven parts. Part III is the heart of the book; the other parts prepare you for it or help you use it in interviews and at work.

Table 1: The seven parts of the book

Part	Chapters	What it gives you
I. Java for Problem Solving	1–4	Types and their traps, fast I/O, lambdas for comparators, advanced OOP, debugging and testing
II. Java Internals, Complexity and Collections	5–7	The JVM and memory, Big-O analysis, how every major collection works inside
III. Data Structures and Algorithms	8–21	Arrays to dynamic programming, each implemented, traced and analyzed
IV. Modern Java Features Worth Knowing	22–23	Streams in depth, records, pattern matching, concurrency basics
V. Reliable Solutions in Online Assessments	24–25	Templates, verdicts, and a step-by-step workflow
VI. Job Preparation in Bangladesh, 2026	26–28	Model interview answers, backend Java with Spring Boot, a job-search plan
VII. Practice, Revision and Assessment	29–31	Practice problems, a question bank, a revision handbook

Read Parts I and II before Part III unless you are already confident with Java collections and Big-O notation. Inside Part III, the chapters are ordered so that each one only depends on earlier ones. Every chapter lists its prerequisites at the start.

How each topic is taught

Important topics get the full treatment, in this order: **what** the idea is, **why** it exists, **how** it works step by step, how it works **internally**, the **syntax** in detail, a complete **program**, a **line-by-line** explanation, a **dry run**, the **complexity**, the **common mistakes**, **when to use** it, the **interview** angle, and finally **exercises** with separate **solutions**.

Smaller topics use a shorter version of the same pattern. Material that is useful but not needed for most interviews is marked with an Optional badge, like the heading of the last section in this chapter. You can skip optional material on a first reading.

Every chapter starts with a box that lists its goals and prerequisites, followed by a short warm-up quiz, and ends with a summary, exercises and solutions.

Reading the boxes

Six kinds of boxes appear in the text. Each has its own color.

DEFINITION · **Algorithm**
A precise, finite sequence of steps that solves a problem. Definition boxes give the exact meaning of a term the first time it appears. The glossary at the end of the book collects them.

KEY IDEA · **Invariants**
A key-idea box states the one insight that makes a technique work. If you remember only one thing from a section, remember its key idea.

NOTE
A note adds detail that is useful but not essential, such as a version difference or a link to a later chapter.

WARNING · **Integer overflow**
A warning box points out a mistake that compiles without complaint but produces wrong answers, crashes, or a slow program. Many failed interview submissions come from the mistakes in these boxes.

INTERVIEW TIP · **Think aloud**
An interview tip tells you what interviewers usually look for and how to explain your reasoning so that they can follow it.

EXAMPLE · **Choosing a structure**
An example box works through a small concrete case, usually with real values.

Reading the code

All code in this book is Java. Each listing has a header that says what kind of code it is:

- **Complete program**: compiles and runs as shown. Its file name appears in the header.
- **Method**: one method (or a few) taken from a complete program; the file name shows where it lives.
- **Fragment**: a few lines that show a point. A fragment is not a complete program.
- **Pseudocode**: an algorithm written in plain steps; it is not Java.
- **Output** and **Input**: what a program printed, and what it read, when the book was built.

Listings have line numbers so that the explanations can refer to them. Here is a short complete program.

Listing 1 Complete program — `ReverseInPlace.java`

```
 1 import java.util.Arrays;
 2
 3 public class ReverseInPlace {
 4
 5     /** Reverses values in place using two pointers that move toward each other. */
 6     static void reverse(int[] values) {
 7         int left = 0;
 8         int right = values.length - 1;
 9         while (left < right) {
10             int temp = values[left];
11             values[left] = values[right];
12             values[right] = temp;
13             left++;
14             right--;
15         }
16     }
17
18     public static void main(String[] args) {
19         int[] numbers = {3, 8, 1, 9, 4};
20         System.out.println("Before: " + Arrays.toString(numbers));
21         reverse(numbers);
```

```
        System.out.println("After:  " + Arrays.toString(numbers));
    }
}
```

When the book was built, this program printed:

Listing 2 Output ReverseInPlace

```
Before: [3, 8, 1, 9, 4]
After:  [4, 9, 1, 8, 3]
```

To run any program yourself, open a terminal in the folder that contains it and type the two commands below. The first compiles the source code into bytecode; the second starts the Java Virtual Machine (JVM) and runs the `main` method.

Listing 3 Terminal

```
javac ReverseInPlace.java
java ReverseInPlace
```

A sample section: reversing an array in place

This short section shows how a typical topic looks inside a chapter. The task is to reverse an array without creating a second array.

The idea. Put one index, `left`, at the start and another, `right`, at the end. Swap the two values, then move both indices one step toward the middle. Stop when they meet.

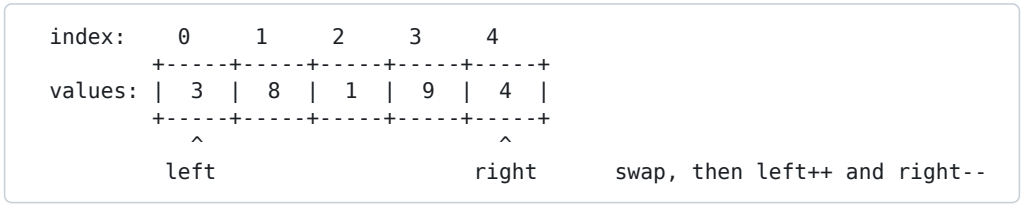


Figure 1: Two pointers moving toward the middle

The method below is the `reverse` method from the program above, shown on its own.

Listing 4 Method ReverseInPlace.java

```
1  /** Reverses values in place using two pointers that move toward each other. */
2  static void reverse(int[] values) {
3      int left = 0;
4      int right = values.length - 1;
5      while (left < right) {
6          int temp = values[left];
7          values[left] = values[right];
8          values[right] = temp;
9          left++;
10         right--;
11     }
12 }
```

Line by line. Line 3 and line 4 place the two indices at both ends. The loop on line 5 runs while `left` is still before `right`; when they meet or cross, every pair has been swapped. Lines 6 to 8 swap two values through the temporary variable `temp`, because writing `values[left] = values[right]` first would lose the old value of `values[left]`. Lines 9 and 10 move both indices inward.

Dry run. For the input `[3, 8, 1, 9, 4]`:

Table 2: Dry run of `reverse` on [3, 8, 1, 9, 4]

Step	left	right	Swap	Array after the step
start	0	4	–	[3, 8, 1, 9, 4]

Step	left	right	Swap	Array after the step
1	0	4	3 ↔ 4	[4, 8, 1, 9, 3]
2	1	3	8 ↔ 9	[4, 9, 1, 8, 3]
stop	2	2	–	`left < right` is false, so the loop ends

Complexity. Each loop iteration does a constant amount of work and moves both indices, so the loop runs about $n/2$ times for an array of length n.

Table 3: Cost of reversing in place

Measure	Cost	Why
Time	O(n)	about n/2 iterations, each O(1)
Extra space	O(1)	only `left`, `right` and `temp`, whatever the array size

> **WARNING** · **Off-by-one**
> Writing `while (left <= right)` also works here, because swapping the middle element with itself changes nothing. But in many two-pointer algorithms the difference between `<` and `<=` decides whether the answer is right. Always ask: *what should happen when the two pointers meet?*

Exercises and solutions

Exercises are numbered by chapter and marked Easy, Medium, Hard or Challenge. Try each one before you look at its solution. The solutions are collected at the end of each chapter; a link under every exercise takes you there, and a link on every solution takes you back.

> **Exercise 0.1** EASY **Reverse part of an array**
> Change `reverse` so that it reverses only the part of the array between index `from` and index `to`, inclusive, and leaves the rest unchanged. What should your method do when `from > to`?
>
> Solution →

> **Solution 0.1** ← Exercise
> Start the pointers at `from` and `to` instead of `0` and `values.length - 1`; the loop body stays the same. When `from > to` the loop condition `left < right` is false at the start, so the method changes nothing, which is a sensible result for an empty range. The method should still reject indices outside the array, for example by throwing an `IndexOutOfBoundsException` before the loop starts.

The companion files OPTIONAL

The book was built from a folder of source files that you can use directly:

Table 4: Companion files

Folder or file	Contents
`code-examples/`	every program in the book, by chapter
`exercises/`	the exercises of each chapter, as separate files
`solutions/`	worked solutions, with solution programs by chapter
`study_roadmap.md`	the 12-week and 16-week study plans
`references.md`	sources and further reading
`validation-report.md`	what was compiled, run and checked when the PDF was built

The `README.md` file in that folder explains how to rebuild this PDF after you change anything.

PART I

Java for Problem Solving

The Java you need to write correct, fast, readable solutions: types and their traps, fast input and output, lambdas for comparators, advanced object-oriented design, and a disciplined way to debug and test.

1 Modern Java Essentials for DSA
2 I/O, Standard-Library Tools and Lambda Essentials

CHAPTER 1

Modern Java Essentials for DSA

IN THIS CHAPTER

After this chapter you will be able to:

- explain what happens between writing a `.java` file and seeing its output, and the roles of the JDK, the compiler, bytecode and the JVM;
- choose a Java version for practice and online assessments, and know which features belong to which version;
- predict exactly what Java does with primitive and reference variables, including when a method can and cannot change its caller's data;
- avoid the numeric traps that cause most "wrong answer" verdicts: overflow, integer division, floating-point error and character arithmetic;
- use == and `equals` correctly, and explain the string pool and the `Integer` cache;
- build strings efficiently and choose between `String`, `StringBuilder` and `StringBuffer`;
- use `final`, `static`, `this` and `super` with intent, handle `null` safely, and copy one- and two-dimensional arrays correctly.

Prerequisites: the Java fundamentals listed in the Preface. Nothing from earlier chapters is needed.

WARM-UP QUIZ · **Diagnostic**

Answer these before you read the chapter. The answers, and the section to read if you missed one, are at the start of the Solutions section.

1. What does `javac` produce, and what program runs it?
2. What does `System.out.println(7 / 2 + 7 % 2);` print?
3. Can an `int` hold 3,000,000,000? If not, which type would you use?
4. What value does `long big = 50_000 * 50_000;` store?
5. After `int[] a = {1, 2}; int[] b = a; b[0] = 9;`, what is `a[0]`?
6. Is Java "pass by value" or "pass by reference"?
7. What is `"abc" == new String("abc")`, and why?
8. Why is `s += x` inside a long loop slow when `s` is a `String`?
9. With `Integer a = 1000, b = 1000;`, what is `a == b`?
10. What are the elements of `new boolean[3]` and of `new String[3]`?
11. `counts` is a `Map<String, Integer>`. What happens on `int n = counts.get("missing");`?
12. What is the difference between `StringBuilder` and `StringBuffer`?

You already write Java. This chapter makes sure you also know *exactly* what your Java does, because in problem solving the small details decide the verdict. A solution with the right idea still fails if a product overflows an `int`, if two strings are compared with ==, or if a loop builds a long string one += at a time. Each section below takes one of these details, explains what the language guarantees, shows a program that was run while this book was built, and points out the traps.

1.1 How Java runs your code

> **DEFINITION · Bytecode and the JVM**
> Bytecode is the instruction set of an imaginary computer, stored in `.class` files. The Java Virtual Machine (JVM) is the program that loads `.class` files and executes their bytecode on a real computer.

Running a Java program takes two steps:

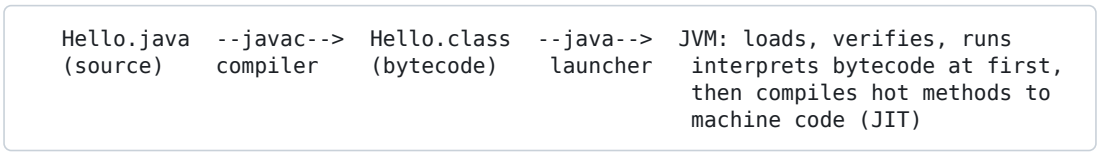


Figure 1.1: From source code to a running program

1. **Compile.** `javac` (the Java compiler) checks your source code for errors and translates each class into a `.class` file containing bytecode.
2. **Run.** `java` starts a JVM, which loads the classes it needs, *verifies* that the bytecode is safe, and executes it. At first the JVM *interprets* bytecode one instruction at a time. Methods that run often ("hot" methods) are then compiled to native machine code by the just-in-time compiler (JIT).

Why two steps? Bytecode is the same on every operating system, so a `.class` file compiled on Windows runs on any JVM, on Linux or macOS too. The JIT then gives speed close to that of natively compiled languages for code that runs long enough to be optimized. Chapter 5 looks inside the JVM in detail.

The program below lets you see real bytecode. It asks `javap`, the JDK's class-file disassembler, to describe the compiled `add` method of the program itself.

Listing 1.1 Complete program — ShowBytecode.java

```java
import java.io.PrintWriter;
import java.io.StringWriter;
import java.util.spi.ToolProvider;

/** Asks the JDK's javap tool to show the bytecode that javac produced for add(). */
public class ShowBytecode {

    static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        ToolProvider javap = ToolProvider.findFirst("javap").orElseThrow();
        StringWriter disassembly = new StringWriter();
        String classPath = System.getProperty("java.class.path");
        javap.run(new PrintWriter(disassembly), new PrintWriter(System.err),
                "-c", "-cp", classPath, "ShowBytecode");

        boolean inAdd = false;  // print only the part of javap's output about add()
        for (String line : disassembly.toString().split("\\R")) {
            if (line.contains("static int add(int, int)")) {
                inAdd = true;
            } else if (inAdd && line.isBlank()) {
                break;
            }
            if (inAdd) {
                System.out.println(line);
            }
        }
    }
}
```

Listing 1.2 Output ShowBytecode

```
static int add(int, int);
  Code:
       0: iload_0
       1: iload_1
       2: iadd
       3: ireturn
```

`iload_0` and `iload_1` push the two `int` parameters onto the method's *operand stack*, `iadd` pops both and pushes their sum, and `ireturn` returns the top value. The JVM is a *stack machine*: almost every instruction takes its inputs from, and leaves its result on, a stack (Chapter 10 builds stacks of its own).

How the program works. Line 13 looks up the `javap` tool through `java.util.spi.ToolProvider`, which lets a Java program run JDK tools in the same process. `findFirst` returns an `Optional<ToolProvider>` (a box that may be empty; Chapter 22) and `orElseThrow()` takes the tool out, or throws if this runtime has no `javap`. Lines 16–17 run it with the options `-c` (show bytecode) and `-cp` (where to find classes), writing into a `StringWriter`. Lines 19–29 print only the lines about `add`. `split("\\R")` splits the text at any kind of line break.

JDK, JRE and JVM. The terms are often confused:

Table 1.1: JVM, JRE and JDK

Term	What it is	You need it to...
JVM	the virtual machine that runs bytecode	run any Java program
JRE	*Java Runtime Environment*: a JVM plus the standard class library	run programs
JDK	*Java Development Kit*: a JRE plus development tools (`javac`, `javap`, `jshell`, `jar`)	write and compile programs

Since Java 11, the JDK is the normal download for everyone; some vendors still publish runtime-only packages. Install a JDK.

> **NOTE · Quick experiments**
> Since Java 11 you can run a single source file without compiling it first: `java Hello.java`. The launcher compiles the file in memory and runs it. `jshell`, the interactive Java shell that ships with the JDK, is even quicker for checking one expression, such as `-7 % 2` or `(int) 3.99`.

1.2 Java versions and LTS releases

Since Java 10 (March 2018), a new Java version has been released every six months, in March and September. Most versions are supported only until the next one appears. Some are **long-term support** (LTS) releases, which vendors maintain for years; companies build and run their software on LTS versions.

Table 1.2: Long-term support releases and the features introduced up to each one

LTS version	Released	Highlights you will use in this book
Java 8	March 2014	lambdas, method references, streams
Java 11	September 2018	`var` (from 10), `String.strip`, `isBlank`, `repeat`, running `java File.java`
Java 17	September 2021	switch expressions (14), text blocks (15), records and `instanceof` patterns (16), sealed classes
Java 21	September 2023	sequenced collections, pattern matching for `switch`, record patterns, virtual threads
Java 25	September 2025	compact source files and instance `main` methods, module import declarations, flexible constructor bodies, scoped values

This book uses **Java 21** as its baseline because it is a widely available LTS release and has every language feature that matters for problem solving. Java 25 is newer; its features are marked "Java 25" wherever they appear. For example, Java 25 finalized *compact source files*, which let a small program skip the class declaration:

Listing 1.3 Complete program (Java 25) HelloCompact.java

```java
// Java 25 only: a compact source file (JEP 512). There is no class declaration;
// the compiler wraps these members in a class named after the file.
void main() {
    IO.println("Hello from a compact source file");
    int[] values = {3, 1, 2};
    IO.println("first value: " + values[0]);
}
```

Listing 1.4 Output HelloCompact

```
Hello from a compact source file
first value: 3
```

`IO.println` is a Java 25 convenience class for console output. The same file does not compile with `javac --release 21`; the compiler reports that implicitly declared classes are not supported in that version. That is exactly why online-assessment code should stay with features your judge supports.

What do online judges run? It varies, and it changes. When this book was written, HackerRank's official *Execution Environment* page listed separate Java 8, 15, 17 and 21 options (Java 21 as OpenJDK 21.0.4), plus an option labelled simply "Java" that runs Java 1.7 (page checked on 2026-10-15). LeetCode and Codeforces do not offer a page that could be checked automatically, so their versions are not stated here. Always look at the language menu of the platform you use.

> **WARNING · Pick the version explicitly**
> On a platform that offers several Java versions, choose the newest one you know works. A solution that uses `var`, `List.of` or switch expressions fails to *compile* on Java 7 or 8, and a compile error usually scores zero. The templates in this book avoid features newer than Java 17 unless they say otherwise.

1.3 Primitive and reference types

Java has two kinds of types, and they behave differently in assignments, comparisons and method calls.

> **DEFINITION · Primitive and reference types**
> A variable of a primitive type (`int`, `long`, `double`, `char`, `boolean`, `byte`, `short`, `float`) holds the value itself. A variable of a reference type (every class, interface, array and enum) holds a *reference*: the information the JVM needs to find an object, or `null` if it refers to nothing.

Why both exist. Primitives are small and fast: an `int` is 4 bytes, and an `int[]` of a million elements is one block of 4 MB. Objects carry extra bookkeeping (Chapter 5), but they can represent anything and can be shared between different parts of a program.

Table 1.3: The eight primitive types

Type	Size	Range	Default	Typical DSA use
`byte`	8 bits	−128 to 127	`0`	compact flags
`short`	16 bits	−32,768 to 32,767	`0`	rarely used
`int`	32 bits	-2^{31} to $2^{31}-1$ (about $\pm 2.1 \times 10^9$)	`0`	indices, counts, most values
`long`	64 bits	-2^{63} to $2^{63}-1$ (about $\pm 9.2 \times 10^{18}$)	`0L`	sums, products, timestamps
`float`	32 bits	about 7 significant decimal digits	`0.0f`	avoid in DSA
`double`	64 bits	about 15–16 significant decimal digits	`0.0`	geometry, probabilities
`char`	16 bits	0 to 65,535 (one UTF-16 code unit)	`'\u0000'`	characters, letter counts
`boolean`	not specified	`true`, `false`	`false`	visited flags

The defaults apply to fields and array elements. Local variables have no default: the compiler refuses to read one before it is assigned.

Listing 1.5 Complete program — ValuesAndReferences.java

```
1  /** Primitive variables hold values; reference variables hold references to objects. */
2  public class ValuesAndReferences {
3
4      static void tryToChange(int number, int[] array) {
5          number = 99;              // changes only this method's copy of the value
6          array[0] = 99;            // changes the array object that the caller also sees
7          array = new int[] {-1};   // re-points only this method's copy of the reference
8      }
9
10     public static void main(String[] args) {
11         int a = 5;
12         int b = a;                // copies the value 5 into b
13         b++;
14         System.out.println("a = " + a + ", b = " + b);
15
16         int[] first = {1, 2, 3};
17         int[] second = first;     // copies the reference: both names, ONE array
18         second[0] = 100;
19         System.out.println("first[0] = " + first[0]);
20
21         int number = 1;
22         int[] data = {1, 2, 3};
23         tryToChange(number, data);
24         System.out.println("number = " + number + ", data[0] = " + data[0]
25                 + ", data.length = " + data.length);
26     }
27 }
```

Listing 1.6 Output — ValuesAndReferences

```
a = 5, b = 6
first[0] = 100
number = 1, data[0] = 99, data.length = 3
```

Line by line. Line 12 copies the *value* 5 into `b`, so incrementing `b` cannot affect `a`. Line 17 copies the *reference* stored in `first`, so `first` and `second` now name the same array, and the change on line 18 is visible through both names. Lines 21–25 call `tryToChange`. Inside it, line 5 changes the method's own copy of `number`, line 6 follows the copied reference to the caller's array and changes it, and line 7 points the method's copy of the reference at a brand-new array, which the caller never sees.

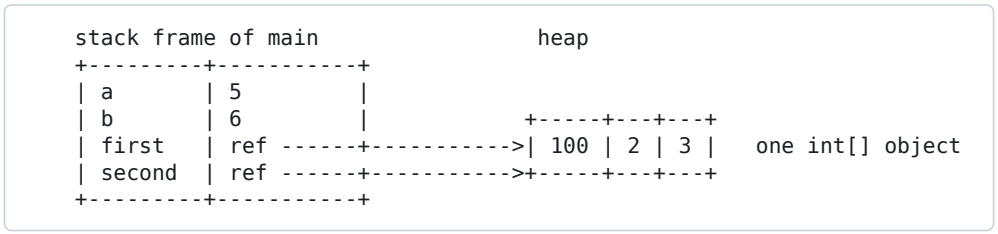


Figure 1.2: Two references to one array after int[] second = first

The diagram is a conceptual model: the JVM may store things differently, but every JVM behaves *as if* it worked like this.

> **INTERVIEW TIP** · **"Is Java pass by reference?"**
> A precise answer: "Java is always pass by value. For an object or array, the value that is copied is the reference. So a method can change the object the caller's reference points to, but it cannot make the caller's variable point to a different object." Then show the three lines of `tryToChange`.

1.4 Numbers: conversions, overflow and precision

More online-assessment submissions fail because of arithmetic details than because of wrong algorithms. The program below shows the details that matter most; each group of lines is explained after the output.

Listing 1.7 Complete program — NumericTraps.java

```java
/** Arithmetic results that surprise people: overflow, division, floating point, chars. */
public class NumericTraps {

    public static void main(String[] args) {
        int big = Integer.MAX_VALUE;                        // 2147483647
        System.out.println("MAX_VALUE + 1    = " + (big + 1));
        long widened = big + 1L;                            // 1L is a long: the sum is long
        System.out.println("MAX_VALUE + 1L   = " + widened);
        try {
            Math.addExact(big, 1);                          // checks for overflow
        } catch (ArithmeticException e) {
            System.out.println("Math.addExact    -> " + e);
        }

        int a = 100_000;
        int b = 100_000;
        long wrong = a * b;          // int * int overflows BEFORE the result becomes long
        long right = (long) a * b;   // a becomes long first, so the product is long
        System.out.println("long wrong = a * b        = " + wrong);
        System.out.println("long right = (long) a * b = " + right);

        System.out.println(" 7 / 2 = " + (7 / 2) + ",   7 % 2 = " + (7 % 2));
        System.out.println("-7 / 2 = " + (-7 / 2) + ",  -7 % 2 = " + (-7 % 2));
        System.out.println("Math.floorDiv(-7, 2) = " + Math.floorDiv(-7, 2)
                + ", Math.floorMod(-7, 2) = " + Math.floorMod(-7, 2));
        System.out.println("7 / 2.0 = " + (7 / 2.0));

        double sum = 0.1 + 0.2;
        System.out.println("0.1 + 0.2 = " + sum + ", == 0.3? " + (sum == 0.3));
        System.out.println("within 1e-9 of 0.3? " + (Math.abs(sum - 0.3) < 1e-9));
        System.out.println("(int) 3.99 = " + (int) 3.99
                + ", (int) -3.99 = " + (int) -3.99);
        System.out.println("Math.round(2.5) = " + Math.round(2.5)
                + ", Math.round(-2.5) = " + Math.round(-2.5));
        System.out.println("(int) 1e10 = " + (int) 1e10);
        System.out.println("1.0 / 0 = " + (1.0 / 0) + ", 0.0 / 0 = " + (0.0 / 0));

        char letter = 'c';
        System.out.println("'c' - 'a' = " + (letter - 'a'));
        System.out.println("'a' + 1   = " + ('a' + 1));
        System.out.println("(char) ('a' + 1) = " + (char) ('a' + 1));
        System.out.println("'7' - '0' = " + ('7' - '0'));

        int low = 2_000_000_000;
        int high = 2_100_000_000;
        System.out.println("(low + high) / 2       = " + (low + high) / 2);
        System.out.println("low + (high - low) / 2 = " + (low + (high - low) / 2));
    }
}
```

Listing 1.8 Output — NumericTraps

```
MAX_VALUE + 1    = -2147483648
MAX_VALUE + 1L   = 2147483648
Math.addExact    -> java.lang.ArithmeticException: integer overflow
long wrong = a * b        = 1410065408
long right = (long) a * b = 10000000000
 7 / 2 = 3,   7 % 2 = 1
-7 / 2 = -3,  -7 % 2 = -1
Math.floorDiv(-7, 2) = -4, Math.floorMod(-7, 2) = 1
7 / 2.0 = 3.5
0.1 + 0.2 = 0.30000000000000004, == 0.3? false
```

```
within 1e-9 of 0.3? true
(int) 3.99 = 3, (int) -3.99 = -3
Math.round(2.5) = 3, Math.round(-2.5) = -2
(int) 1e10 = 2147483647
1.0 / 0 = Infinity, 0.0 / 0 = NaN
'c' - 'a' = 2
'a' + 1   = 98
(char) ('a' + 1) = b
'7' - '0' = 7
(low + high) / 2       = -97483648
low + (high - low) / 2 = 2050000000
```

1.4.1 Overflow

> **DEFINITION · Integer overflow**
> Integer overflow happens when the exact result of an `int` or `long` operation does not fit in the type. Java does not report it: the result silently *wraps around*, keeping only the lowest 32 (or 64) bits.

Line 6 adds 1 to the largest `int` and gets the *smallest* `int`, −2,147,483,648. Chapter 19 explains why, using the binary representation of negative numbers (two's complement). Line 7 writes `1L`, a `long` literal, so Java converts `big` to `long` before adding and the sum is correct. `Math.addExact` (line 10) is the checked version: it throws `ArithmeticException` instead of wrapping. `Math` also has `subtractExact`, `multiplyExact`, `incrementExact`, `negateExact` and `toIntExact`.

Widening happens too late (lines 15–20). This is the most common overflow bug in interviews. In `long wrong = a * b;`, both operands are `int`, so Java multiplies them as `int` values. The product 10^{10} overflows and wraps to 1,410,065,408, and only *then* is that wrong value converted to `long`. The variable's type on the left does not influence how the right side is computed. Casting one operand first, `(long) a * b`, makes the multiplication itself use `long`.

> **WARNING · Check the constraints for overflow**
> If a problem says values go up to 10^9, any sum of two values, and any product, may overflow `int`. If n values up to 10^9 are added, the total can reach $n \times 10^9$. Use `long` for sums and products, and cast *before* multiplying. Chapter 19 covers modular arithmetic for answers that do not fit even in a `long`.

1.4.2 Conversions between numeric types

Java converts automatically in the "safe" direction, from a smaller type to a larger one: `byte` → `short` → `int` → `long` → `float` → `double`, and `char` → `int`. This is called *widening*. The other direction, *narrowing*, needs an explicit cast such as `(int) 3.99`, because information can be lost.

Two rules explain most surprises:

- **Arithmetic promotes small types to `int`.** In `'a' + 1`, the `char` becomes the `int` 97, so the result is the `int` 98 (line 40). To get a character back, cast: `(char) ('a' + 1)` is `'b'` (line 41). The same rule makes `byteValue + 1` an `int`.
- **Mixed arithmetic uses the larger type.** In `7 / 2.0`, the `int` 7 becomes a `double`, so the division is a `double` division: 3.5 (line 26).

Character arithmetic is used constantly in string problems. Because the letters `'a'` to `'z'` and the digits `'0'` to `'9'` have consecutive codes, `ch - 'a'` turns a lowercase letter into an index from 0 to 25, and `ch - '0'` turns a digit character into its value (lines 39 and 42).

Casting a `double` to an `int` removes the fractional part, rounding *toward zero*: 3.99 becomes 3 and −3.99 becomes −3 (lines 31–32). If the value is too large for an `int`, the cast does not wrap; it gives the nearest `int`, here 2,147,483,647 for 10^{10} (line 35). `Math.round` rounds to the nearest whole number, with halves rounded *up* (toward positive infinity): 2.5 becomes 3 but −2.5 becomes −2 (lines 33–34).

1.4.3 Integer division and remainder

Integer division discards the fraction, rounding toward zero: `7 / 2` is 3 and `-7 / 2` is −3 (lines 22–23). The remainder operator `%` follows the same rule, so `a % b` has the sign of `a`: `-7 % 2` is −1, not 1. This breaks code like `if (x % 2 == 1)` for negative odd numbers. Use `x % 2 != 0` to test for odd numbers, or `Math.floorMod(x, 2)`, which always returns a result between 0 and `b - 1` for positive `b` (lines 24–25). `Math.floorDiv` rounds toward negative infinity instead.

Division by zero behaves differently for the two kinds of number. `5 / 0` with `int` operands throws `ArithmeticException`. With `double` operands, `1.0 / 0` is `Infinity` and `0.0 / 0` is `NaN` ("not a number"), and no exception is thrown (line 36). `NaN` is not equal to anything, even itself, so test it with `Double.isNaN`.

1.4.4 Floating-point precision

A `double` stores numbers in binary, and most decimal fractions, such as 0.1, have no exact binary representation. The stored value is the closest one available, so small errors appear: `0.1 + 0.2` is `0.30000000000000004`, and comparing it with `0.3` using == gives `false` (line 29).

Compare doubles with a tolerance instead (line 30): `Math.abs(x - y) < 1e-9`. Choose the tolerance from the problem; many geometry problems state the precision they require. Better still, avoid `double` when the problem can be solved with integers: compare `a / b` with `c / d` as `a * d` with `c * b` (using `long`), and store money as a whole number of the smallest unit (paisa or cents), or use `BigDecimal` (Chapter 2).

1.4.5 The midpoint bug

Binary search computes the middle of a range. The obvious formula `(low + high) / 2` overflows when `low + high` exceeds the `int` range, which happens on large indices or when searching over a range of values (Chapter 12). Lines 44–47 show it: the result is a *negative* index. The safe formula is `low + (high - low) / 2`, because `high - low` cannot overflow when both are non-negative and `low <= high`.

Table 1.4: Choosing a numeric type

Value range	Use	Notes
indices, counts, values up to about 2×10^9	`int`	the default; fastest and smallest
sums or products that may pass 2×10^9	`long`	cast *before* multiplying
results larger than 9.2×10^{18}	`BigInteger` or modular arithmetic	`BigInteger` is slow; see Chapters 2 and 19
real numbers, ratios, geometry	`double`	compare with a tolerance; never use `float`
money, exact decimals	`long` (smallest unit) or `BigDecimal`	never `double`

1.5 Equality: == versus equals

> **KEY IDEA · What == compares**
>
> For primitives, == compares *values*. For references, == compares *identity*: whether the two references point to the very same object. To compare the *contents* of two objects, call `equals`, which each class defines for itself.

Listing 1.9 Complete program — EqualityDemo.java

```
1  import java.util.Arrays;
2
3  /** == compares primitive values but object references; equals compares content. */
4  public class EqualityDemo {
5
6      public static void main(String[] args) {
7          String literal1 = "java";
8          String literal2 = "java";            // the same pooled object as literal1
9          String created = new String("java"); // a new object with the same characters
10         String constant = "ja" + "va";       // a compile-time constant: also pooled
11         String part = "ja";
12         String computed = part + "va";       // built while the program runs: a new object
```

```
13          System.out.println("literal1 == literal2         " + (literal1 == literal2));
14          System.out.println("literal1 == created          " + (literal1 == created));
15          System.out.println("literal1.equals(created)     " + literal1.equals(created));
16          System.out.println("literal1 == constant         " + (literal1 == constant));
17          System.out.println("literal1 == computed         " + (literal1 == computed));
18          String interned = computed.intern();  // the pooled copy of the same text
19          System.out.println("literal1 == computed.intern() " + (literal1 == interned));
20
21          Integer small1 = 127;
22          Integer small2 = 127;
23          Integer large1 = 1000;
24          Integer large2 = 1000;
25          System.out.println("Integer 127 == 127           " + (small1 == small2));
26          System.out.println("Integer 1000 == 1000         " + (large1 == large2));
27          System.out.println("large1.equals(large2)        " + large1.equals(large2));
28
29          int[] x = {1, 2, 3};
30          int[] y = {1, 2, 3};
31          System.out.println("x == y                       " + (x == y));
32          System.out.println("x.equals(y)                  " + x.equals(y));
33          System.out.println("Arrays.equals(x, y)          " + Arrays.equals(x, y));
34      }
35  }
```

Listing 1.10 Output — EqualityDemo

```
literal1 == literal2         true
literal1 == created          false
literal1.equals(created)     true
literal1 == constant         true
literal1 == computed         false
literal1 == computed.intern() true
Integer 127 == 127           true
Integer 1000 == 1000         false
large1.equals(large2)        true
x == y                       false
x.equals(y)                  false
Arrays.equals(x, y)          true
```

Strings (lines 7–19). Two string literals with the same text are the *same object* (line 13 prints `true`), because the JVM keeps one shared copy of every literal in the string pool. The Java Language Specification guarantees this, and also that a *constant expression* such as `"ja" + "va"` is pooled (line 16), since the compiler computes it before the program runs. But `new String("java")` always creates a new object (line 14), and so does concatenation at run time (line 17). `intern()` returns the pooled copy of a string's text, which is why line 19 prints `true`. In everyday code you never need `intern`; you need `equals`, which compares the characters (line 15).

Boxed integers (lines 21–27). `Integer small1 = 127;` is compiled to `Integer.valueOf(127)`. The documentation of `Integer.valueOf` promises that values from −128 to 127 are cached, so the same object is returned each time, and `==` happens to give `true`. For 1000 there is no such promise. With default JVM settings a new object is created each time, and `==` gives `false`. Code that compares `Integer` objects with `==` therefore passes small tests and fails large ones; Chapter 5 shows where this bites in collections.

Arrays (lines 29–33). Arrays do not override `equals`, so `x.equals(y)` is the same as `x == y`: identity. Use `Arrays.equals` to compare contents, and `Arrays.deepEquals` for arrays of arrays.

Table 1.5: Choosing between == and equals

To compare…	Use
primitives (`int`, `char`, `double`, …)	`==` (for `double`, a tolerance)
any object's contents (`String`, `Integer`, …)	`a.equals(b)`
objects that might be `null`	`Objects.equals(a, b)`
a string with a constant	`"constant".equals(s)` (safe when `s` is `null`)
array contents	`Arrays.equals(a, b)`, `Arrays.deepEquals` for 2D
enum constants, or whether something is `null`	`==`

How `equals` must be written for your own classes, and why `hashCode` must always agree with it, is the subject of Chapter 3.

1.6 Strings, StringBuilder and StringBuffer

> **DEFINITION · Immutable**
> An object is immutable if its state can never change after it is created. A `String` is immutable: every method that seems to modify a string, such as `toUpperCase`, `replace` or `concat`, returns a *new* string and leaves the original untouched.

Why `String` is immutable. Because a string can never change, it can be shared safely: by many variables, by many threads, and through the string pool. Its hash code can be computed once and cached, which makes strings fast map keys (Chapter 7). And code that receives a string, such as a file name or a password check, knows nobody can change it afterwards.

How strings are stored. Since Java 9, OpenJDK stores a string whose characters all fit in one byte (Latin-1) with one byte per character, and other strings with two bytes per character (a feature called *compact strings*). Either way, `charAt` and `length` are O(1). Methods that produce a new string, such as `substring`, `concat`, `replace` and `toUpperCase`, copy characters and cost time proportional to the length of the result.

1.6.1 Building strings efficiently

Listing 1.11 Complete program — `StringBuilding.java`

```java
/** Strings never change; building text with += copies it again and again. */
public class StringBuilding {

    /** Builds "0,1,2,..," with +=; each += makes a new string and copies the old. */
    static String joinNaive(int n) {
        String result = "";
        for (int i = 0; i < n; i++) {
            result += i + ",";
        }
        return result;
    }

    /** Builds the same text with one growable buffer. */
    static String joinWithBuilder(int n) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            sb.append(i).append(',');
        }
        return sb.toString();
    }

    /** Characters that joinNaive copies in total: every new string is written in full. */
    static long charactersCopiedNaive(int n) {
        long copied = 0;
        long length = 0;
        for (int i = 0; i < n; i++) {
            length += String.valueOf(i).length() + 1;  // the string after this step
            copied += length;
        }
        return copied;
    }

    public static void main(String[] args) {
        String word = "stack";
        String upper = word.toUpperCase();  // returns a NEW string; word is unchanged
        word.concat("s");                   // the result is thrown away: a classic bug
        System.out.println(word + " " + upper);

        boolean same = joinNaive(2_000).equals(joinWithBuilder(2_000));
        System.out.println("same text: " + same);
```

```
41          for (int n : new int[] {1_000, 10_000, 100_000}) {
42              System.out.printf("n = %,7d: final length %,9d, += copies %,16d characters%n",
43                      n, joinWithBuilder(n).length(), charactersCopiedNaive(n));
44          }
45
46          StringBuilder sb = new StringBuilder("level");
47          sb.insert(0, '[').append(']');      // [level]
48          sb.setCharAt(1, 'L');               // [Level]
49          sb.deleteCharAt(sb.length() - 1);  // [Level
50          System.out.println(sb + ", length " + sb.length() + ", [2] " + sb.charAt(2));
51          System.out.println(new StringBuilder("stressed").reverse());
52      }
53  }
```

Listing 1.12 Output StringBuilding

```
stack STACK
same text: true
n =   1,000: final length     3,890, += copies        1,896,995 characters
n =  10,000: final length    48,890, += copies      239,429,495 characters
n = 100,000: final length   588,890, += copies   28,939,799,495 characters
[Level, length 6, [2] e
desserts
```

Line by line. Lines 34–37 show immutability: `toUpperCase` returns a new string, and `word.concat("s")` creates `"stacks"` and throws it away, so `word` is still `"stack"`. The naive method (lines 4–11) runs `result += i + ","` n times. Each += creates a brand new string and copies every character of the old one into it. The method `charactersCopiedNaive` (lines 22–31) does not build anything; it adds up the length of every intermediate string, which is the number of characters copied. The output shows the effect: for n = 100,000 the final text has 588,890 characters, but += copies about 29 billion characters to build it. The total is roughly proportional to n^2, which is why the naive version takes seconds or minutes where `StringBuilder` takes milliseconds.

`StringBuilder` (lines 13–20) keeps one growable character array, like the `IntStack` of Chapter 10, and `append` adds to its end in O(1) amortized time. `toString()` copies the result into a new `String` once, at the end.

> **NOTE · What javac does with +**
> Since Java 9, the compiler turns a single expression like `a + b + c` into one efficient call that sizes the result exactly, so one-line concatenations are fine. What it cannot do is merge the iterations of a *loop*: each iteration is a separate concatenation that copies the string built so far. Use `StringBuilder` whenever a string grows inside a loop.

Other `StringBuilder` methods (lines 46–51). `insert`, `setCharAt`, `deleteCharAt`, `charAt`, `length` and `reverse` work much like their names suggest. `append` returns the builder itself, so calls can be chained. `insert` and `deleteCharAt` must shift the characters after the position, so they cost O(n), except at the end.

`StringBuffer` has the same methods as `StringBuilder`, but each method is `synchronized` so that several threads can share one buffer. That locking costs time and is useless in single-threaded code. It is an older class (Java 1.0); `StringBuilder` was added in Java 5 as the faster replacement. Use `StringBuilder`.

Table 1.6: Costs of common String and StringBuilder operations (n and m are lengths)

Operation	Cost	Notes
`s.length()`, `s.charAt(i)`	O(1)	
`s.substring(a, b)`	O(b − a)	copies the characters
`s + t`, `s.concat(t)`	O(n + m)	creates a new string
`s.equals(t)`, `s.compareTo(t)`	O(min(n, m))	stops at the first difference
`s.hashCode()`	O(n) once, then O(1)	the hash is cached in the object
`s.indexOf(t)`, `s.contains(t)`	O(n × m) worst case	the API promises no algorithm; assume a simple scan

Operation	Cost	Notes
`s.toCharArray()`, `String.valueOf(chars)`	O(n)	both copy
`sb.append(x)`	O(1) amortized per character	
`sb.insert(i, x)`, `sb.deleteCharAt(i)`	O(n)	shifts the rest
`sb.reverse()`, `sb.toString()`	O(n)	

1.6.2 String methods you will use constantly

Listing 1.13 Complete program — `StringMethods.java`

```
1  import java.util.Arrays;
2
3  /** The String methods that appear most often in DSA problems, and their traps. */
4  public class StringMethods {
5
6      public static void main(String[] args) {
7          String s = "racecar";
8          System.out.println(s.length() + " " + s.charAt(0) + " " + s.indexOf('c')
9                  + " " + s.lastIndexOf('c') + " " + s.substring(2, 5)
10                 + " " + s.contains("cec"));
11         char[] letters = s.toCharArray();       // a copy you may modify
12         Arrays.sort(letters);
13         System.out.println(new String(letters) + " " + String.valueOf(letters, 0, 3));
14
15         System.out.println("apple".compareTo("banana") + " " + "b".compareTo("a")
16                 + " " + "Apple".equalsIgnoreCase("APPLE"));
17
18         System.out.println(Arrays.toString("a,b,,c,,".split(",")));  // trailing "" lost
19         System.out.println(Arrays.toString("1.5.2".split(".")));     // "." is a regex!
20         System.out.println(Arrays.toString("1.5.2".split("\\.")));
21         System.out.println(Arrays.toString("  two   spaces ".strip().split("\\s+")));
22
23         System.out.println(String.join("-", "x", "y", "z") + " " + "ab".repeat(3)
24                 + " " + String.valueOf(42) + " " + Integer.parseInt("-17"));
25
26         char c = 'G';
27         System.out.println(Character.isLetter(c) + " " + Character.isDigit('7')
28                 + " " + Character.toLowerCase(c) + " " + Character.isLetterOrDigit('_'));
29     }
30 }
```

Listing 1.14 Output — `StringMethods`

```
7 r 2 4 cec true
aaccerr aac
-1 1 true
[a, b, , c]
[]
[1, 5, 2]
[two, spaces]
x-y-z ababab 42 -17
true true g false
```

Most of these are familiar; the important details are:

- `substring(2, 5)` (line 9) takes indices 2, 3 and 4: the end index is *excluded*. This half-open convention, from `start` included to `end` excluded, is used throughout the Java library, for example in `Arrays.copyOfRange` and `List.subList`.
- `toCharArray()` (line 11) returns a *copy* that you may sort or modify (line 12); the string itself cannot change. Sorting the characters of a string is a common first step in anagram problems (Chapter 11).
- `compareTo` (line 15) returns a negative number, zero or a positive number. For two strings that first differ at some position, it returns the difference of those two characters, so do not rely on the result being exactly −1 or 1.

- `split` takes a *regular expression*, not plain text (lines 18–21). A dot means "any character" in a regular expression, so `"1.5.2".split(".")` treats every character as a separator and returns an empty array; escape it as `"\\."`. `split` also drops empty strings at the end of the result, and `"\\s+"` splits on runs of whitespace. Call `strip()` first if the text may begin with whitespace, or the first element is `""`.
- `strip()` (Java 11) removes leading and trailing whitespace, including Unicode spaces; the older `trim()` removes only characters up to the space character.
- `Character.isLetterOrDigit`, `isDigit`, `isLetter` and `toLowerCase` (lines 26–28) work for all of Unicode. When input is guaranteed to be ASCII, comparisons like `ch >= 'a' && ch <= 'z'` are just as correct and slightly faster.

> **INTERVIEW TIP** · **"Why are strings immutable in Java?"**
> Give three reasons: *safety* (no one can change a string after another part of the program has checked it, and immutable objects are automatically thread-safe), *sharing* (the string pool can give every identical literal one shared object), and *speed in hash tables* (the hash code is computed once and cached). Then mention the cost: building a string piece by piece needs `StringBuilder`.

1.7 final, static, this and super

These four keywords appear in almost every program. The example shows what each one means in practice.

Listing 1.15 Complete program — FinalAndStatic.java

```
1  import java.util.Arrays;
2
3  /** What final, static, this and super mean in the code you write for problems. */
4  public class FinalAndStatic {
5
6      static final int MOD = 1_000_000_007;  // a constant: one copy, never reassigned
7      static int created = 0;                // one copy, shared by every object
8
9      private final String name;             // set once, in the constructor
10     private final int[] counts;            // the reference is final; the array is not
11
12     FinalAndStatic(String name, int size) {
13         this.name = name;                  // this.name is the field, name the parameter
14         this.counts = new int[size];
15         created++;
16     }
17
18     void record(int index) {
19         counts[index] = (counts[index] + 1) % MOD;  // allowed: changes the contents
20         // counts = new int[3];                     // would not compile: counts is final
21     }
22
23     @Override
24     public String toString() {
25         return name + Arrays.toString(counts);
26     }
27
28     static class Labelled extends FinalAndStatic {
29         Labelled(String name) {
30             super(name, 3);                // runs the parent constructor first
31         }
32
33         @Override
34         public String toString() {
35             return "<" + super.toString() + ">";  // reuses the parent's version
36         }
37     }
38
39     public static void main(String[] args) {
40         FinalAndStatic plain = new FinalAndStatic("plain", 3);
41         FinalAndStatic labelled = new Labelled("labelled");
42         plain.record(1);
```

```
        plain.record(1);
        labelled.record(2);
        System.out.println(plain + " " + labelled);
        System.out.println("objects created: " + FinalAndStatic.created);
    }
}
```

Listing 1.16 Output FinalAndStatic

```
plain[0, 2, 0] <labelled[0, 0, 1]>
objects created: 2
```

- **`static`** (lines 6–7) means "belongs to the class, not to one object". There is exactly one `created` counter however many objects exist, which is why line 46 prints 2. In problem solving, `static` is used for helper methods and constants. Be careful with `static` *mutable* fields: if a judge runs several test cases inside one JVM, a static field keeps its value from one test case to the next. Reset it at the start of each test, or keep the state in local variables.
- **`final`** on a variable means it can be assigned only once. `static final` makes a constant such as `MOD` (line 6). A `final` field must be assigned exactly once, by the end of the constructor (lines 9–10, 13–14). `final` makes the *reference* fixed, not the object: line 19 may change the contents of `counts`, but line 20, if it were not a comment, would not compile. On a class, `final` forbids subclasses; on a method, it forbids overriding.
- **`this`** (line 13) refers to the current object. It is needed when a parameter has the same name as a field: `this.name` is the field, `name` the parameter. `this(...)` as the first statement of a constructor calls another constructor of the same class.
- **`super`** refers to the parent class. `super(name, 3)` on line 30 runs the parent's constructor, and `super.toString()` on line 35 calls the parent's version of a method that this class overrides. Chapter 3 covers inheritance and overriding in depth.

A local variable that is never reassigned is called *effectively final*, even without the keyword. Lambdas (Chapter 2) may use local variables only if they are effectively final.

1.8 null and NullPointerException

`null` is the value of a reference that points to no object. Using it as if it pointed to an object, by calling a method, reading a field or an array element, or unboxing it, throws a NullPointerException (NPE).

Listing 1.17 Complete program NullHandling.java

```
import java.util.Objects;

/** Where NullPointerException comes from, and simple ways to avoid it. */
public class NullHandling {

    /** Rejects null at the start of a method, with a clear message. */
    static int countVowels(String text) {
        Objects.requireNonNull(text, "text must not be null");
        int count = 0;
        for (int i = 0; i < text.length(); i++) {
            if ("aeiouAEIOU".indexOf(text.charAt(i)) >= 0) {
                count++;
            }
        }
        return count;
    }

    public static void main(String[] args) {
        String missing = null;
        try {
            System.out.println(missing.length());
        } catch (NullPointerException e) {
            System.out.println("NPE: " + e.getMessage());
        }
        Integer boxed = null;
```

```
26          try {
27              int value = boxed;                    // unboxing null
28              System.out.println(value);
29          } catch (NullPointerException e) {
30              System.out.println("NPE (unboxing): " + e.getMessage());
31          }
32
33          String answer = null;
34          System.out.println("\"yes\".equals(answer)       " + "yes".equals(answer));
35          boolean same = Objects.equals(answer, null);  // true: both are null
36          System.out.println("Objects.equals(answer, null) " + same);
37          System.out.println("requireNonNullElse          "
38                  + Objects.requireNonNullElse(answer, "(no answer)"));
39
40          try {
41              countVowels(null);
42          } catch (NullPointerException e) {
43              System.out.println("guard: " + e.getMessage());
44          }
45      }
46  }
```

Listing 1.18 Output NullHandling

```
NPE: Cannot invoke "String.length()" because "<local1>" is null
NPE (unboxing): Cannot invoke "java.lang.Integer.intValue()" because "<local2>" is null
"yes".equals(answer)       false
Objects.equals(answer, null) true
requireNonNullElse          (no answer)
guard: text must not be null
```

Since Java 15, NPE messages say exactly what was `null` and what the program tried to do with it ("helpful NullPointerExceptions"; the feature arrived in Java 14, switched off by default). The names `<local1>` and `<local2>` appear because this book's examples are compiled without debugging information; compile with `javac -g`, as IDEs do, and the messages show the real variable names.

The second NPE (lines 25–31) deserves attention, because it is the most common NPE in DSA code. Unboxing a `null Integer` into an `int` throws. Exactly this happens with `int count = map.get(key);` when the key is not in the map, because `get` returns `null`. Use `map.getOrDefault(key, 0)` instead (Chapter 7).

Habits that prevent most NPEs:

- Write constant-first comparisons, `"yes".equals(answer)` (line 34), which are `false` rather than an exception when `answer` is `null`, or use `Objects.equals(a, b)` when either side may be `null` (lines 35–36).
- Check arguments at the start of public methods with `Objects.requireNonNull(value, "message")` (line 8). The program fails immediately, with a clear message, instead of somewhere far away.
- Use `Objects.requireNonNullElse(value, fallback)` (line 38) to substitute a default.
- Return empty arrays, strings and collections instead of `null` from your own methods.

1.9 Arrays

Arrays are the most important data structure in DSA: fixed-size, indexed from 0, with O(1) access to any element. Chapter 8 covers array algorithms; this section covers how arrays behave in Java.

Listing 1.19 Complete program ArraysBasics.java

```
1  import java.util.Arrays;
2
3  /** Creating, copying and printing arrays, including the traps with 2D arrays. */
4  public class ArraysBasics {
5
6      public static void main(String[] args) {
7          int[] scores = new int[4];            // every slot starts at 0
8          String[] names = new String[2];       // every slot starts at null
```

```
 9          boolean[] seen = new boolean[3];      // every slot starts at false
10          int[] primes = {2, 3, 5, 7};          // an array initializer
11          int[][] grid = new int[2][3];         // 2 rows, each with 3 columns
12          int[][] triangle = new int[3][];      // 3 rows, not created yet (all null)
13          for (int r = 0; r < triangle.length; r++) {
14              triangle[r] = new int[r + 1];     // a "jagged" array: row r has r + 1 slots
15          }
16          System.out.println(Arrays.toString(scores) + " " + Arrays.toString(names)
17                  + " " + Arrays.toString(seen) + " " + Arrays.toString(primes));
18          System.out.println(Arrays.deepToString(grid) + " "
19                  + Arrays.deepToString(triangle));
20
21          int[] whole = primes.clone();                  // same length, same values
22          int[] longer = Arrays.copyOf(primes, 6);       // extra slots are 0
23          int[] middle = Arrays.copyOfRange(primes, 1, 3);  // indices 1 and 2 (3 excluded)
24          int[] shifted = new int[5];
25          System.arraycopy(primes, 0, shifted, 1, 4);    // from, start, to, start, count
26          System.out.println(Arrays.toString(whole) + " " + Arrays.toString(longer) + " "
27                  + Arrays.toString(middle) + " " + Arrays.toString(shifted));
28
29          int[][] original = {{1, 2}, {3, 4}};
30          int[][] shallow = original.clone();            // copies the row REFERENCES
31          shallow[0][0] = 99;                            // ...so this changes original too
32          int[][] deep = new int[original.length][];
33          for (int r = 0; r < original.length; r++) {
34              deep[r] = original[r].clone();             // copy every row
35          }
36          deep[1][1] = -1;                               // original is not affected
37          System.out.println("original " + Arrays.deepToString(original)
38                  + ", deep " + Arrays.deepToString(deep));
39
40          int[] oneRow = new int[3];
41          int[][] rows = new int[3][];
42          Arrays.fill(rows, oneRow);                     // three references to ONE array
43          rows[0][0] = 7;
44          System.out.println("fill trap " + Arrays.deepToString(rows));
45      }
46  }
```

Listing 1.20 Output — ArraysBasics

```
[0, 0, 0, 0] [null, null] [false, false, false] [2, 3, 5, 7]
[[0, 0, 0], [0, 0, 0]] [[0], [0, 0], [0, 0, 0]]
[2, 3, 5, 7] [2, 3, 5, 7, 0, 0] [3, 5] [0, 2, 3, 5, 7]
original [[99, 2], [3, 4]], deep [[99, 2], [3, -1]]
fill trap [[7, 0, 0], [7, 0, 0], [7, 0, 0]]
```

Creating arrays (lines 7–15). `new int[4]` creates an array whose elements have the default value of their type: `0`, `false` or `null`. An *array initializer* such as `{2, 3, 5, 7}` creates and fills an array in one step. Every array has a `length` field (no parentheses), fixed when the array is created; strings use `length()`, and collections use `size()`.

Two-dimensional arrays are arrays of arrays. `new int[2][3]` creates an outer array of 2 references, each pointing to its own `int[3]`. Because every row is a separate array, rows can have different lengths: line 12 creates only the outer array, and lines 13–15 give row `r` exactly `r + 1` slots. Such an array is called *jagged*.

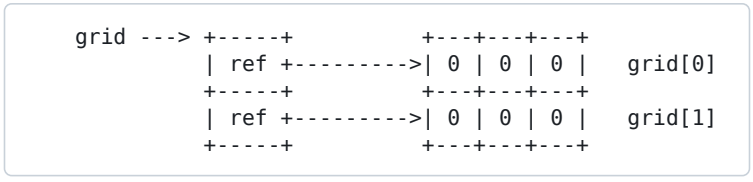


Figure 1.3: int[][] grid = new int[2][3] in memory

Copying (lines 21–25). All four ways copy element by element in O(n):

- `clone()` returns an array of the same length with the same elements;

- `Arrays.copyOf(a, n)` copies into a new array of length `n`, padding with default values or cutting off the end;
- `Arrays.copyOfRange(a, from, to)` copies the half-open range `from` to `to - 1`;
- `System.arraycopy(src, srcPos, dest, destPos, length)` copies into an *existing* array; it also handles overlapping ranges within one array correctly.

Shallow and deep copies (lines 29–36). `clone()` on a 2D array copies the outer array only: the copy contains the same row references, so changing `shallow[0][0]` also changes `original[0][0]`, as the output shows. A deep copy copies every row as well (lines 32–35).

The fill trap (lines 40–43). `Arrays.fill(rows, oneRow)` stores *the same reference* in every slot, so all three rows are one array, and writing `rows[0][0]` changes "every" row. To create independent rows, allocate each row separately, or use `new int[3][3]`.

> **WARNING · Printing arrays**
> `System.out.println(array)` prints something like `[I@1b6d3586`: the type and an identity hash, not the contents. Use `Arrays.toString(array)` for one dimension and `Arrays.deepToString(grid)` for more. In fast-output code (Chapter 2), print elements with a `StringBuilder` instead.

1.10 Summary

Java source code is compiled to bytecode, which the JVM verifies, interprets and, for hot code, compiles to machine code. Java 21 is this book's baseline; online judges vary, so choose the Java version explicitly. Primitive variables hold values; reference variables hold references, and Java passes both *by value*. Integer arithmetic silently overflows, integer division truncates toward zero, `%` keeps the sign of the dividend, and `double` values are approximations; most of these traps are avoided by using `long`, casting before multiplying, and comparing doubles with a tolerance. `==` compares references for objects, so compare contents with `equals`, `Objects.equals` or `Arrays.equals`. Strings are immutable, so build them with `StringBuilder`. Arrays have default values, 2D arrays are arrays of arrays, and copies are shallow unless you copy every row.

Table 1.7: Key takeaways

Topic	Remember
Running Java	`javac` → bytecode → JVM (interpreter + JIT)
Versions	LTS: 8, 11, 17, 21, 25; this book targets 21; check your judge
Pass by value	methods get copies of values and of references
Overflow	silent; use `long`, cast before multiplying, `Math.addExact` to detect
Division	truncates toward zero; `%` has the sign of the dividend; `floorMod`
Doubles	compare with a tolerance; avoid for money
Midpoint	`low + (high - low) / 2`
Equality	`==` for primitives and identity, `equals` for content
`Integer` cache	only −128 to 127 is guaranteed; never compare boxes with `==`
Strings	immutable; `StringBuilder` in loops; `split` takes a regex
`null`	constant-first `equals`, `requireNonNull`, beware unboxing `map.get`
Arrays	`clone` of a 2D array is shallow; `Arrays.fill` with a row shares it

1.11 Exercises

> **Exercise 1.1** EASY **Predict the output**
> Without running it, write down what each line prints. Then explain each answer using the rules from this chapter.

Listing 1.21 Fragment (not a complete program)

```
System.out.println(5 / 2 * 2.0);
System.out.println(5 / 2.0 * 2);
System.out.println('a' + 'b');
System.out.println("" + 'a' + 'b');
System.out.println(1 + 2 + "3" + 4 + 5);
System.out.println(-17 % 5);
System.out.println((byte) 200);
System.out.println(Integer.MIN_VALUE == -Integer.MIN_VALUE);
```

Solution →

Exercise 1.2 EASY Fix the midpoint

A binary search computes `int mid = (low + high) / 2;` where `0 <= low <= high`. Give values of `low` and `high` for which `mid` is wrong, and write two different correct formulas. Explain why each one is correct.

Solution →

Exercise 1.3 EASY The role check that only works in tests

A program checks `if (role == "admin")`, where `role` was read from the user. The programmer's tests, which call the method with the literal `"admin"`, all pass, but real users are never recognized as admins. Explain why, and fix the method so that it also works when `role` is `null`.

Solution →

Exercise 1.4 EASY A real deep copy

Write `int[][] deepCopy(int[][] source)` that returns a completely independent copy of a 2D array. It must work for jagged arrays, for rows that are `null`, and for `source` itself being `null`.

Solution →

Exercise 1.5 MEDIUM Sums without silent overflow

Write two methods for an `int[]`: `long sumAsLong(int[] values)`, which always returns the correct total, and `int sumExact(int[] values)`, which returns the total as an `int` but throws `ArithmeticException` if any intermediate sum overflows. Can `sumAsLong` itself overflow? Justify your answer.

Solution →

Exercise 1.6 EASY Letter counts with an array

Write `String letterCounts(String text)` that counts the lowercase letters `a` to `z` in `text` (ignoring every other character) and returns them in alphabetical order as letter followed by count, skipping letters that do not occur. For example, `"banana"` gives `"a3b1n2"`. Use an `int[26]`, not a map.

Solution →

Exercise 1.7 MEDIUM Reverse the words

Write `String reverseWords(String sentence)` that returns the words in reverse order, separated by single spaces, with no leading or trailing spaces. Words are separated by one or more whitespace characters. For example, `"  the sky  is blue "` gives `"blue is sky the"`, and a string of only spaces gives `""`.

Solution →

Exercise 1.8 EASY Trace the calls

Predict the output, and explain it in terms of values and references.

Listing 1.22 Fragment (not a complete program)

```
static void swapValues(int x, int y) { int t = x; x = y; y = t; }
static void swapFirstTwo(int[] a) { int t = a[0]; a[0] = a[1]; a[1] = t; }
static void replace(int[] a) { a = new int[] {7, 7}; a[0] = 8; }

int a = 1, b = 2;
int[] pair = {1, 2};
swapValues(a, b);
swapFirstTwo(pair);
replace(pair);
System.out.println(a + " " + b + " " + Arrays.toString(pair));
```

Solution →

Exercise 1.9 MEDIUM **Valid palindrome**

Write `boolean isPalindrome(String text)` that ignores every character that is not a letter or digit, and ignores case. `"A man, a plan, a canal: Panama"` is a palindrome; `"race a car"` is not. Use O(1) extra space: do not build a cleaned copy of the string.

Solution →

Exercise 1.10 MEDIUM **Factorial modulo a prime**

Compute n! modulo 1,000,000,007 for n up to 10^6. Explain why `int result = 1; result = result * i % MOD;` gives wrong answers even though every result is reduced modulo `MOD`, and write a correct version.

Solution →

Exercise 1.11 HARD **Run-length encoding in linear time**

Write `String encode(String text)` that replaces every run of equal characters with the character followed by the run length: `"aaabcc"` becomes `"a3b1c2"`. Your method must run in O(n). Then analyze a version that builds the result with +=: how many characters does it copy in the worst case?

Solution →

Exercise 1.12 HARD **Detect overflow yourself**

Without using `Math.multiplyExact`, write `boolean multiplyOverflows(int a, int b)` and `boolean multiplyOverflows(long a, long b)`. For the `long` version there is no larger primitive type to compute in. Test your methods on the edge cases `0`, `1`, `-1`, `Long.MIN_VALUE` and `Long.MAX_VALUE`.

Solution →

1.12 Solutions

1.12.1 Diagnostic quiz answers

1. Bytecode in `.class` files; the JVM runs it. (See Section 1.1.)
2. 4: `7 / 2` is 3 and `7 % 2` is 1. (Section 1.4.3)
3. No; the largest `int` is 2,147,483,647. Use `long`. (Section 1.3)
4. −1,794,967,296: the `int` product overflows before it is stored in the `long`. (Section 1.4.1)
5. 9: `a` and `b` refer to the same array. (Section 1.3)
6. Always pass by value; for objects the value is a reference. (Section 1.3)
7. `false`: == compares references and `new String` creates a new object. (Section 1.5)
8. Each += copies the whole string so far, so the total work grows with the square of the length. (Section 1.6)
9. `false` with default settings: only −128 to 127 are guaranteed to be cached. (Section 1.5)
10. `false, false, false` and `null, null, null`. (Section 1.9)
11. `NullPointerException`: `get` returns `null`, and unboxing `null` throws. (Section 1.8)
12. Same methods; `StringBuffer` synchronizes every call and is slower. Use `StringBuilder`. (Section 1.6)

If you missed three or more questions, read the whole chapter carefully. Otherwise, read the sections for the questions you missed and skim the rest.

Solution 1.1

← Exercise

Listing 1.23 Output — PredictOutput

```
4.0
5.0
195
ab
3345
-2
-56
true
```

- `5 / 2 * 2.0`: `5 / 2` is integer division, 2; then `2 * 2.0` is 4.0.
- `5 / 2.0 * 2`: `5 / 2.0` is 2.5; times 2 is 5.0.
- `'a' + 'b'`: both `char`s are promoted to `int`: 97 + 98 = 195.
- `"" + 'a' + 'b'`: `+` is evaluated left to right; `"" + 'a'` is the string `"a"`, so the next `+` also concatenates: `"ab"`.
- `1 + 2 + "3" + 4 + 5`: `1 + 2` is 3 (numbers); then `3 + "3"` is `"33"`, and every later `+` concatenates: `"3345"`.
- `-17 % 5`: the remainder has the sign of the dividend: −2.
- `(byte) 200`: 200 does not fit in a `byte` (−128 to 127). The cast keeps the lowest 8 bits, which represent 200 − 256 = −56.
- `Integer.MIN_VALUE == -Integer.MIN_VALUE`: negating -2^{31} gives 2^{31}, which overflows back to -2^{31}. So the value is its own negation, and the comparison is `true`. For the same reason, `Math.abs(Integer.MIN_VALUE)` is negative.

Solution 1.2

← Exercise

With `low = 1_500_000_000` and `high = 2_000_000_000`, the sum 3.5×10^9 overflows and the "middle" becomes negative.

Listing 1.24 Methods — Midpoint.java

```java
static int midOverflowing(int low, int high) {
    return (low + high) / 2;              // low + high can exceed Integer.MAX_VALUE
}

static int midSafe(int low, int high) {
    return low + (high - low) / 2;        // no overflow when 0 <= low <= high
}

static int midUnsignedShift(int low, int high) {
    return (low + high) >>> 1;            // treats the overflowed sum as unsigned
}
```

Listing 1.25 Output — Midpoint

```
overflowing:    -397483648
safe:           1750000000
unsigned shift: 1750000000
```

`low + (high - low) / 2` is correct because `high - low` is between 0 and `high`, so it fits, and adding half of it to `low` gives a value between `low` and `high`, which also fits. `(low + high) >>> 1` is correct for non-negative `low` and `high` for a different reason: the true sum is below 2^{32}, so even when it overflows into a negative `int`, its 32 bits are still the correct *unsigned* value. The unsigned right shift `>>>` divides that unsigned value by 2 (Chapter 19). The first formula is the clearest; use it.

Solution 1.3

← Exercise

`==` compares references. A literal `"admin"` in the test and the literal inside the method are the same pooled object, so the buggy version passes the test. Text typed by a user is built at run time into a *new* `String` object, so `==` is `false` even though the characters are equal.

Listing 1.26 Methods — RoleCheck.java

```
static boolean isAdminBuggy(String role) {
    return role == "admin";               // compares references
}

static boolean isAdmin(String role) {
    return "admin".equals(role);          // compares characters; null-safe
}
```

Listing 1.27 Output — RoleCheck

```
literal: buggy true, fixed true
typed:   buggy false, fixed true
null:    fixed false
```

Writing the constant first, `"admin".equals(role)`, compares characters, and it returns `false` instead of throwing when `role` is `null`. The lesson for testing is that tests built only from literals can hide this bug; include input built at run time.

Solution 1.4

← Exercise

Copy the outer array, then copy every row that is not `null`.

Listing 1.28 Method — DeepCopy.java

```
static int[][] deepCopy(int[][] source) {
    if (source == null) {
        return null;
    }
    int[][] copy = new int[source.length][];
    for (int r = 0; r < source.length; r++) {
        copy[r] = (source[r] == null) ? null : source[r].clone();
    }
    return copy;
}
```

Listing 1.29 Output — DeepCopy

```
original [[1, 2, 3], null, [4]]
copy     [[100, 2, 3], null, [9, 9]]
```

Changing the copy, both an element and a whole row, leaves the original unchanged. The method copies every element once, so it runs in time proportional to the total number of elements. For deeper nesting (`int[][][]`), the same idea is applied recursively.

Solution 1.5

← Exercise

Listing 1.30 Methods — SafeSum.java

```
/** Correct for any int array: the total of up to 2^31 ints always fits in a long. */
static long sumAsLong(int[] values) {
    long total = 0;
    for (int value : values) {
        total += value;                   // value is widened to long before adding
    }
    return total;
}

/** Returns an int sum, but throws ArithmeticException instead of overflowing. */
static int sumExact(int[] values) {
    int total = 0;
    for (int value : values) {
        total = Math.addExact(total, value);
    }
```

```
    return total;
}
```

Listing 1.31 Output — SafeSum

```
long sum: 3000000000
exact sum: java.lang.ArithmeticException: integer overflow
```

`sumAsLong` cannot overflow for any array Java can create. An array has at most about 2^{31} elements, and each `int` has absolute value at most 2^{31}, so the total is at most about 2^{62} in absolute value, below the `long` limit of about 2^{63}. In `sumExact`, the check must happen at *every* addition, as `Math.addExact` does: an intermediate sum can overflow even when the final total would fit, as in the example, where $2 \times 10^9 + 2 \times 10^9$ overflows before the -10^9 is added.

Solution 1.6

← Exercise

Listing 1.32 Method — LetterCounts.java

```java
static String letterCounts(String text) {
    int[] counts = new int[26];
    for (int i = 0; i < text.length(); i++) {
        char ch = text.charAt(i);
        if (ch >= 'a' && ch <= 'z') {
            counts[ch - 'a']++;            // 'a' -> 0, 'b' -> 1, ..., 'z' -> 25
        }
    }
    StringBuilder result = new StringBuilder();
    for (int letter = 0; letter < 26; letter++) {
        if (counts[letter] > 0) {
            result.append((char) ('a' + letter)).append(counts[letter]);
        }
    }
    return result.toString();
}
```

Listing 1.33 Output — LetterCounts

```
a3b1n2
d1e1h1l3o2r1w1
[]
```

`ch - 'a'` maps `'a'` to 0 through `'z'` to 25, and `(char) ('a' + letter)` maps back. The method is O(n + 26) = O(n) time with O(1) extra space, since the array always has 26 elements. An array is preferred over a `HashMap<Character, Integer>` here because it avoids boxing every character and count, and it keeps the letters in order for free.

Solution 1.7

← Exercise

Listing 1.34 Method — ReverseWords.java

```java
static String reverseWords(String sentence) {
    String trimmed = sentence.strip();
    if (trimmed.isEmpty()) {
        return "";
    }
    String[] words = trimmed.split("\\s+");      // one or more whitespace characters
    StringBuilder result = new StringBuilder();
    for (int i = words.length - 1; i >= 0; i--) {
        result.append(words[i]);
        if (i > 0) {
            result.append(' ');
        }
    }
```

```
14      return result.toString();
15  }
```

Listing 1.35 Output — ReverseWords

```
[blue is sky the]
[single]
[]
```

`strip()` removes the outer spaces first; otherwise `split` would return an empty first element for the leading spaces. The empty-string check is needed because `"".split(...)` returns an array containing one empty string. The method is O(n). An in-place version over a `char[]` (reverse the whole array, then reverse each word) is a classic follow-up; it uses the two-pointer technique of Chapter 8.

Solution 1.8

← Exercise

Listing 1.36 Output — SwapTrace

```
1 2 [2, 1]
```

`swapValues` swaps its own copies of the two values, so `a` and `b` are unchanged. `swapFirstTwo` receives a copy of the *reference* to `pair` and swaps the elements of that same array, so `pair` becomes `[2, 1]`. `replace` makes its copy of the reference point to a new array and changes that new array; the caller's `pair` is untouched.

Solution 1.9

← Exercise

Use two pointers from the two ends. Skip characters that are not letters or digits, and compare the rest case-insensitively.

Listing 1.37 Method — Palindrome.java

```
1  static boolean isPalindrome(String text) {
2      int left = 0;
3      int right = text.length() - 1;
4      while (left < right) {
5          char a = text.charAt(left);
6          char b = text.charAt(right);
7          if (!Character.isLetterOrDigit(a)) {
8              left++;                                  // skip punctuation and spaces
9          } else if (!Character.isLetterOrDigit(b)) {
10             right--;
11         } else if (Character.toLowerCase(a) != Character.toLowerCase(b)) {
12             return false;
13         } else {
14             left++;
15             right--;
16         }
17     }
18     return true;
19 }
```

Listing 1.38 Output — Palindrome

```
true
false
true
```

Each iteration moves at least one pointer inward, so the loop runs at most n times: O(n) time and O(1) extra space. A string with no letters or digits counts as a palindrome. The two-pointer pattern itself is the subject of Chapter 8.

Solution 1.10

← Exercise

Listing 1.39 Method — FactorialMod.java

```
static int factorialMod(int n) {
    long result = 1;                        // long: result * i can reach about 10^15
    for (int i = 2; i <= n; i++) {
        result = result * i % MOD;          // reduce after EVERY multiplication
    }
    return (int) result;                    // < MOD, so it fits in an int
}
```

Listing 1.40 Output — FactorialMod

```
n =         5: correct           120   buggy           120
n =        13: correct   227,020,758   buggy   932,053,497
n =        20: correct   146,326,063   buggy  -238,337,185
n = 1,000,000: correct   641,102,369   buggy  -739,369,273
```

`result * i % MOD` is evaluated as `(result * i) % MOD`. With `int` variables, the product of a value just below 10^9 and `i` overflows *before* `%` is applied, so the remainder is taken of an already wrong number. The correct version stores `result` as a `long`: the product is below about $10^9 \times 10^6 = 10^{15}$, which fits easily in a `long`, and the remainder is below `MOD`, so it fits back into an `int` at the end. The buggy output above (from `factorialModBuggy` in the same file) is already wrong at n = 13, because 13! is larger than the largest `int`.

Solution 1.11

← Exercise

Listing 1.41 Method — RunLength.java

```
static String encode(String text) {
    StringBuilder out = new StringBuilder();
    int i = 0;
    while (i < text.length()) {
        char current = text.charAt(i);
        int runEnd = i;
        while (runEnd < text.length() && text.charAt(runEnd) == current) {
            runEnd++;                           // extend the run of equal characters
        }
        out.append(current).append(runEnd - i);
        i = runEnd;                             // continue after the run
    }
    return out.toString();
}
```

Listing 1.42 Output — RunLength

```
a3b1c2
a1b1c1
[]
z12
```

Each character is examined once by the inner loop and the outer index jumps to the end of each run, so the method is O(n), and `StringBuilder` makes each append O(1) amortized.

With +=, the worst case is a string with no repeated neighbours, such as `"abab…"`, which produces n runs. Each += copies the whole result so far, whose length after k runs is at least 2k (a character and at least one digit). The total number of characters copied is at least 2 + 4 + … + 2n = n(n + 1), so the += version is $O(n^2)$.

Solution 1.12

← Exercise

Listing 1.43 Methods — MultiplyOverflow.java

```
/** For int: compute the exact product in a long and see whether it fits in an int. */
static boolean multiplyOverflows(int a, int b) {
    long exact = (long) a * b;              // any int * int fits in a long
```

```
    return exact < Integer.MIN_VALUE || exact > Integer.MAX_VALUE;
}

/** For long there is no wider type, so check by dividing the result back. */
static boolean multiplyOverflows(long a, long b) {
    if (a == 0 || b == 0) {
        return false;
    }
    if ((a == -1 && b == Long.MIN_VALUE) || (b == -1 && a == Long.MIN_VALUE)) {
        return true;                            // -MIN_VALUE does not exist in long
    }
    long product = a * b;                       // may have wrapped around
    return product / b != a;                    // wrapped results fail this test
}
```

Listing 1.44 Output MultiplyOverflow

```
false true
false true
true
```

For `int`, the exact product always fits in a `long` (it is at most 2^{62} in absolute value), so compute it there and compare with the `int` range. For `long`, multiply anyway and divide back: if no overflow happened, `product / b` equals `a`. If the product wrapped around, the division cannot give `a` back. One case escapes this test: `Long.MIN_VALUE * -1` wraps to `Long.MIN_VALUE`, and `Long.MIN_VALUE / -1` *also* overflows to `Long.MIN_VALUE`, so that pair is checked separately. The output shows the boundaries: $46{,}340^2$ fits in an `int` but $46{,}341^2$ does not, and $3{,}037{,}000{,}499^2$ fits in a `long` but $3{,}037{,}000{,}500^2$ does not. The self-check program compares both methods with `Math.multiplyExact` on thousands of pairs, including all the edge cases.

CHAPTER 2

I/O, Standard-Library Tools and Lambda Essentials

IN THIS CHAPTER

After this chapter you will be able to:

- read input quickly and correctly with `Scanner`, `BufferedReader` with `StringTokenizer`, and a hand-written fast reader, and explain why their speeds differ;
- write large outputs efficiently, flush them reliably, and format numbers in a way that does not depend on the computer's language settings;
- handle the standard input shapes: multiple test cases, reading until the end of the input, and character grids;
- use `BigInteger` and `BigDecimal` when `long` and `double` are not enough;
- use the most useful methods of `Arrays` and `Collections`;
- read and write lambdas and method references, and use them with `Map.merge` and `computeIfAbsent`;
- sort arrays and lists in any order with safe comparators;
- start every online assessment from a reusable, fully understood template.

Prerequisites: Chapter 1 (types, overflow, strings, arrays).

WARM-UP QUIZ

1. Which is faster for reading a million integers: `Scanner` or `BufferedReader`? Why?
2. Your program prints its answers with a `PrintWriter` but the judge sees no output. What did you most likely forget?
3. What does `(a, b) -> a - b` do wrong as a comparator?
4. What does `Arrays.binarySearch` return when the key is not in the array?
5. What is the difference between `Integer::compare` and `(x, y) -> Integer.compare(x, y)`?

Online judges give your program its input as text on *standard input* and read its answer from *standard output*. When the input has a million numbers, reading it the slow way can take longer than the whole time limit, even if your algorithm is perfect. This chapter starts with input and output, then covers the library tools you will use in almost every solution: big numbers, array and collection utilities, lambdas and comparators. It ends with a template that puts all of it together.

2.1 Reading input: Scanner and BufferedReader

2.1.1 Scanner

`java.util.Scanner` is the easiest way to read input. It splits the input into *tokens* (pieces separated by whitespace) and converts them for you.

Listing 2.1 Complete program — ScannerSum.java

```
1 import java.util.Scanner;
2
3 /** Reads n and then n integers with Scanner, and prints their sum. */
4 public class ScannerSum {
```

```
 5
 6     public static void main(String[] args) {
 7         Scanner in = new Scanner(System.in);
 8         int n = in.nextInt();
 9         long sum = 0;
10         for (int i = 0; i < n; i++) {
11             sum += in.nextInt();
12         }
13         System.out.println(sum);
14     }
15 }
```

Listing 2.2 Input — ScannerSum.in

```
5
3 -1 4 1 5
```

Listing 2.3 Output — ScannerSum

```
12
```

`new Scanner(System.in)` (line 7) wraps standard input. `nextInt()` skips any whitespace, including line breaks, reads the next token and converts it to an `int`, so it does not matter how the numbers are spread over lines. The sum is a `long` (line 9) because the sum of many `int` values can overflow an `int` (Chapter 1).

Why it is slow. The Scanner documentation describes it as a scanner that parses "using regular expressions". Every `nextInt` call matches the input against patterns, which is flexible and convenient but does much more work than the task needs. For a few thousand numbers this does not matter. For a million, it does.

> **WARNING · nextInt() followed by nextLine()**
> `nextInt()` reads the number but leaves the rest of the line, including the line break, unread. A following `nextLine()` returns that empty rest of the line, not the next line. If you mix the two, call `nextLine()` once more to skip the leftover line break, or read everything with `nextLine()` and parse the numbers yourself.

2.1.2 BufferedReader and StringTokenizer

`java.io.BufferedReader` reads characters in large blocks and hands out whole lines. You split each line into tokens and convert them yourself.

Listing 2.4 Complete program — BufferedSum.java

```
 1 import java.io.BufferedReader;
 2 import java.io.IOException;
 3 import java.io.InputStreamReader;
 4 import java.util.StringTokenizer;
 5
 6 /** The same task as ScannerSum, with BufferedReader and StringTokenizer. */
 7 public class BufferedSum {
 8
 9     public static void main(String[] args) throws IOException {
10         BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
11         int n = Integer.parseInt(reader.readLine().trim());
12         StringTokenizer tokens = new StringTokenizer(reader.readLine());
13         long sum = 0;
14         for (int i = 0; i < n; i++) {
15             sum += Integer.parseInt(tokens.nextToken());
16         }
17         System.out.println(sum);
18     }
19 }
```

Listing 2.5 Output — BufferedSum

```
12
```

Syntax breakdown

- `new BufferedReader(new InputStreamReader(System.in))` (line 10) builds a chain: `System.in` delivers raw bytes, `InputStreamReader` decodes them into characters, and `BufferedReader` adds a buffer and the `readLine()` method.
- `throws IOException` (line 9) is required because `readLine` declares that checked exception (Chapter 4). In contest code, declaring it on `main` is the usual choice.
- `reader.readLine()` (line 11) returns the next line without its line break, or `null` at the end of the input. `trim()` removes stray spaces, which `Integer.parseInt` would reject.
- `new StringTokenizer(line)` (line 12) splits the line at whitespace; `nextToken()` (line 15) returns the tokens one by one.

`StringTokenizer` is officially a *legacy* class: its documentation discourages it in new code and recommends `String.split` instead. In competitive programming it is still widely used because it does not use regular expressions and is fast. `split("\\s+")` is fine when speed does not matter.

This version assumes that all n numbers are on the second line. Many inputs spread numbers over several lines; Exercise 2.1 builds a `next()` method that reads more lines whenever the current one runs out of tokens.

2.1.3 How much faster?

The program `IoBenchmark.java` in `code-examples/ch02` generates one million random integers as text, ten per line, and reads them three times: with `Scanner`, with `BufferedReader` and `StringTokenizer`, and with the fast reader of the next section. It then prints a million numbers in three ways. When this book was built, it printed:

Listing 2.6 Output — IoBenchmark

```
Scanner                      294 ms  (1.00 of the first)
BufferedReader+Tokenizer      51 ms  (0.18 of the first)
FastReader                    13 ms  (0.05 of the first)
(all three read the same sum: 1048757670)
println, autoflush stream     47 ms  (1.00 of the first)
PrintWriter, one flush        32 ms  (0.68 of the first)
StringBuilder, one write      15 ms  (0.32 of the first)
```

The exact numbers depend on the computer and change from run to run; compare the ratios. `BufferedReader` with `StringTokenizer` was several times faster than `Scanner`, and the fast reader was faster still. All three computed the same sum, which the program checks. The output part prints to a "null" stream that discards everything, so it measures only the cost inside Java; with real output to a terminal or file, flushing after every line costs even more, because each flush is a separate request to the operating system.

2.2 A fast reader

When input is large and contains only numbers and simple words, you can do better than any general-purpose class by reading raw bytes and converting digits yourself. This is the fast reader pattern used by many competitive programmers.

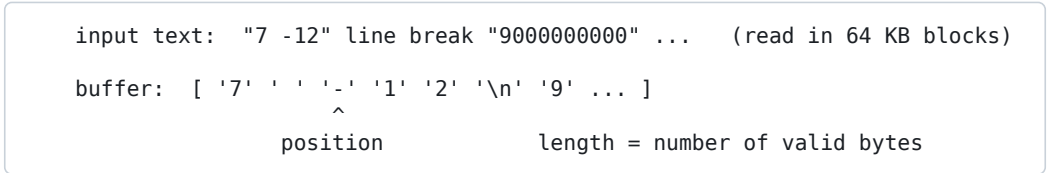


Figure 2.1: The fast reader keeps a block of input bytes and an index into it

Listing 2.7 Fields and methods — FastReader.java

```
1 private final InputStream in;
2 private final byte[] buffer;
3 private int length;    // how many bytes the buffer currently holds
4 private int position;  // index of the next unread byte
5
```

```
public FastReader(InputStream in) {
    this(in, 1 << 16);  // 64 KB
}

FastReader(InputStream in, int bufferSize) {
    this.in = in;
    this.buffer = new byte[bufferSize];
}

/** Makes sure at least one unread byte is buffered; returns false at end of input. */
private boolean fill() throws IOException {
    if (position < length) {
        return true;
    }
    length = in.read(buffer, 0, buffer.length);
    position = 0;
    if (length <= 0) {
        length = 0;
        return false;
    }
    return true;
}

/** True if another token follows; skips the whitespace before it. */
public boolean hasNext() throws IOException {
    while (fill()) {
        if ((buffer[position] & 0xFF) > ' ') {
            return true;
        }
        position++;
    }
    return false;
}

public long nextLong() throws IOException {
    if (!hasNext()) {
        throw new EOFException("no more input");
    }
    boolean negative = buffer[position] == '-';
    if (negative) {
        position++;
    }
    long value = 0;
    while (fill() && buffer[position] >= '0' && buffer[position] <= '9') {
        value = value * 10 + (buffer[position] - '0');
        position++;
    }
    return negative ? -value : value;
}

public int nextInt() throws IOException {
    return (int) nextLong();  // assumes the input respects the int range
}

public String next() throws IOException {
    if (!hasNext()) {
        throw new EOFException("no more input");
    }
    StringBuilder token = new StringBuilder();
    while (fill() && (buffer[position] & 0xFF) > ' ') {
        token.append((char) buffer[position]);
        position++;
    }
    return token.toString();
}
```

Listing 2.8 Input `FastReader.in`

```
7 -12
9000000000
  3

-4 0
100
```

Listing 2.9 Output `FastReader`

```
7 numbers, sum 9000000094
```

Line by line

- Lines 1–4: the reader holds the underlying `InputStream`, a byte array `buffer`, the number of valid bytes in it (`length`) and the index of the next unread byte (`position`).
- Lines 6–13: the public constructor uses a 64 KB buffer (`1 << 16` is 2^{16} = 65,536). A second, package-private constructor lets the self-check program use tiny buffers, to prove that numbers split across two blocks are still read correctly.
- Lines 16–27, `fill`: if unread bytes remain, there is nothing to do. Otherwise `in.read(buffer, 0, buffer.length)` reads up to a whole block and returns how many bytes it read, or −1 at the end of the input. The method returns whether at least one unread byte is now available.
- Lines 30–38, `hasNext`: skips whitespace. Every character with a code of 32 (`' '`) or less is treated as whitespace: spaces, tabs, `\n` and `\r`. `buffer[position] & 0xFF` converts the `byte`, which Java treats as signed (−128 to 127), to a value from 0 to 255, so that non-ASCII bytes are not mistaken for whitespace.
- Lines 40–54, `nextLong`: after skipping whitespace, an optional minus sign is consumed (lines 44–47). Then each digit is added to the value: `value * 10 + (digit - '0')` (lines 49–52). The loop calls `fill()` before every byte, so a number that is cut in two by a block boundary is still read correctly.
- Line 57, `nextInt`: converts the `long` to an `int`. If the input breaks its stated constraints, the value is silently truncated; judges guarantee the constraints, so this is acceptable in contest code.
- Lines 60–70, `next`: reads a word, one byte per character.

Dry run of `nextLong` on the bytes `-12` followed by a line break:

Table 2.1: Parsing "−12" byte by byte

Step	Byte at `position`	Action	`value`
1	`-`	set `negative = true`, advance	0
2	`1`	`value = 0 × 10 + 1`	1
3	`2`	`value = 1 × 10 + 2`	12
4	`\n`	not a digit: loop ends	12
end		return `-value`	−12

Complexity. Every input byte is examined a constant number of times, so reading the whole input costs O(total input size), with O(1) extra memory beyond the fixed buffer.

Limitations. The reader treats every byte as one character, which is correct for ASCII input, the usual case for numeric problems. For text in Bengali or other non-ASCII scripts, use `BufferedReader`, which decodes characters properly. It also does not check for malformed numbers or overflow. If you are allowed to choose, `BufferedReader` with `StringTokenizer` is fast enough for almost every problem; a byte-level reader is the safe choice when the input has millions of tokens.

2.3 Writing output fast

Output has the same problem as input. `System.out.println` is convenient, but every call goes through a synchronized `PrintStream`, and the stream may flush its data to the operating system often. The standard fixes:

1. **Collect the output in a `StringBuilder`** and print it once at the end. This is simple and fast, but it holds the whole output in memory.

2. **Use a `PrintWriter` over a `BufferedOutputStream`** and call `flush()` once at the end. Output is written in large blocks while the program runs, so memory stays small.

> **WARNING · Always flush**
> A `PrintWriter` created as `new PrintWriter(new BufferedOutputStream(System.out))` keeps its data in memory until its buffer fills up or you call `flush()` (or `close()`). If the program ends without flushing, the last part of the output, possibly *all* of it, is never written, and the judge reports "wrong answer" for a correct program. Put `out.flush()` at the end of `main`, and make sure no `return` statement skips it.

2.3.1 Formatting numbers, and the locale trap

`System.out.printf` and `String.format` format numbers: `%d` for integers, `%.2f` for a `double` with two decimal places, `%,d` with thousands separators, and `%n` for a line break. These methods follow a *locale*: a setting that describes a language and region's conventions for writing numbers and dates. By default they use the locale of the computer the program runs on.

Listing 2.10 Method — LocaleTrap.java

```java
Locale bangla = Locale.forLanguageTag("bn-BD");
String pattern = "%.2f  %,d";
System.out.println("bn-BD   " + String.format(bangla, pattern, average, big));
Locale german = Locale.GERMANY;
System.out.println("de-DE   " + String.format(german, pattern, average, big));
Locale neutral = Locale.ROOT;
System.out.println("ROOT    " + String.format(neutral, pattern, average, big));
```

Listing 2.11 Output — LocaleTrap

```
bn-BD   ৩.৫০  ১,২৩৪,৫৬৭
de-DE   3,50  1.234.567
ROOT    3.50  1,234,567
```

The same format prints **Bengali digits** under the Bangladeshi locale `bn-BD`, and a decimal *comma* under the German locale. On a judge, the default locale is usually US English, so this rarely fails there. On your own computer, if Windows is set to Bangla (Bangladesh), your local tests can print `৩.৫০` where the expected output says `3.50`. Pass `Locale.ROOT` (a neutral locale) whenever you format numbers for a judge, a file or another program: `String.format(Locale.ROOT, "%.2f", x)`.

`StringBuilder.append(double)` and `String.valueOf(double)` do not depend on the locale, but they print values such as `0.30000000000000004` or `1.0E-4` instead of a fixed number of decimals. When a problem says "print the answer with exactly 6 digits after the decimal point", use `String.format(Locale.ROOT, "%.6f", value)`.

2.4 Input patterns

2.4.1 Multiple test cases

Many problems begin with a number T, followed by T independent test cases. The template's `main` method reads T and calls `solve` once per test case:

Listing 2.12 Method — OaTemplate.java

```java
public static void main(String[] args) throws IOException {
    FastIn in = new FastIn(System.in);
    PrintWriter out = new PrintWriter(new BufferedOutputStream(System.out));
    int tests = in.nextInt();
    for (int t = 1; t <= tests; t++) {
        solve(in, out);
    }
    out.flush();  // without this, buffered output may never appear
}
```

Listing 2.13 Method — OaTemplate.java

```
static void solve(FastIn in, PrintWriter out) throws IOException {
    int n = in.nextInt();
    long min = Long.MAX_VALUE;
    long max = Long.MIN_VALUE;
    long sum = 0;
    for (int i = 0; i < n; i++) {
        long value = in.nextLong();
        min = Math.min(min, value);
        max = Math.max(max, value);
        sum += value;
    }
    out.println(min + " " + max + " " + sum);
}
```

Listing 2.14 Input — OaTemplate.in

```
3
4
5 -2 9 1
1
42
6
-7 -7 3 3 0 8
```

Listing 2.15 Output — OaTemplate

```
-2 9 13
42 42 42
-7 8 0
```

Keeping each test case in its own `solve` call has a practical benefit: every variable is fresh for each test, so nothing leaks from one test into the next. If you keep arrays or counters in `static` fields to reuse memory, reset them at the start of `solve`. The complete template, including its reader `FastIn`, is explained in Section 2.9.

2.4.2 Reading until the end of the input

Some inputs do not say how many lines follow. Read until `readLine()` returns `null`:

Listing 2.16 Complete program — ReadUntilEof.java

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.Locale;

/** Reads "name score" lines until the input ends; the number of lines is not given. */
public class ReadUntilEof {

    public static void main(String[] args) throws IOException {
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder out = new StringBuilder();
        int count = 0;
        long total = 0;
        String line;
        while ((line = reader.readLine()) != null) {  // null means end of input
            line = line.strip();
            if (line.isEmpty()) {
                continue;                             // tolerate blank lines
            }
            String[] parts = line.split("\\s+");
            String name = parts[0];
            int score = Integer.parseInt(parts[1]);
            count++;
            total += score;
            out.append(name).append(": ").append(score).append('\n');
        }
```

```
27          out.append("students: ").append(count)
28                  .append(", average: ")
29                  .append(String.format(Locale.ROOT, "%.2f", (double) total / count));
30          System.out.println(out);
31      }
32  }
```

Listing 2.17 Input — ReadUntilEof.in

```
rahim 78
karim 91

sadia 85
nadia 64
```

Listing 2.18 Output — ReadUntilEof

```
rahim: 78
karim: 91
sadia: 85
nadia: 64
students: 4, average: 79.50
```

The condition on line 15 does two things at once: `(line = reader.readLine())` assigns the next line to `line`, and the value of the whole assignment expression is then compared with `null`. Blank lines are skipped (lines 17–19), which makes the program robust against an extra empty line at the end of the input, a common difference between local test files and judge input. When typing input by hand in a terminal, end it with Ctrl+Z then Enter on Windows, or Ctrl+D on Linux and macOS.

2.4.3 Reading a grid

Grid problems give the size and then one line per row:

Listing 2.19 Complete program — GridInput.java

```
1  import java.io.BufferedReader;
2  import java.io.IOException;
3  import java.io.InputStreamReader;
4  import java.util.StringTokenizer;
5
6  /** Reads a grid of characters: a line "rows cols", then one line per row. */
7  public class GridInput {
8
9      public static void main(String[] args) throws IOException {
10         BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
11         StringTokenizer first = new StringTokenizer(reader.readLine());
12         int rows = Integer.parseInt(first.nextToken());
13         int cols = Integer.parseInt(first.nextToken());
14         char[][] grid = new char[rows][];
15         for (int r = 0; r < rows; r++) {
16             grid[r] = reader.readLine().toCharArray();  // each row as a char array
17         }
18
19         int walls = 0;
20         int startRow = -1;
21         int startCol = -1;
22         for (int r = 0; r < rows; r++) {
23             for (int c = 0; c < cols; c++) {
24                 if (grid[r][c] == '#') {
25                     walls++;
26                 } else if (grid[r][c] == 'S') {
27                     startRow = r;
28                     startCol = c;
29                 }
30             }
31         }
32         System.out.println(rows + "x" + cols + " grid, " + walls + " walls, start at ("
```

```
33                 + startRow + ", " + startCol + ")");
34     }
35 }
```

Listing 2.20 Input GridInput.in

```
4 6
#.....
#.##.#
..S#..
######
```

Listing 2.21 Output GridInput

```
4x6 grid, 12 walls, start at (2, 2)
```

`readLine().toCharArray()` (line 16) turns each row into a `char[]`, so `grid[r][c]` is the cell in row `r` and column `c`. Remember the order: **row first, then column**, which is (y, x) rather than (x, y). Most grid bugs come from swapping the two.

2.4.4 Common input and output mistakes

Table 2.2: Common input/output errors and their fixes

Symptom	Likely cause	Fix
Compilation error on the judge	class not named `Main`, or a `package` line	follow the platform's naming rule; remove `package`
"Time limit exceeded" with a fast algorithm	`Scanner`, or `println` per line on large I/O	`BufferedReader` or a fast reader; `PrintWriter` or `StringBuilder`
No output at all	`PrintWriter` never flushed	`out.flush()` at the end of `main`
`NumberFormatException`	spaces around a number, or reading a whole line as a number	`trim()`, or tokenize first
`NullPointerException` from `readLine()`	fewer lines than expected; or the next number is on the same line	read tokens, not lines, with a `next()` helper
Wrong answer only on your computer	locale-dependent formatting	`Locale.ROOT` in `String.format`
Wrong answer with "correct" numbers	wrong output format: missing case labels, extra spaces, wrong precision	copy the format from the problem's sample output
Wrong answer on large tests only	overflow in a sum or product	`long`, cast before multiplying (Chapter 1)

2.5 Big numbers: BigInteger and BigDecimal

Chapter 1's table showed when `int`, `long` and `double` are enough. When they are not, Java offers two classes in `java.math`.

> **DEFINITION · BigInteger and BigDecimal**
> BigInteger represents integers of any size, limited only by memory. BigDecimal represents decimal numbers exactly, as an integer value plus a *scale* (the number of digits after the decimal point). Both are immutable: every operation returns a new object.

Listing 2.22 Fragment of main — BigNumbers.java

```
1  BigInteger factorial = BigInteger.ONE;
2  for (int i = 2; i <= 30; i++) {
3      factorial = factorial.multiply(BigInteger.valueOf(i));  // a NEW object
4  }
5  System.out.println("30! = " + factorial);
6  BigInteger twoTo100 = BigInteger.TWO.pow(100);
7  System.out.println("2^100 = " + twoTo100 + " (" + twoTo100.toString().length()
8          + " digits)");
9  BigInteger mod = BigInteger.valueOf(1_000_000_007);
10 System.out.println("2^100 mod 1e9+7 = " + twoTo100.mod(mod)
11         + ", via modPow: " + BigInteger.TWO.modPow(BigInteger.valueOf(100), mod));
12 System.out.println("compareTo: " + factorial.compareTo(twoTo100)
13         + ", equals parsed 30!: "
14         + factorial.equals(new BigInteger("265252859812191058636308480000000")));
```

Listing 2.23 Fragment of main — BigNumbers.java

```
1  BigDecimal a = new BigDecimal("0.1");      // from a String: exactly 0.1
2  BigDecimal b = new BigDecimal("0.2");
3  System.out.println("0.1 + 0.2 = " + a.add(b));
4  System.out.println("new BigDecimal(0.1) = " + new BigDecimal(0.1));  // the double
5  BigDecimal two = new BigDecimal("2.0");
6  BigDecimal twoPointZeroZero = new BigDecimal("2.00");
7  System.out.println("2.0 equals 2.00? " + two.equals(twoPointZeroZero)
8          + ", compareTo: " + two.compareTo(twoPointZeroZero));
9  BigDecimal three = new BigDecimal("3");
10 BigDecimal third = BigDecimal.ONE.divide(three, 10, RoundingMode.HALF_UP);
11 System.out.println("1/3 to 10 places = " + third);
```

Listing 2.24 Output — BigNumbers

```
30! = 265252859812191058636308480000000
2^100 = 1267650600228229401496703205376 (31 digits)
2^100 mod 1e9+7 = 976371285, via modPow: 976371285
compareTo: 1, equals parsed 30!: true
0.1 + 0.2 = 0.3
new BigDecimal(0.1) = 0.1000000000000000055511151231257827021181583404541015625
2.0 equals 2.00? false, compareTo: 0
1/3 to 10 places = 0.3333333333
```

What to notice

- Arithmetic uses methods, not operators: `add`, `subtract`, `multiply`, `divide`, `mod`, `pow`, `modPow`. Because the objects are immutable, you must keep the result: `factorial = factorial.multiply(...)` (line 3 of the first fragment). Writing `factorial.multiply(...)` alone computes a value and throws it away.
- Compare with `compareTo` (negative, zero or positive), not with < or ==.
- `BigInteger.valueOf(long)` converts small values; `new BigInteger("…")` parses text of any length.
- `new BigDecimal("0.1")` is exactly 0.1, but `new BigDecimal(0.1)` is the exact value of the *double* closest to 0.1, a long string of digits. Always create `BigDecimal` values from strings (or with `BigDecimal.valueOf(double)`, which goes through the shortest string representation).
- `BigDecimal.equals` compares the scale too, so `2.0` and `2.00` are *not* equal, while `compareTo` says they are the same number. Use `compareTo` to compare values.
- Division that does not terminate, such as 1/3, throws `ArithmeticException` unless you give a scale and a `RoundingMode`.

Cost. Operations on big numbers take time that grows with the number of digits: addition is linear in the number of digits, and multiplication is slower than that. They are also objects, so arrays of them use a lot of memory. In contest problems, a result "modulo 1,000,000,007" is a strong hint that the intended solution uses `long` with modular arithmetic (Chapter 19), not `BigInteger`.

2.6 Arrays and Collections utilities

`java.util.Arrays` works on arrays; `java.util.Collections` works on lists and other collections. These are the methods you will use most often.

Listing 2.25 Fragment of main — LibraryUtilities.java

```
int[] values = {40, 10, 30, 20, 50};
Arrays.sort(values);                                  // ascending, in place
System.out.println("sorted       " + Arrays.toString(values));
System.out.println("search 30    " + Arrays.binarySearch(values, 30));  // index
System.out.println("search 35    " + Arrays.binarySearch(values, 35));  // -3 - 1
int[] part = {9, 7, 5, 3, 1};
Arrays.sort(part, 1, 4);                              // only indices 1..3
System.out.println("range sort   " + Arrays.toString(part));
int[] filled = new int[5];
Arrays.fill(filled, -1);
System.out.println("fill         " + Arrays.toString(filled));
```

Listing 2.26 Fragment of main — LibraryUtilities.java

```
String[] words = {"pear", "apple", "fig"};
List<String> view = Arrays.asList(words);             // fixed size, array-backed
view.set(0, "plum");                                  // allowed: writes through
System.out.println("asList view  " + view + ", array " + Arrays.toString(words));
try {
    view.add("kiwi");                                 // not allowed: fixed size
} catch (UnsupportedOperationException e) {
    System.out.println("asList add   UnsupportedOperationException");
}
List<String> copy = new ArrayList<>(Arrays.asList(words));  // growable copy
copy.add("kiwi");
System.out.println("copy         " + copy);
```

Listing 2.27 Fragment of main — LibraryUtilities.java

```
List<Integer> numbers = new ArrayList<>(List.of(5, 3, 8, 3, 1));
Collections.sort(numbers);
System.out.println("sort         " + numbers);
Collections.reverse(numbers);
System.out.println("reverse      " + numbers);
System.out.println("max, min     " + Collections.max(numbers) + ", "
        + Collections.min(numbers));
System.out.println("frequency 3  " + Collections.frequency(numbers, 3));
Collections.swap(numbers, 0, numbers.size() - 1);
System.out.println("swap ends    " + numbers);
System.out.println("nCopies      " + Collections.nCopies(3, "ab"));
List<Integer> fixed = List.of(1, 2, 3);               // immutable
try {
    fixed.set(0, 9);
} catch (UnsupportedOperationException e) {
    System.out.println("List.of set  UnsupportedOperationException");
}
```

Listing 2.28 Output — LibraryUtilities

```
sorted       [10, 20, 30, 40, 50]
search 30    2
search 35    -4
range sort   [9, 3, 5, 7, 1]
fill         [-1, -1, -1, -1, -1]
asList view  [plum, apple, fig], array [plum, apple, fig]
asList add   UnsupportedOperationException
copy         [plum, apple, fig, kiwi]
sort         [1, 3, 3, 5, 8]
reverse      [8, 5, 3, 3, 1]
max, min     8, 1
frequency 3  2
```

```
swap ends    [1, 5, 3, 3, 8]
nCopies      [ab, ab, ab]
List.of set  UnsupportedOperationException
```

Details that matter

- `Arrays.binarySearch` requires a *sorted* array. If the key is present, it returns its index (if it appears several times, any one of them). If not, it returns `-(insertion point) - 1`, where the insertion point is the index of the first element greater than the key. For 35 in `[10, 20, 30, 40, 50]`, the insertion point is 3, so the result is −4. The formula guarantees a negative result whenever the key is missing, even when the insertion point is 0. Chapter 12 builds binary search from scratch.
- `Arrays.sort(part, 1, 4)` sorts the half-open range from index 1 up to, but not including, index 4.
- `Arrays.asList(array)` returns a *fixed-size* list that is a view of the array: setting an element writes through to the array (the output shows `plum` in both), and adding or removing throws `UnsupportedOperationException`. For a normal list, copy it: `new ArrayList<>(Arrays.asList(array))`. `Arrays.asList` on an `int[]` gives a `List<int[]>` with one element, not a list of numbers.
- `List.of(...)` returns an *unmodifiable* list, and it rejects `null` elements. It is perfect for constants and test data; copy it into an `ArrayList` when you need to change it.

Table 2.3: Frequently used Arrays and Collections methods

Method	What it does	Cost
`Arrays.sort(a)`, `Arrays.sort(a, from, to)`	sort ascending, in place	O(n log n)
`Arrays.binarySearch(a, key)`	search a sorted array	O(log n)
`Arrays.fill(a, value)`	set every element	O(n)
`Arrays.copyOf`, `copyOfRange`, `equals`, `toString`	see Chapter 1	O(n)
`Collections.sort(list)`, `list.sort(cmp)`	stable sort of a list	O(n log n)
`Collections.reverse`, `max`, `min`, `frequency`	reverse, extremes, count	O(n)
`Collections.swap(list, i, j)`	swap two elements	O(1) for `ArrayList`
`Collections.nCopies(n, x)`	an unmodifiable list of n copies	O(1) to create

2.7 Lambda and method-reference essentials

Lambdas appear in DSA code from the first chapter of problems onwards: in every comparator, in every `computeIfAbsent`, in `PriorityQueue` constructors. This section teaches what you need now; Chapter 22 covers functional programming and streams in depth.

> **DEFINITION · Functional interface and lambda**
> A functional interface is an interface with exactly one abstract method, such as `Comparator<T>` (method `compare`) or `Runnable` (method `run`). A lambda expression is a short way to create an object that implements a functional interface: you write only the parameters and the body of that one method.

Why lambdas exist. Before Java 8, sorting by length needed an anonymous class with five lines of ceremony around one line of logic. A lambda keeps only the logic.

Listing 2.29 Fragment of main LambdaBasics.java

```
1  Comparator<String> byLength = (String a, String b) -> Integer.compare(a.length(),
2          b.length());                                          // full form
3  Comparator<String> byLengthShort =
4          (a, b) -> Integer.compare(a.length(), b.length());   // types inferred
5  Runnable hello = () -> System.out.println("hello from a Runnable");
6  BinaryOperator<Integer> add = (x, y) -> {             // a block body needs return
7      int sum = x + y;
8      return sum;
9  };
10 hello.run();
11 System.out.println(byLength.compare("fig", "pear") + " "
12         + byLengthShort.compare("pear", "fig") + " " + add.apply(2, 3));
```

Syntax breakdown

- `(String a, String b) -> Integer.compare(a.length(), b.length())` (lines 1–2) is the full form: a parameter list, the arrow `->`, and an expression whose value is returned.
- `(a, b) -> ...` (lines 3–4) leaves out the parameter types. The compiler infers them from the *target type*: the variable is a `Comparator<String>`, whose `compare` method takes two `String`s. A lambda has no type of its own; it always takes the type of the functional interface it is assigned or passed to.
- `() -> System.out.println(...)` (line 5) has no parameters. With exactly one parameter, the parentheses are optional: `x -> x * 2`.
- A body in braces (lines 6–9) can contain several statements and must use `return` to return a value.

The variable `add` has type `BinaryOperator<Integer>`, a functional interface from `java.util.function` that takes two values of a type and returns one of the same type. `apply` (line 12) calls it.

2.7.1 Method references

When a lambda only calls one existing method, a method reference is shorter:

Listing 2.30 Fragment of main LambdaBasics.java

```
1  Comparator<Integer> natural = Integer::compare;        // static method
2  Comparator<String> byLengthRef = Comparator.comparingInt(String::length);
3  Supplier<List<String>> newList = ArrayList::new;       // constructor
4  List<String> words = newList.get();
5  words.add("banana");
6  words.add("fig");
7  words.add("apple");
8  words.sort(byLengthRef);
9  System.out.println(natural.compare(7, 3) + " " + words);
```

Table 2.4: The four kinds of method reference

Kind	Example	Equivalent lambda
static method	`Integer::compare`	`(x, y) -> Integer.compare(x, y)`
instance method of a parameter	`String::length`	`s -> s.length()`
instance method of a given object	`System.out::println`	`x -> System.out.println(x)`
constructor	`ArrayList::new`	`() -> new ArrayList<>()`

`Comparator.comparingInt(String::length)` (line 2) builds a comparator from a function that extracts an `int` key: here, the length of each string. `Supplier<List<String>>` (line 3) is a functional interface whose method `get()` takes nothing and returns a value; `ArrayList::new` supplies a new list each time.

2.7.2 Lambdas with maps

Two `Map` methods take lambdas and replace several lines of everyday code:

Listing 2.31 Fragment of main LambdaBasics.java

```
1  String text = "to be or not to be is the question";
2  Map<String, Integer> counts = new HashMap<>();
3  for (String word : text.split(" ")) {
4      counts.merge(word, 1, Integer::sum);              // add 1, or start at 1
5  }
6  Map<Integer, List<String>> byLen = new TreeMap<>();
7  for (String word : text.split(" ")) {
8      byLen.computeIfAbsent(word.length(), k -> new ArrayList<>()).add(word);
9  }
10 System.out.println(new TreeMap<>(counts));
11 System.out.println(byLen);
```

- `counts.merge(word, 1, Integer::sum)` (line 4) means: if `word` is not in the map, put 1; otherwise replace the old value with `Integer.sum(old, 1)`. It is the shortest correct way to count.
- `byLen.computeIfAbsent(key, k -> new ArrayList<>())` (line 8) returns the list stored under `key`, first creating and storing an empty one if there is none. Chaining `.add(word)` then adds to that list. This is the standard way to build a "map of lists". The lambda runs only when the key is missing.

2.7.3 Capturing local variables

Listing 2.32 Fragment of main LambdaBasics.java

```
1  int limit = 3;                                       // effectively final
2  Runnable show = () -> System.out.println("limit inside the lambda: " + limit);
3  show.run();
4  // limit = 4;   // would make the lambda above a compile error
```

A lambda may read local variables of the enclosing method only if they are *effectively final*: never reassigned after initialization. If line 4 were not a comment, the compiler would reject the lambda on line 2. The usual workaround in contest code is to copy the value into a new final variable just before the lambda, or to use a one-element array or a field when the lambda must update a value.

Listing 2.33 Output LambdaBasics

```
hello from a Runnable
-1 1 5
1 [fig, apple, banana]
{be=2, is=1, not=1, or=1, question=1, the=1, to=2}
{2=[to, be, or, to, be, is], 3=[not, the], 8=[question]}
limit inside the lambda: 3
```

2.8 Sorting with built-in APIs and comparators

2.8.1 What Java guarantees

Table 2.5: What the Java SE 21 documentation says about sorting

Call	Algorithm (documentation)	Stable?	Time
`Arrays.sort(int[])` and other primitives	implementation note: Dual-Pivot Quicksort, O(n log n) on all data sets	not applicable	O(n log n)
`Arrays.sort(Object[])`, `Arrays.sort(T[], Comparator)`	any algorithm, but stability is required	yes (guaranteed)	O(n log n)
`Collections.sort(list)`, `list.sort(cmp)`	delegates to `List.sort`; stability required	yes (guaranteed)	O(n log n)

DEFINITION · Stable sort

A sort is stable if elements that compare as equal keep their original relative order. Stability matters when you sort by one key after another, for example first by name and then, stably, by score.

For primitive arrays, "stable" has no meaning, because equal numbers are indistinguishable. Chapter 12 explains the algorithms behind these guarantees.

2.8.2 Comparators

> **DEFINITION · Comparator**
> A Comparator <T> is a functional interface whose method `compare(a, b)` returns a negative number if `a` should come before `b`, zero if they are equal in this order, and a positive number if `a` should come after `b`.

Listing 2.34 Fragment of main — SortingWithComparators.java

```
1  Integer[] boxed = {5, 1, 4, 2, 3};
2  Arrays.sort(boxed, Collections.reverseOrder());        // only for object arrays
3  System.out.println("boxed desc   " + Arrays.toString(boxed));
4
5  int[] primitive = {5, 1, 4, 2, 3};
6  Arrays.sort(primitive);                                // ascending...
7  for (int i = 0, j = primitive.length - 1; i < j; i++, j--) {
8      int temp = primitive[i];                           // ...then reverse in place
9      primitive[i] = primitive[j];
10     primitive[j] = temp;
11 }
12 System.out.println("int[] desc   " + Arrays.toString(primitive));
```

Descending order. `Collections.reverseOrder()` (line 2) is a comparator that reverses the natural order, but comparators only work with *objects*, so it needs an `Integer[]`. For an `int[]`, there is no `Arrays.sort(int[], Comparator)`. Either box the values, which costs memory and time (Exercise 2.6), or sort ascending and reverse the array in place (lines 6–11), which is the usual choice.

Listing 2.35 Fragment of main — SortingWithComparators.java

```
1  int[][] intervals = {{5, 8}, {1, 9}, {1, 3}, {2, 6}};
2  Arrays.sort(intervals, Comparator.comparingInt((int[] a) -> a[0])
3          .thenComparingInt(a -> a[1]));                 // by start, then by end
4  System.out.println("intervals    " + Arrays.deepToString(intervals));
```

Sorting arrays of arrays. `int[][] intervals` is an array of `int[]` objects, so it can be sorted with a comparator. `Comparator.comparingInt((int[] a) -> a[0])` orders by the first element, and `.thenComparingInt(a -> a[1])` breaks ties with the second. Notice the explicit type `(int[] a)` in the first lambda: when a method call is chained after it, the compiler cannot infer the parameter type from the target, so it needs the hint. Without the chain, `Comparator.comparingInt(a -> a[0])` would compile.

Listing 2.36 Fragment of main — SortingWithComparators.java

```
1  List<Student> students = new ArrayList<>(List.of(
2          new Student("Rafi", 82), new Student("Anika", 95),
3          new Student("Tanvir", 82), new Student("Mim", 70)));
4  students.sort(Comparator.comparingInt(Student::score).reversed()
5          .thenComparing(Student::name));                // score desc, then name
6  StringBuilder shown = new StringBuilder("students    ");
7  for (Student s : students) {
8      shown.append(' ').append(s.name()).append('=').append(s.score());
9  }
10 System.out.println(shown);
```

Sorting objects by several keys. `Student` is a *record* declared in the same file: `record Student(String name, int score) { }`. A record is a compact way to declare a class whose fields never change; it automatically gets a constructor, accessor methods `name()` and `score()`, and `equals`, `hashCode` and `toString` (Chapter 3). `Comparator.comparingInt(Student::score).reversed()` sorts by score, highest first, and `.thenComparing(Student::name)` puts students with equal scores in alphabetical order.

Listing 2.37 Fragment of main — SortingWithComparators.java

```
Integer[] extremes = {Integer.MIN_VALUE, 1, Integer.MAX_VALUE, 0};
Integer[] wrong = extremes.clone();
Arrays.sort(wrong, (a, b) -> a - b);                    // subtraction can overflow
Integer[] right = extremes.clone();
Arrays.sort(right, Integer::compare);
System.out.println("a - b        " + Arrays.toString(wrong));
System.out.println("compare      " + Arrays.toString(right));
```

Listing 2.38 Output — SortingWithComparators

```
boxed desc   [5, 4, 3, 2, 1]
int[] desc   [5, 4, 3, 2, 1]
intervals    [[1, 3], [1, 9], [2, 6], [5, 8]]
students     Anika=95 Rafi=82 Tanvir=82 Mim=70
a - b        [0, 1, 2147483647, -2147483648]
compare      [-2147483648, 0, 1, 2147483647]
```

WARNING · Never compare by subtraction

`(a, b) -> a - b` looks like a neat comparator, and it works in every small test. But `a - b` overflows when the values are far apart: here `Integer.MIN_VALUE - 1` wraps around to a large *positive* number, so the comparator claims that the smallest `int` is larger than 1. The result is the scrambled order in the output, and with longer arrays, Java's sort may even throw `IllegalArgumentException: Comparison method violates its general contract!`. Always use `Integer.compare(a, b)`, `Long.compare`, `Comparator.comparingInt`, or `Comparator.reverseOrder()`.

Table 2.6: Comparator recipes

Need	Comparator
natural order / reversed	`Comparator.naturalOrder()` / `Comparator.reverseOrder()`
by an `int` key	`Comparator.comparingInt(x -> key)`
by a key, descending	`Comparator.comparingInt(...).reversed()`
tie-breaker	`.thenComparing(...)`, `.thenComparingInt(...)`
`int[]` by element 0, then element 1	`Comparator.comparingInt((int[] a) -> a[0]).thenComparingInt(a -> a[1])`
`long` values	`Long::compare` or `Comparator.comparingLong(...)`
`null` allowed	`Comparator.nullsFirst(...)` or `nullsLast(...)`

2.9 A reusable template

Every online assessment should start from a template you understand completely. Here is the full template from this chapter. It needs only Java 8 features, so it compiles on any judge that offers Java 8 or later.

Listing 2.39 Complete program — OaTemplate.java

```
import java.io.BufferedOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.PrintWriter;

/**
 * A reusable online-assessment template (Java 8 or later). Rename the class to what the
 * judge requires, usually Main, and replace solve().
 * Example problem: for each test case, read n and n integers; print min, max and sum.
 */
public class OaTemplate {

    public static void main(String[] args) throws IOException {
        FastIn in = new FastIn(System.in);
        PrintWriter out = new PrintWriter(new BufferedOutputStream(System.out));
        int tests = in.nextInt();
```

```
        for (int t = 1; t <= tests; t++) {
            solve(in, out);
        }
        out.flush();  // without this, buffered output may never appear
    }

    static void solve(FastIn in, PrintWriter out) throws IOException {
        int n = in.nextInt();
        long min = Long.MAX_VALUE;
        long max = Long.MIN_VALUE;
        long sum = 0;
        for (int i = 0; i < n; i++) {
            long value = in.nextLong();
            min = Math.min(min, value);
            max = Math.max(max, value);
            sum += value;
        }
        out.println(min + " " + max + " " + sum);
    }

    /** Minimal fast reader for whitespace-separated integers and words. */
    static final class FastIn {
        private final InputStream stream;
        private final byte[] buf = new byte[1 << 16];
        private int len;
        private int pos;

        FastIn(InputStream stream) {
            this.stream = stream;
        }

        private int read() throws IOException {
            if (pos == len) {
                len = stream.read(buf, 0, buf.length);
                pos = 0;
                if (len <= 0) {
                    len = 0;
                    return -1;              // end of input
                }
            }
            return buf[pos++] & 0xFF;
        }

        long nextLong() throws IOException {
            int c = read();
            while (c != -1 && c <= ' ') {
                c = read();                 // skip spaces and line breaks
            }
            boolean negative = c == '-';
            if (negative) {
                c = read();
            }
            long value = 0;
            while (c >= '0' && c <= '9') {
                value = value * 10 + (c - '0');
                c = read();
            }
            return negative ? -value : value;
        }

        int nextInt() throws IOException {
            return (int) nextLong();
        }

        String next() throws IOException {
            int c = read();
            while (c != -1 && c <= ' ') {
                c = read();
            }
```

```
            StringBuilder sb = new StringBuilder();
            while (c > ' ') {
                sb.append((char) c);
                c = read();
            }
            return sb.toString();
        }
    }
}
```

How the parts fit

- **Lines 1–4: imports.** Only the classes the template uses. Many contestants write `import java.io.*;` and `import java.util.*;` instead, which is shorter and equally correct.
- **The class name.** Most judges require the class that contains `main` to be called `Main` and to be in no package. Rename `OaTemplate` before you submit; LeetCode instead gives you a `Solution` class and calls your method directly, without any input reading.
- **`main` (lines 13–21).** Creates the reader and writer, reads the number of test cases, calls `solve` for each, and flushes once at the end.
- **`solve` (lines 23–35).** The only part you replace for a new problem.
- **`FastIn` (lines 37–93).** A compact version of `FastReader`: `read()` returns the next byte, or −1 at the end of the input, refilling the 64 KB buffer when it is empty; `nextLong()` skips whitespace, reads an optional minus sign and then digits; `next()` reads a word. It is a `static` nested class so that `main` can create it without an instance of the outer class (Chapter 3).

To adapt the template to a problem without test cases, replace the loop in `main` by a single `solve(in, out)` call. To read a grid, call `next()` once per row and use `toCharArray()`. If you prefer `BufferedReader`, the `next()` helper of Exercise 2.1 gives you the same token-based reading.

> **INTERVIEW TIP · Practise with your template before the assessment**
> Type your template from memory a few times, and submit it to an easy problem on the platform you will be tested on. During a timed assessment you want to spend your attention on the problem, not on remembering how to flush a `PrintWriter`.

2.10 Summary

For large inputs, `Scanner` is too slow; use `BufferedReader` with `StringTokenizer`, or a byte-level fast reader. Build output in a `StringBuilder` or a buffered `PrintWriter`, and always flush. Format numbers with `Locale.ROOT`, because the default locale can change digits and decimal separators. Handle the standard input shapes: T test cases, reading until `readLine()` returns `null`, and grids read row by row. Use `long` before reaching for `BigInteger`, and create `BigDecimal` values from strings. `Arrays` and `Collections` provide sorting, searching and other utilities; know which lists are fixed-size or unmodifiable. Lambdas and method references implement functional interfaces concisely; `merge` and `computeIfAbsent` make counting and grouping one line each. Build comparators with `Integer.compare`, `comparingInt`, `thenComparing` and `reversed`, never with subtraction.

Table 2.7: Key takeaways

Topic	Remember
Fast input	`BufferedReader` + `StringTokenizer`, or a byte-level reader
Fast output	`StringBuilder` or `PrintWriter(new BufferedOutputStream(System.out))`
Flush	`out.flush()` at the end of `main`, or the output is lost
Locale	`String.format(Locale.ROOT, "%.2f", x)`
Until EOF	`while ((line = reader.readLine()) != null)`
Big numbers	`BigInteger` for huge integers; `BigDecimal` from strings; `compareTo`
`binarySearch`	index if found, otherwise `-(insertion point) - 1`
`Arrays.asList` / `List.of`	fixed-size view / unmodifiable

Topic	Remember
Lambdas	implement a functional interface; capture only effectively final locals
Counting, grouping	`merge(key, 1, Integer::sum)`, `computeIfAbsent(key, k -> new ArrayList<>())`
Comparators	`Integer.compare`, `comparingInt`, `thenComparing`, `reversed`; never `a - b`
Template	read T, `solve` per test, flush once; rename the class to `Main`

2.11 Exercises

Exercise 2.1 EASY **Tokens across lines**

`BufferedSum` assumes that all n numbers are on one line. Write a `next()` helper, based on `BufferedReader` and `StringTokenizer`, that returns the next token wherever it is: on the current line, on a later line, or after blank lines. It should return `null` at the end of the input. Use it to sum n numbers spread over several lines.

Solution →

Exercise 2.2 EASY **Count lines, words and characters**

Read the whole input until its end and print the number of lines, the number of words (maximal runs of non-whitespace characters) and the number of characters, not counting line breaks. The input may contain blank lines and lines that start with spaces.

Solution →

Exercise 2.3 EASY **By length, then alphabetically**

Sort an array of words by length, shortest first; words of equal length should be in alphabetical order. Write the comparator with `Comparator` methods, not with a hand-written `if` chain.

Solution →

Exercise 2.4 MEDIUM **Break the subtraction comparator**

Find two `Integer` values for which `(a, b) -> a - b` gives the wrong sign, and show an array that it sorts incorrectly. Then fix the comparator. Under what condition on the values would subtraction be safe?

Solution →

Exercise 2.5 MEDIUM **Sort intervals**

Sort an `int[][]` of intervals `{start, end}` by start in ascending order, and for equal starts by end in *descending* order (the longest interval first). This order is used by several interval algorithms in Chapter 21.

Solution →

Exercise 2.6 MEDIUM **Descending primitives**

Write two methods that sort an `int[]` in descending order: one that boxes the values into an `Integer[]` and uses `Collections.reverseOrder()`, and one that never boxes. Compare them in memory use and speed, and explain the difference.

Solution →

Exercise 2.7 MEDIUM **Group by first letter**

Given an array of words, return a map from each first letter to the list of words that start with it, in their original order, with the letters in alphabetical order. Skip empty words. Use `computeIfAbsent`.

Solution →

Exercise 2.8 EASY **Word frequencies**

Count how often each word occurs in a text, ignoring case and punctuation, and return the words with their counts, most frequent first; words with equal counts should be in alphabetical order. Use `Map.merge` to count.

Solution →

Exercise 2.9 MEDIUM **Beyond long**

Use `BigInteger` to compute the sum of the digits of 100! and the number of decimal digits of 2^{1000}. Could either answer be computed with `long` arithmetic alone? Explain.

Solution →

Exercise 2.10 MEDIUM **Exact money**

A shop sells items at 19.99, 5.01 and 0.10, and a customer buys 3, 2 and 1 of them. Compute the total exactly with `BigDecimal`, rounded to two decimal places, and compare with the same computation in `double`. Then explain why `new BigDecimal("70.09")` and `new BigDecimal("70.090")` are not `equals` but `compareTo` returns 0.

Solution →

Exercise 2.11 HARD **Distinct values per test case**

The input has T test cases. Each has n followed by n integers; the total of all n is at most 2×10^5. For each test case, print `Case #t:` followed by every distinct value in increasing order, each written as `value x count` (for example `4x3`). Make the input and output fast, and explain the time complexity.

Solution →

Exercise 2.12 CHALLENGE **Parse a line by hand**

Write `int[] parseInts(String line)` that returns all integers on a line, in order, without using `split`, `StringTokenizer` or `Scanner`. Numbers may be negative and may be separated by any number of spaces or tabs. Use it to print the sum of every line of the input until its end.

Solution →

2.12 Solutions

2.12.1 Warm-up quiz answers

1. `BufferedReader` (with `StringTokenizer` or manual parsing). `Scanner` matches the input against regular expressions for every token, which costs far more than reading characters from a buffer. (Section 2.1)
2. `out.flush()` (or `close()`) at the end of `main`. (Section 2.3)
3. `a - b` overflows when the values are far apart, giving the wrong sign. (Section 2.8)
4. `-(insertion point) - 1`, which is always negative. (Section 2.6)
5. Nothing that matters: both create a `Comparator` that calls `Integer.compare`; the method reference is just shorter. (Section 2.7)

Solution 2.1 ← Exercise

Keep one `StringTokenizer` and replace it with the next line's tokenizer whenever it has no more tokens. Blank lines produce an empty tokenizer, so the loop simply reads on.

Listing 2.40 Fields and methods — TokenReaderSum.java

```
private static BufferedReader reader;
private static StringTokenizer tokens = new StringTokenizer("");

/** Returns the next token, reading more lines as needed (null at end of input). */
static String next() throws IOException {
    while (!tokens.hasMoreTokens()) {
        String line = reader.readLine();
        if (line == null) {
            return null;
        }
        tokens = new StringTokenizer(line);   // blank lines simply give no tokens
    }
    return tokens.nextToken();
}

public static void main(String[] args) throws IOException {
    reader = new BufferedReader(new InputStreamReader(System.in));
```

```
    int n = Integer.parseInt(next());
    long sum = 0;
    for (int i = 0; i < n; i++) {
        sum += Integer.parseInt(next());
    }
    System.out.println(sum);
}
```

Listing 2.41 Input — TokenReaderSum.in

```
6
10 20
30

40 50 60
```

Listing 2.42 Output — TokenReaderSum

```
210
```

Each line is read once and each token is produced once, so reading takes time proportional to the input size. Returning `null` at the end of the input lets a caller write `while ((token = next()) != null)` to read until the end.

Solution 2.2

← Exercise

Listing 2.43 Fragment of main — WordCount.java

```
BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
int lines = 0;
int words = 0;
long characters = 0;
String line;
while ((line = reader.readLine()) != null) {
    lines++;
    characters += line.length();          // readLine removes the line break
    words += new StringTokenizer(line).countTokens();
}
System.out.println(lines + " lines, " + words + " words, " + characters
        + " characters (not counting line breaks)");
```

Listing 2.44 Input — WordCount.in

```
the quick brown fox
  jumps over

the lazy dog
```

Listing 2.45 Output — WordCount

```
4 lines, 9 words, 43 characters (not counting line breaks)
```

`readLine()` returns lines without their line breaks, so `line.length()` counts exactly the characters on the line. `StringTokenizer.countTokens()` counts the words on a line without creating them, and a blank line simply contributes zero words. The input has four lines, one of them blank.

Solution 2.3

← Exercise

Listing 2.46 Method — SortByLength.java

```
static void sortByLength(String[] words) {
    Arrays.sort(words, Comparator.comparingInt(String::length)
            .thenComparing(Comparator.naturalOrder()));
}
```

Listing 2.47 Output — SortByLength

```
[fig, date, kiwi, pear, plum, apple, banana]
```

`Comparator.comparingInt(String::length)` compares the lengths, and `thenComparing(Comparator.naturalOrder())` compares equal-length words alphabetically. The sort is O(n log n) comparisons, and each comparison costs O(length of the words) in the worst case.

Solution 2.4

← Exercise

Listing 2.48 Fields — ComparatorBug.java

```
1 static final Comparator<Integer> BUGGY = (a, b) -> a - b;
2 static final Comparator<Integer> FIXED = Integer::compare;
```

Listing 2.49 Output — ComparatorBug

```
BUGGY.compare(low, high) = 294967296  (positive: claims low > high)
FIXED.compare(low, high) = -1
buggy sort: [2000000000, -2000000000]
fixed sort: [-2000000000, 2000000000]
```

For `low = -2,000,000,000` and `high = 2,000,000,000`, the true difference -4×10^9 does not fit in an `int` and wraps around to the positive value 294,967,296, so the comparator says `low > high`, and the two-element array stays in the wrong order. Subtraction is safe only when every possible difference fits in an `int`, for example when all values are known to be between -2^{30} and 2^{30}. Rather than reasoning about that each time, use `Integer::compare`, which is always correct.

Solution 2.5

← Exercise

Listing 2.50 Method — SortIntervals.java

```
1 static void sortIntervals(int[][] intervals) {
2     Arrays.sort(intervals, (a, b) -> {
3         if (a[0] != b[0]) {
4             return Integer.compare(a[0], b[0]);   // smaller start first
5         }
6         return Integer.compare(b[1], a[1]);       // then larger end first
7     });
8 }
```

Listing 2.51 Output — SortIntervals

```
[[0, 1], [1, 7], [1, 3], [2, 9], [2, 4]]
```

When the starts differ, compare starts; otherwise compare the ends with the arguments *swapped* (`b[1]`, `a[1]`), which reverses that order. The same comparator can be built from parts: `Comparator.comparingInt((int[] a) -> a[0]).thenComparing((int[] a) -> a[1], Comparator.reverseOrder())`, but the explicit version is easier to check.

Solution 2.6

← Exercise

Listing 2.52 Methods — SortDescending.java

```
1 /** Boxes every value: simple, but uses far more memory and is slower. */
2 static int[] descendingBoxed(int[] values) {
3     Integer[] boxed = new Integer[values.length];
4     for (int i = 0; i < values.length; i++) {
5         boxed[i] = values[i];
6     }
7     Arrays.sort(boxed, Collections.reverseOrder());
8     int[] result = new int[values.length];
9     for (int i = 0; i < values.length; i++) {
```

```
        result[i] = boxed[i];
    }
    return result;
}

/** Sorts the primitives ascending, then reverses them in place. */
static void sortDescendingInPlace(int[] values) {
    Arrays.sort(values);
    for (int i = 0, j = values.length - 1; i < j; i++, j--) {
        int temp = values[i];
        values[i] = values[j];
        values[j] = temp;
    }
}
```

Listing 2.53 Output — SortDescending

```
[7, 7, 3, 0, -1]
[7, 7, 3, 0, -1]
```

The boxed version creates an `Integer` object for every value outside the cached range −128 to 127, and each object takes several times the 4 bytes of an `int` (Chapter 5), plus a reference in the `Integer[]`. It also sorts with a comparator, which is slower than sorting primitives. The in-place version uses no extra memory beyond what `Arrays.sort` needs internally, and the reversal is O(n). Both are O(n log n) overall; the in-place version is the better choice.

Solution 2.7

← Exercise

Listing 2.54 Method — GroupByFirstLetter.java

```
static Map<Character, List<String>> group(String[] words) {
    Map<Character, List<String>> groups = new TreeMap<>();  // keys kept sorted
    for (String word : words) {
        if (word.isEmpty()) {
            continue;                                       // no first letter
        }
        groups.computeIfAbsent(word.charAt(0), letter -> new ArrayList<>()).add(word);
    }
    return groups;
}
```

Listing 2.55 Output — GroupByFirstLetter

```
{a=[apple, avocado], b=[banana, blueberry], c=[cherry], f=[fig]}
```

A `TreeMap` keeps its keys sorted (Chapter 7), so the letters come out in alphabetical order; an `ArrayList` keeps the words in their original order. `computeIfAbsent` creates each list exactly once, the first time its letter appears. The empty string is skipped, because `charAt(0)` would throw on it.

Solution 2.8

← Exercise

Listing 2.56 Method — WordFrequencies.java

```
static List<String> frequencies(String text) {
    Map<String, Integer> counts = new HashMap<>();
    for (String word : text.toLowerCase().split("\\W+")) {  // split on non-letters
        if (!word.isEmpty()) {
            counts.merge(word, 1, Integer::sum);
        }
    }
    List<Map.Entry<String, Integer>> entries = new ArrayList<>(counts.entrySet());
    entries.sort(Map.Entry.<String, Integer>comparingByValue().reversed()
            .thenComparing(Map.Entry.comparingByKey()));
    List<String> result = new ArrayList<>();
    for (Map.Entry<String, Integer> entry : entries) {
```

```
        result.add(entry.getKey() + "=" + entry.getValue());
    }
    return result;
}
```

Listing 2.57 Output — WordFrequencies

```
[the=3, cat=2, dog=2, saw=2, a=1]
```

`split("\\W+")` splits on runs of characters that are not letters, digits or underscores, which removes punctuation. The map entries are copied into a list and sorted with `Map.Entry.comparingByValue().reversed()` followed by `comparingByKey()`. The explicit type arguments `<String, Integer>` are needed for the same reason as the explicit `(int[] a)` in the interval comparator: the call is the start of a chain.

Solution 2.9

← Exercise

Listing 2.58 Methods — BigDigits.java

```
static int digitSumOfFactorial(int n) {
    BigInteger factorial = BigInteger.ONE;
    for (int i = 2; i <= n; i++) {
        factorial = factorial.multiply(BigInteger.valueOf(i));
    }
    int sum = 0;
    for (char digit : factorial.toString().toCharArray()) {
        sum += digit - '0';
    }
    return sum;
}

static int digitsOfPowerOfTwo(int exponent) {
    return BigInteger.TWO.pow(exponent).toString().length();
}
```

Listing 2.59 Output — BigDigits

```
digit sum of 100! = 648
digits in 2^1000  = 302
```

100! has 158 digits and 2^{1000} has 302, far beyond the 19 digits of a `long`, so neither number can be computed in `long`. The *number of digits* of 2^{1000} can be found without big numbers, though: it is $\lfloor 1000 \times \log_{10} 2 \rfloor + 1 = 302$, using `Math.log10`. The digit sum of 100! has no such shortcut.

Solution 2.10

← Exercise

Listing 2.60 Method — PriceTotal.java

```
static BigDecimal total(String[] prices, int[] quantities) {
    BigDecimal total = BigDecimal.ZERO;
    for (int i = 0; i < prices.length; i++) {
        BigDecimal quantity = BigDecimal.valueOf(quantities[i]);
        BigDecimal line = new BigDecimal(prices[i]).multiply(quantity);
        total = total.add(line);                  // BigDecimal is immutable: reassign
    }
    return total.setScale(2, RoundingMode.HALF_EVEN);
}
```

Listing 2.61 Output — PriceTotal

```
BigDecimal total: 70.09
double total:     70.08999999999999
equals: false, compareTo: 0
```

The `BigDecimal` total is exactly 70.09; the `double` total is 70.08999999999999, because none of 19.99, 5.01 and 0.10 is exactly representable in binary. `equals` compares both the value and the scale, and 70.09 has scale 2 while 70.090 has scale 3, so they are not equal; `compareTo` compares only the numeric value. For money, store amounts as `BigDecimal` from strings (or as whole paisa in a `long`), and compare them with `compareTo`.

Solution 2.11

← Exercise

Sort each test case; equal values become neighbours, so one pass groups them.

Listing 2.62 Method — DistinctCounts.java

```
public static void main(String[] args) throws IOException {
    Reader in = new Reader(System.in);
    PrintWriter out = new PrintWriter(new BufferedOutputStream(System.out));
    int tests = in.nextInt();
    for (int t = 1; t <= tests; t++) {
        int n = in.nextInt();
        int[] values = new int[n];
        for (int i = 0; i < n; i++) {
            values[i] = in.nextInt();
        }
        Arrays.sort(values);                    // equal values become neighbours
        StringBuilder line = new StringBuilder("Case #").append(t).append(':');
        int i = 0;
        while (i < n) {
            int j = i;
            while (j < n && values[j] == values[i]) {
                j++;                            // values[i..j-1] are all equal
            }
            line.append(' ').append(values[i]).append('x').append(j - i);
            i = j;
        }
        out.println(line);
    }
    out.flush();
}
```

Listing 2.63 Input — DistinctCounts.in

```
2
7
4 1 4 9 1 4 2
3
-5 -5 -5
```

Listing 2.64 Output — DistinctCounts

```
Case #1: 1x2 2x1 4x3 9x1
Case #2: -5x3
```

Sorting costs O(n log n) per test case, and the grouping pass is O(n), because the inner loop's index `j` only moves forward and the outer loop restarts from it. Over all test cases the time is O(N log N) for N total numbers. Building each line in a `StringBuilder` and printing it through a buffered `PrintWriter` keeps output fast even when every value is distinct. (A `TreeMap` from value to count would also work, in O(N log N) time, but with much more memory.)

Solution 2.12

← Exercise

Listing 2.65 Method — LineParser.java

```
/** All integers on the line, in order. Accepts any amount of spaces and tabs. */
static int[] parseInts(String line) {
    int[] values = new int[line.length() / 2 + 1];  // more than enough slots
    int count = 0;
    int i = 0;
    while (i < line.length()) {
```

```
        char c = line.charAt(i);
        if (c == ' ' || c == '\t') {
            i++;
            continue;
        }
        boolean negative = c == '-';
        if (negative) {
            i++;
        }
        int value = 0;
        while (i < line.length() && line.charAt(i) >= '0' && line.charAt(i) <= '9') {
            value = value * 10 + (line.charAt(i) - '0');
            i++;
        }
        values[count++] = negative ? -value : value;
    }
    return Arrays.copyOf(values, count);
}
```

Listing 2.66 Input — LineParser.in

```
1 2 3
  -4   5

100
```

Listing 2.67 Output — LineParser

```
line 1: [1, 2, 3] sum 6
line 2: [-4, 5] sum 1
line 3: [] sum 0
line 4: [100] sum 100
```

The parser walks the line once: spaces and tabs are skipped, an optional minus sign is consumed, and digits are accumulated as in the fast reader. A line of length L contains at most L/2 + 1 numbers, since every number needs at least one digit and numbers are separated by at least one space, so that array size is always enough; `Arrays.copyOf` trims it at the end. Each line costs O(L), and the empty third line correctly gives an empty array.

PART III

Data Structures and Algorithms

The core of the book. Each structure and algorithm is explained as an idea first, then implemented in Java, traced by hand, analyzed, and practiced in interview-style problems.

10 Stacks, Queues and Deques

CHAPTER 10

Stacks, Queues and Deques

IN THIS CHAPTER

After this chapter you will be able to:

- explain what stacks, queues and deques are, and why each one exists;
- implement a stack with an array and with linked nodes, and a queue with a circular array;
- use `Deque` and `ArrayDeque` correctly as a stack and as a queue, and explain why the legacy `Stack` class is usually avoided;
- solve bracket-matching and expression-evaluation problems with stacks;
- recognize and apply the *monotonic stack* and *monotonic queue* patterns;
- explain why a queue gives shortest paths in breadth-first search;
- analyze all of these, including their amortized costs, and discuss them in an interview.

Prerequisites: Big-O notation and amortized analysis (Chapter 6); the `Deque` interface's place in the Collections Framework (Chapter 7); linked nodes (Chapter 9).

WARM-UP QUIZ

1. You push 1, 2 and 3 onto a stack and then pop once. Which value comes out?
2. You add 1, 2 and 3 to a queue and then remove once. Which value comes out?
3. What does `peek()` return on an empty `ArrayDeque`, and what does `pop()` do?
4. A growable array doubles its size when it is full. Why is pushing onto it "O(1) amortized" rather than simply O(1)?
5. Name one problem you have solved (in any language) that needed "the most recent unfinished thing" first.

A surprising number of problems have the same shape: things arrive one at a time, and the program must decide *which waiting thing to deal with next*. If the answer is "the most recent one", you need a stack. If it is "the oldest one", you need a queue. If you sometimes need one end and sometimes the other, you need a deque. These three structures are simple, but they are the engine inside many famous algorithms: bracket matching, expression evaluation, depth-first and breadth-first search, and the monotonic-stack techniques that turn many $O(n^2)$ solutions into $O(n)$ ones.

10.1 Stacks: last in, first out

DEFINITION · Stack

A stack is a collection where the element removed next is always the one added most recently. This rule is called LIFO: *last in, first out*. The end where elements are added and removed is the **top**.

Why it exists. Many tasks must be finished in the reverse order of how they were started. When you open a bracket `(` and then a bracket `[`, the `[` must be closed first. When method `a` calls `b` and `b` calls `c`, then `c` returns first, then `b`, then `a`; the JVM keeps these unfinished calls on a stack, which is why it is called the *call stack* (Chapter 5). An editor's *undo* list, a browser's *back* button, and depth-first search all follow the same rule.

How it works. A stack offers a few operations, and all of them touch only the top:

Table 10.1: The stack operations

Operation	Meaning	Typical cost
`push(x)`	put x on top	O(1)
`pop()`	remove the top element and return it	O(1)
`peek()`	return the top element without removing it	O(1)
`isEmpty()`	is the stack empty?	O(1)
`size()`	how many elements are on the stack?	O(1)

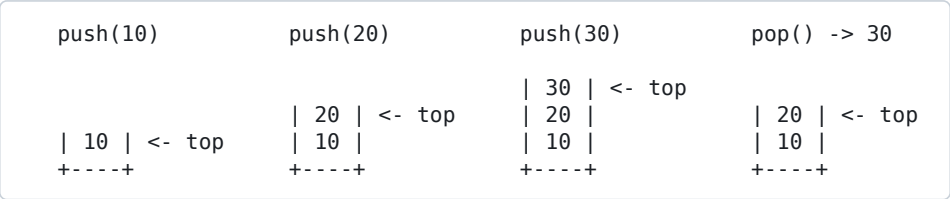


Figure 10.1: A stack after push(10), push(20), push(30), then pop()

Because only the top is ever touched, a stack never needs to search or shift anything. That is why every operation can be O(1). The next two sections build a stack twice, once on an array and once on linked nodes, because the two designs show different trade-offs that interviewers like to ask about.

10.1.1 Implementing a stack with an array

The simplest stack keeps its elements in an array. The bottom element sits at index `0` and the top element at index `size - 1`. Pushing writes to index `size`; popping reads from index `size - 1`. When the array is full, we create a bigger one and copy the elements across.

Listing 10.1 Complete program `IntStack.java`

```
1  import java.util.Arrays;
2  import java.util.NoSuchElementException;
3
4  /** A last-in, first-out stack of int values stored in a growable array. */
5  public class IntStack {
6
7      private int[] items = new int[2];  // items[0..size-1] hold the stack
8      private int size;                  // how many values are on the stack
9
10     public void push(int value) {
11         if (size == items.length) {
12             items = Arrays.copyOf(items, items.length * 2);  // full: double the array
13         }
14         items[size] = value;          // the top is always items[size - 1]
15         size++;
16     }
17
18     public int pop() {
19         if (size == 0) {
20             throw new NoSuchElementException("pop from an empty stack");
21         }
22         size--;
23         return items[size];
24     }
25
26     public int peek() {
27         if (size == 0) {
28             throw new NoSuchElementException("peek at an empty stack");
29         }
30         return items[size - 1];
31     }
32
33     public boolean isEmpty() {
34         return size == 0;
35     }
36
37     public int size() {
```

```
38          return size;
39      }
40
41      int capacity() {  // package-private: only for the demonstration below
42          return items.length;
43      }
44
45      public static void main(String[] args) {
46          IntStack stack = new IntStack();
47          for (int value = 10; value <= 50; value += 10) {
48              stack.push(value);
49              System.out.printf("push %d  -> size %d, capacity %d%n",
50                      value, stack.size(), stack.capacity());
51          }
52          System.out.println("peek    -> " + stack.peek());
53          StringBuilder popped = new StringBuilder();
54          while (!stack.isEmpty()) {
55              popped.append(stack.pop()).append(' ');
56          }
57          System.out.println("popped  -> " + popped.toString().trim());
58          try {
59              stack.pop();
60          } catch (NoSuchElementException e) {
61              System.out.println("pop on empty stack -> " + e.getMessage());
62          }
63      }
64  }
```

Syntax breakdown

- `private int[] items = new int[2];` (line 7) declares a field and gives it a starting value. Every new `IntStack` object starts with an array of two `int`s, all `0`. A real implementation would start larger; two is used here so that you can watch the array grow.
- `private int size;` (line 8) has no initializer. Fields of type `int` start at `0`, so a new stack is empty. (Local variables are different: Java refuses to compile code that reads a local variable before assigning it.)
- `Arrays.copyOf(items, items.length * 2)` (line 12) creates a *new* array of the given length, copies the old elements into the front of it, and fills the rest with zeros. The old array is no longer referenced and the garbage collector can reclaim it.
- `throw new NoSuchElementException(...)` (line 20) creates an exception object and throws it. `NoSuchElementException` is an *unchecked* exception from `java.util`, the same one that `ArrayDeque` throws, so callers are not forced to catch it.
- `int capacity()` (line 41) has no access modifier, so it is *package-private*: other classes in the same folder (the same package) can call it, but it is not part of the public interface. The demonstration and the self-check program use it.
- `System.out.printf("push %d  -> ...%n", ...)` (lines 49–50) formats text: each `%d` is replaced by the next `int` argument, and `%n` writes the platform's line separator.

Line by line. `push` first checks whether the array is full (line 11). If so, line 12 replaces `items` with a copy twice as long. Line 14 stores the value in the first free slot, and line 15 records that the stack is one element bigger. `pop` refuses to work on an empty stack (lines 19–21); otherwise it shrinks `size` (line 22) and returns the element that was on top (line 23). Lines 22 and 23 could be written as `return items[--size];`, but two separate lines are easier to read. `peek` is the same as `pop` except that it does not change `size`.

The `main` method pushes five values, prints the size and capacity after each push, and then pops everything:

Listing 10.2 Output IntStack

```
push 10  -> size 1, capacity 2
push 20  -> size 2, capacity 2
push 30  -> size 3, capacity 4
push 40  -> size 4, capacity 4
push 50  -> size 5, capacity 8
peek    -> 50
popped  -> 50 40 30 20 10
pop on empty stack -> pop from an empty stack
```

The capacity goes 2, 2, 4, 4, 8: the array doubles exactly when a push finds it full. The values come out in reverse order, which is the LIFO rule in action. The last line shows the exception message when we pop from an empty stack.

KEY IDEA · Doubling makes push O(1) amortized

A push that triggers a resize copies every element, so it costs O(n). But resizes are rare: after doubling from n to 2n slots, the next n pushes are cheap. Over any sequence of n pushes, the copying work is at most 1 + 2 + 4 + … + n < 2n element copies, so the *average* cost per push is a constant. We say push is O(1) amortized. If the array grew by a fixed amount instead (say, 10 slots), the copying would add up to about $n^2/20$, and push would become O(n) on average. Chapter 6 explains amortized analysis in detail.

Table 10.2: Cost of the array-based stack

Operation	Time	Why
`push`	O(1) amortized, O(n) worst case	a resize copies all n elements, but rarely
`pop`, `peek`	O(1)	one array access
`isEmpty`, `size`	O(1)	read one field
Space	O(n)	the array is at most about twice as large as needed

WARNING · Obsolete references in object stacks

`IntStack` stores `int` values, so leaving an old value in the array after `pop` is harmless. A stack of *objects* must also clear the slot (`items[size] = null;`) after popping. Otherwise the array still refers to the popped object, the garbage collector cannot reclaim it, and the program slowly leaks memory. Joshua Bloch's *Effective Java* uses exactly this stack bug as its main example of an "obsolete reference".

10.1.2 Implementing a stack with linked nodes

A stack can also be a chain of nodes, where each node holds one value and a reference to the node below it. The stack only needs to remember the top node.

Listing 10.3 Complete program LinkedStack.java

```
1  import java.util.NoSuchElementException;
2
3  /** A generic stack built from linked nodes; every operation is O(1) in the worst case. */
4  public class LinkedStack<T> {
5
6      private static final class Node<E> {
7          final E value;
8          final Node<E> next;  // the node below this one
9
10         Node(E value, Node<E> next) {
11             this.value = value;
12             this.next = next;
13         }
14     }
15
16     private Node<T> top;  // null when the stack is empty
17     private int size;
18
```

```
19      public void push(T value) {
20          top = new Node<>(value, top);
21          size++;
22      }
23
24      public T pop() {
25          if (top == null) {
26              throw new NoSuchElementException("pop from an empty stack");
27          }
28          T value = top.value;
29          top = top.next;  // the old top node is now unreachable
30          size--;
31          return value;
32      }
33
34      public T peek() {
35          if (top == null) {
36              throw new NoSuchElementException("peek at an empty stack");
37          }
38          return top.value;
39      }
40
41      public boolean isEmpty() {
42          return top == null;
43      }
44
45      public int size() {
46          return size;
47      }
48
49      public static void main(String[] args) {
50          LinkedStack<String> pages = new LinkedStack<>();
51          pages.push("home");
52          pages.push("search");
53          pages.push("results");
54          System.out.println("current page: " + pages.peek());
55          String left = pages.pop();
56          System.out.println("left " + left + ", back on " + pages.peek());
57          System.out.println("pages left: " + pages.size());
58      }
59  }
```

Syntax breakdown

- `public class LinkedStack<T>` (line 4) declares a *generic* class. `T` is a type parameter: a `LinkedStack<String>` holds strings and a `LinkedStack<Integer>` holds integers, and the compiler checks every use. Chapter 3 covers generics in depth.
- `private static final class Node<E>` (line 6) is a *static nested class*. `static` means a `Node` object does not carry a hidden reference to a `LinkedStack` object, which saves memory and is the usual choice for node classes. `final` means no class can extend it. It declares its own type parameter `E`; when `LinkedStack<T>` writes `Node<T>`, `E` becomes `T`.
- `final E value;` and `final Node<E> next;` (lines 7–8) are assigned once, in the constructor, and never change. Nodes in this stack are never modified after creation.
- `new Node<>(value, top)` (line 20) uses the *diamond operator* `<>`: the compiler works out that the type is `Node<T>` from the variable it is assigned to.

How push and pop work. `push` creates a node whose `next` points to the current top, then makes the new node the top (line 20). `pop` saves the top value (line 28) and moves `top` down one node (line 29). Nothing refers to the old top node any more, so the garbage collector can reclaim it; unlike the array version, no explicit clean-up is needed.

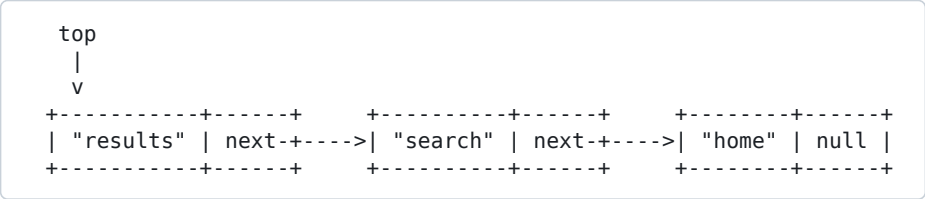


Figure 10.2: LinkedStack after pushing "home", "search" and "results"

Listing 10.4 Output LinkedStack

```
current page: results
left results, back on search
pages left: 2
```

Array or linked nodes? Both give O(1) push and pop, but they differ in detail:

Table 10.3: Two ways to build a stack

	Array-based	Linked nodes
push	O(1) amortized; occasionally O(n) when it resizes	O(1) every time
Memory per element	one array slot (4 bytes for `int`), plus unused capacity	a whole node object: header, value reference, next reference
Memory locality	elements sit next to each other, which is cache-friendly	nodes are scattered across the heap
Typical speed	faster in practice	slower in practice

The exact size of an object depends on the JVM (Chapter 5), but a node is always much larger than one array slot. For coding interviews and online judges, an array-based stack, either your own `int[]` or Java's `ArrayDeque`, is almost always the right choice.

10.2 Queues: first in, first out

> **DEFINITION · Queue**
> A queue is a collection where the element removed next is always the one that has waited longest. This rule is called FIFO: *first in, first out*. Elements join at the **back** (or *tail*) and leave from the **front** (or *head*).

Why it exists. Whenever work must be handled fairly, in arrival order, a queue is the natural model: print jobs, requests to a web server, messages between programs, and the cells of a maze explored by breadth-first search (Section 10.8). The operations are the queue versions of push and pop:

Table 10.4: The queue operations

Operation	Meaning	Typical cost
`enqueue(x)`	add x at the back	O(1)
`dequeue()`	remove and return the front element	O(1)
`peek()`	return the front element without removing it	O(1)

The obvious array implementation is too slow. If the front element always lives at index 0, then every `dequeue` must shift all remaining elements one place left, which is O(n). The fix is to let the front *move* instead of the elements.

10.2.1 A circular array queue

Keep an index `head` that points at the front element, and a count `size`. The back of the queue is `size` slots after `head`. When an index runs off the end of the array, it wraps around to 0, as if the array were bent into a ring. This structure is called a circular buffer or *ring buffer*.

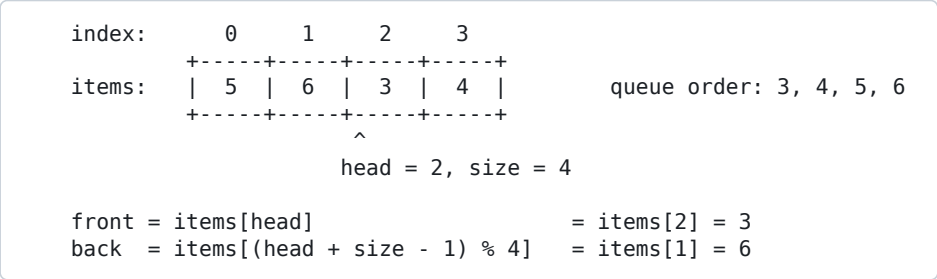


Figure 10.3: A circular array of capacity 4 holding the queue 3, 4, 5, 6

The listing shows only the queue logic. The complete file in `code-examples/ch10` also has a `main` method and a small `layout` method that prints the raw array, with _ for unused slots.

Listing 10.5 Fields and methods IntQueue.java

```
1  private int[] items = new int[4];
2  private int head;  // index of the front value
3  private int size;  // how many values are in the queue
4
5  public void enqueue(int value) {
6      if (size == items.length) {
7          grow();
8      }
9      int tail = (head + size) % items.length;  // first free slot after the back
10     items[tail] = value;
11     size++;
12 }
13
14 public int dequeue() {
15     if (size == 0) {
16         throw new NoSuchElementException("dequeue from an empty queue");
17     }
18     int value = items[head];
19     head = (head + 1) % items.length;  // step forward, wrapping to 0 at the end
20     size--;
21     return value;
22 }
23
24 public int peek() {
25     if (size == 0) {
26         throw new NoSuchElementException("peek at an empty queue");
27     }
28     return items[head];
29 }
30
31 public boolean isEmpty() {
32     return size == 0;
33 }
34
35 public int size() {
36     return size;
37 }
38
39 private void grow() {
40     int[] bigger = new int[items.length * 2];
41     for (int i = 0; i < size; i++) {
42         bigger[i] = items[(head + i) % items.length];  // copy in queue order
43     }
44     items = bigger;
45     head = 0;
46 }
```

Syntax breakdown. The `%` operator (lines 9, 19 and 42) is the *remainder* operator: `a % b` is what is left after dividing `a` by `b`. With a capacity of 4, the index after 3 is `(3 + 1) % 4 = 0`, which is how the queue wraps around. Both operands are non-negative here, which matters: in Java, `-1 % 4` is `-1`, not 3.

Line by line. `enqueue` grows the array if it is full (lines 6–8), computes the first free slot after the back (line 9), stores the value there (line 10) and increases `size` (line 11). `dequeue` refuses to work on an empty queue (lines 15–17), reads the front value (line 18), moves `head` forward with wrap-around (line 19), and decreases `size` (line 20). `grow` (lines 39–46) cannot simply call `Arrays.copyOf`, because the queue may wrap around the end of the old array. It copies the elements *in queue order* into the start of a new array (lines 41–43) and resets `head` to 0 (line 45).

The demonstration enqueues 1 to 4, dequeues twice, enqueues 5 and 6 (which wrap around to the start of the array), and then enqueues 7, which forces a resize:

Listing 10.6 Output `IntQueue`

```
after enqueue 1..4   [1 2 3 4] head=0 size=4
dequeue -> 1, dequeue -> 2
after two dequeues   [_ _ 3 4] head=2 size=2
after enqueue 5, 6   [5 6 3 4] head=2 size=4
after enqueue 7      [3 4 5 6 7 _ _ _] head=0 size=5
dequeue order: 3 4 5 6 7
```

Notice the fourth line: the array is `[5 6 3 4]`, but the queue is 3, 4, 5, 6, because `head` is 2. After the resize, the elements are stored in order again starting at index 0.

> **WARNING · Full or empty?**
> Some textbook queues store only `head` and `tail` indices. Then `head == tail` could mean "empty" or "full", and the code needs an extra rule, such as always leaving one slot unused. Keeping an explicit `size`, as `IntQueue` does, removes the ambiguity. Exercise 10.3 asks you to build a fixed-capacity version with this design.

Table 10.5: Cost of the circular array queue

Operation	Time
enqueue	O(1) amortized (O(n) when it resizes)
dequeue, peek	O(1)
Space	O(n)

10.3 Deques and Java's ArrayDeque

> **DEFINITION · Deque**
> A deque (pronounced "deck", short for *double-ended queue*) allows insertion, removal and inspection at **both** ends. A deque can therefore act as a stack (use one end) or as a queue (add at one end, remove at the other).

In Java you will rarely write your own stack or queue. The `java.util.Deque` interface describes a deque, and `java.util.ArrayDeque` implements it with a circular array, just like `IntQueue` but generic and growable at both ends. Each operation comes in two forms: one throws an exception when it fails, the other returns a special value (`null` or `false`).

Table 10.6: The Deque methods (Java SE 21 API)

Operation	Throws an exception on failure	Returns a special value
Insert at front	`addFirst(e)`, `push(e)`	`offerFirst(e)`
Insert at back	`addLast(e)`, `add(e)`	`offerLast(e)`, `offer(e)`
Remove from front	`removeFirst()`, `pop()`, `remove()`	`pollFirst()`, `poll()`
Remove from back	`removeLast()`	`pollLast()`
Look at front	`getFirst()`, `element()`	`peekFirst()`, `peek()`
Look at back	`getLast()`	`peekLast()`

Two consequences of this table matter in every problem:

- **As a stack**, use `push`, `pop` and `peek`. They all work on the *front* of the deque, so the front is the top of the stack.
- **As a queue**, use `offer`, `poll` and `peek`. `offer` adds at the *back*; `poll` and `peek` work on the front.

`peek` is the same method in both roles: it always looks at the front. On an empty deque, `peek` and `poll` return `null`, while `pop`, `remove` and `getFirst` throw `NoSuchElementException`. The program below shows all of this.

Listing 10.7 Complete program — DequeTour.java

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.NoSuchElementException;
import java.util.Stack;

/** How ArrayDeque behaves as a stack and as a queue, and how legacy Stack differs. */
public class DequeTour {

    public static void main(String[] args) {
        Deque<Integer> stack = new ArrayDeque<>();
        stack.push(1);
        stack.push(2);
        stack.push(3);                                  // 3 is now the top (the front)
        System.out.println("stack        " + stack);  // printed from the top down
        System.out.println("pop()        " + stack.pop());
        System.out.println("peek()       " + stack.peek());

        Deque<Integer> queue = new ArrayDeque<>();
        queue.offer(1);
        queue.offer(2);
        queue.offer(3);                                 // 3 is at the back
        System.out.println("queue        " + queue);  // printed from the front
        System.out.println("poll()       " + queue.poll());
        System.out.println("peek()       " + queue.peek());

        Deque<Integer> empty = new ArrayDeque<>();
        System.out.println("empty peek() " + empty.peek());   // null, no exception
        System.out.println("empty poll() " + empty.poll());   // null, no exception
        try {
            empty.pop();                                      // throws
        } catch (NoSuchElementException e) {
            System.out.println("empty pop()  NoSuchElementException");
        }
        try {
            empty.push(null);                                 // nulls are not allowed
        } catch (NullPointerException e) {
            System.out.println("push(null)   NullPointerException");
        }

        Stack<Integer> legacy = new Stack<>();
        legacy.push(1);
        legacy.push(2);
        legacy.push(3);
        System.out.println("legacy Stack " + legacy);  // printed from the BOTTOM up
        legacy.add(0, 99);                              // compiles: Stack is a Vector
        System.out.println("add(0, 99)   " + legacy);
    }
}
```

Listing 10.8 Output DequeTour

```
stack        [3, 2, 1]
pop()        3
peek()       2
queue        [1, 2, 3]
poll()       1
peek()       2
empty peek() null
empty poll() null
empty pop()  NoSuchElementException
push(null)   NullPointerException
legacy Stack [1, 2, 3]
add(0, 99)   [99, 1, 2, 3]
```

What the output shows

- Lines 10–16: after pushing 1, 2, 3, printing the deque shows `[3, 2, 1]`. The `toString` method lists elements from the front, and `push` adds at the front, so the stack prints from the top down. `pop` returns 3, and then `peek` sees 2.
- Lines 18–24: after offering 1, 2, 3, the deque prints `[1, 2, 3]`: front first. `poll` removes 1, the oldest element.
- Lines 26–38: on an empty deque, `peek` and `poll` return `null`, `pop` throws, and `push(null)` throws `NullPointerException`. `ArrayDeque` forbids `null` elements because `null` is the value that `poll` and `peek` use to say "empty"; a stored `null` would be impossible to tell apart from an empty deque.
- Lines 40–46: the legacy `Stack` class prints from the *bottom* up, and it happily accepts `add(0, 99)`, which inserts at the bottom of a "stack". The next section explains why that matters.

Inside ArrayDeque. In OpenJDK 21, `ArrayDeque` stores its elements in an `Object[]` used as a circular array, with a `head` index and a `tail` index. `addFirst` (and so `push`) moves `head` one step *backwards* with wrap-around and stores the element there; `addLast` (and so `offer`) stores at `tail` and moves it forward. A new `ArrayDeque()` starts with 17 slots; when the array is full it doubles while small and then grows by 50% at a time. Those numbers are implementation details that can change between versions. What the API documentation *guarantees* is the behavior: most operations run in amortized constant time, while `remove(Object)`, `contains`, `removeFirstOccurrence`, `removeLastOccurrence` and iterator removal are linear.

> **NOTE · Java 21: sequenced collections**
> Since Java 21, `Deque` extends the new `SequencedCollection` interface, which adds `reversed()`. For example, `stack.reversed()` gives a view of a stack from the bottom up, which is handy for printing. Methods such as `getFirst()` and `getLast()` already existed on `Deque`.

10.3.1 Why not java.util.Stack?

Java has had a class called `Stack` since version 1.0, and many old tutorials use it. Its own documentation now says that the `Deque` interface "should be used in preference to this class". There are three concrete reasons:

1. **It is a `Vector`.** `Stack` extends `Vector`, an early growable array whose methods are all `synchronized`, which means they take a lock on every call so that several threads can share it. A single-threaded algorithm pays for that locking and gains nothing.
2. **It breaks the stack rule.** Because `Stack` inherits every `Vector` method, code can call `add(0, x)`, `remove(index)` or `get(index)` on it, as `DequeTour` did. A data structure that allows those operations is not really a stack.
3. **Its iteration order is surprising.** Iterating over a `Stack` (or printing it) goes from the bottom up, while an `ArrayDeque` used with `push` goes from the top down. Code that switches between them can silently change behavior.

`LinkedList` also implements `Deque`, so it can act as a stack or queue too. It allows `null` elements and allocates a node object for every element, so it uses more memory, and the `ArrayDeque` documentation notes that `ArrayDeque` is likely to be faster when used as a queue.

Table 10.7: ArrayDeque, Stack and LinkedList compared

	`ArrayDeque`	`Stack`	`LinkedList`
Structure	circular array	growable array (`Vector`)	doubly linked nodes
Thread-safe	no	yes (synchronized)	no
`null` elements	not allowed	allowed	allowed
Index access (`get(i)`)	no	yes (breaks LIFO)	yes, O(n)
Iteration order as a stack	top to bottom	bottom to top	top to bottom
Recommended for stacks and queues	**yes**	no	rarely

WARNING · Comparing boxed values with ==

A `Deque<Integer>` stores `Integer` objects, not `int` values. The fragment below looks right but compares two *references*:

Listing 10.9 Fragment (not a complete program)

```
if (stackA.peek() == stackB.peek()) { ... }       // wrong: compares objects
if (stackA.peek().equals(stackB.peek())) { ... }  // right: compares values
int a = stackA.peek();                            // also right: unbox first,
if (a == stackB.peek()) { ... }                   // then int == Integer unboxes
```

The wrong version often *passes small tests*, because Java caches `Integer` objects for values from −128 to 127, and fails on larger values. Chapter 5 explains the cache. When at least one side of `==` is a primitive `int`, Java unboxes the other side and compares values.

INTERVIEW TIP · What to say when you declare a stack

Write `Deque<Integer> stack = new ArrayDeque<>();` and, if the interviewer seems interested, add one sentence: "I'm using `ArrayDeque` instead of `Stack` because `Stack` is a synchronized `Vector` and isn't recommended any more." It shows you know the library, and it costs five seconds.

10.4 Balanced brackets

The problem. Given a string, decide whether its brackets `()`, `[]` and `{}` are balanced: every opening bracket is closed by the *same kind* of bracket, in the right order. Other characters are ignored. For example, `"([]{})"` and `"a[b(c)d]e"` are balanced; `"([)]"`, `"(()"` and `"())"` are not. This is one of the most common first questions in coding interviews (LeetCode 20, *Valid Parentheses*, is the standard version).

A brute-force idea. In a balanced string of brackets, some pair like `()` must appear side by side. Delete it, and the rest is still balanced. So we can keep deleting adjacent pairs until nothing changes, and check whether the string became empty.

Listing 10.10 Method BalancedParentheses.java

```
/** Brute force: keep deleting adjacent pairs "()", "[]", "{}" until none remain. */
static boolean isBalancedByErasing(String brackets) {
    String current = brackets;
    while (true) {
        String next = current.replace("()", "").replace("[]", "").replace("{}", "");
        if (next.equals(current)) {
            return current.isEmpty();
        }
        current = next;
    }
}
```

This is correct, but slow. Each pass scans the whole string, O(n), and a string like `"((((...))))"` loses only one pair per pass, so there can be about n/2 passes. The total is $O(n^2)$.

Finding the fast idea. Look at what the brute force deletes: always a closing bracket together with the *nearest unclosed* opening bracket before it. So, while scanning from left to right, we only ever need to know the most recent opening bracket that has not been closed yet. "The most recent unfinished thing" is exactly what a stack gives us.

> **KEY IDEA · Bracket matching**
> Push every opening bracket. When a closing bracket arrives, the top of the stack must be the matching opening bracket; pop it. At the end, the stack must be empty.

Listing 10.11 Method — BalancedParentheses.java

```
1  static boolean isBalanced(String text) {
2      Deque<Character> open = new ArrayDeque<>();  // brackets not yet closed
3      for (int i = 0; i < text.length(); i++) {
4          char ch = text.charAt(i);
5          if (ch == '(' || ch == '[' || ch == '{') {
6              open.push(ch);
7          } else if (ch == ')' || ch == ']' || ch == '}') {
8              if (open.isEmpty() || open.pop() != matchingOpen(ch)) {
9                  return false;  // nothing to close, or the wrong kind of bracket
10             }
11         }
12         // any other character is ignored
13     }
14     return open.isEmpty();  // leftover openers were never closed
15 }
16
17 private static char matchingOpen(char close) {
18     return switch (close) {
19         case ')' -> '(';
20         case ']' -> '[';
21         default -> '{';
22     };
23 }
```

Syntax breakdown

- `Deque<Character> open` (line 2) is a deque of `Character` objects. Generic types cannot hold primitives such as `char`, so Java uses the wrapper class `Character`.
- `open.push(ch)` (line 6) passes a `char` where a `Character` is expected; Java converts it automatically. This is called *autoboxing*.
- `open.pop() != matchingOpen(ch)` (line 8) compares a `Character` (from `pop`) with a `char` (from `matchingOpen`). Because one side is a primitive, Java *unboxes* the `Character` and compares the two characters by value, which is what we want.
- `open.isEmpty() || open.pop() != ...` relies on *short-circuit evaluation*: if the stack is empty, `||` already knows the result is `true` and never calls `pop()`, which would otherwise throw an exception.
- Lines 18–22 use a *switch expression* (Java 14 and later): `case ')' -> '(';` means "if `close` is `')'`, the value of the whole switch is `'('`". The `default` case covers `'}'`. Chapter 23 covers switch expressions fully.

Line by line. The loop visits every character (line 3). An opening bracket is pushed (lines 5–6). A closing bracket is checked (lines 7–10): if nothing is open, or the most recent open bracket is of a different kind, the string is not balanced and the method returns immediately. Any other character falls through both `if` tests and is ignored (line 12). After the loop, the string is balanced only if no opening bracket is still waiting (line 14).

Dry run. First the balanced string `"([]{})"`:

Table 10.8: `isBalanced("([]{})")`

i	ch	Action	Stack (bottom → top)
0	(	push	(
1	[	push	([
2	]	pop [, it matches	(
3	{	push	({

i	ch	Action	Stack (bottom → top)
4	}	pop {, it matches	(
5	)	pop (, it matches	(empty)
end		stack empty: return `true`	

Now `"([)]"`: after pushing `(` and `[`, the third character `)` pops `[`, which does not match, so the method returns `false` at index 2 without reading the rest.

Listing 10.12 Output `BalancedParentheses`

```
"([]{})"     true
"([)]"       false
"(()"        false
"())"        false
""           true
"a[b(c)d]e"  true
```

Complexity. Each character is pushed at most once and popped at most once, so the time is O(n). In the worst case, a string of only opening brackets, all n characters are on the stack, so the extra space is O(n).

Edge cases and common mistakes

- *Calling `pop()` on an empty stack.* A string that starts with a closing bracket, such as `")("`, must return `false`, not crash. The `isEmpty()` test protects line 8.
- *Forgetting the final check.* `"(("` never triggers a mismatch inside the loop; only `return open.isEmpty()` catches it.
- *Counting instead of matching.* Counting opens and closes gets `"([)]"` wrong: the counts balance, but the order does not. A counter is enough only when there is a single bracket type (see the interview follow-ups below).
- *The empty string* is balanced: nothing is left open.

> **INTERVIEW TIP** · **Follow-up questions to expect**
> **"What if there is only one kind of bracket?"** Then a single integer `depth` replaces the stack: add 1 for `(`, subtract 1 for `)`, fail if it ever goes below 0, and require 0 at the end. That uses O(1) extra space. **"Can you report where the error is?"** Push indices instead of characters; Exercise 10.2 asks for this. **"What if the input is a stream too large for memory?"** The algorithm reads each character once, left to right, so it works on a stream; only the stack of open brackets must fit in memory.

10.5 Evaluating expressions with stacks

Stacks are the classic tool for evaluating arithmetic expressions. They also explain how the JVM itself runs your code: the bytecode instruction `iadd`, for example, pops two `int` values from an *operand stack* and pushes their sum (Chapter 5).

10.5.1 Postfix notation

> **DEFINITION** · **Postfix notation**
> In postfix notation, also called *Reverse Polish notation* (RPN), each operator comes *after* its two operands. The usual form, `3 + 4`, is called infix notation; its postfix form is `3 4 +`. The infix expression `(3 + 4) * 2` becomes `3 4 + 2 *`.

Postfix needs no parentheses and no precedence rules: the order of the tokens alone says what to compute. That makes it easy to evaluate with a stack. Read the tokens from left to right; push every number; when an operator arrives, pop two numbers, apply the operator, and push the result. (LeetCode 150, *Evaluate Reverse Polish Notation*, is this problem.)

Listing 10.13 Method EvaluateRpn.java

```
1  static int evaluate(String[] tokens) {
2      Deque<Integer> operands = new ArrayDeque<>();
3      for (String token : tokens) {
4          switch (token) {
5              case "+", "-", "*", "/" -> {
6                  int right = operands.pop();  // the top is the RIGHT operand
7                  int left = operands.pop();
8                  operands.push(apply(token, left, right));
9              }
10             default -> operands.push(Integer.parseInt(token));
11         }
12     }
13     if (operands.size() != 1) {
14         throw new IllegalArgumentException("not a complete expression");
15     }
16     return operands.pop();
17 }
18
19 private static int apply(String operator, int left, int right) {
20     return switch (operator) {
21         case "+" -> left + right;
22         case "-" -> left - right;
23         case "*" -> left * right;
24         case "/" -> left / right;  // Java int division rounds toward zero
25         default -> throw new IllegalArgumentException("unknown operator " + operator);
26     };
27 }
```

Syntax breakdown

- `for (String token : tokens)` (line 3) is the enhanced `for` loop: it visits each element of the array in order.
- `switch (token)` (line 4) switches on a `String`, comparing with `equals`, not `==`. `case "+", "-", "*", "/" -> { ... }` (line 5) lists several labels for one branch, and the braces allow several statements in an arrow branch.
- `Integer.parseInt(token)` (line 10) converts text such as `"-7"` to the `int` −7. It throws `NumberFormatException` for text that is not a number. A token like `"-7"` is a number, not the minus operator, because the `case` labels match only the exact string `"-"`.
- `int right = operands.pop();` (line 6) unboxes the `Integer` returned by `pop` into an `int`.

The order of operands. The first value popped is the *right* operand (line 6), and the second is the *left* operand (line 7). For `+` and `*` the order does not matter, but `10 3 -` must compute `10 - 3`, not `3 - 10`. Swapping these two lines is the most common bug in this problem.

Dry run of `5 1 2 + 4 * + 3 -`, which is the infix expression `5 + (1 + 2) * 4 - 3`:

Table 10.9: Evaluating 5 1 2 + 4 * + 3 -

Token	Action	Stack (bottom → top)
5	push 5	5
1	push 1	5 1
2	push 2	5 1 2
+	pop 2 and 1, push 1 + 2 = 3	5 3
4	push 4	5 3 4
*	pop 4 and 3, push 3 × 4 = 12	5 12
+	pop 12 and 5, push 5 + 12 = 17	17
3	push 3	17 3
-	pop 3 and 17, push 17 − 3 = 14	14

Listing 10.14 Output EvaluateRpn

```
3 4 + 2 *           = 14
5 1 2 + 4 * + 3 -   = 14
7 2 /               = 3
-7 2 /              = -3
```

The last two lines show how Java's `int` division works: it rounds toward zero, so `7 / 2` is 3 and `-7 / 2` is −3, not −4. If a problem needs rounding toward negative infinity, use `Math.floorDiv`. The method takes O(n) time and O(n) space for n tokens.

10.5.2 Infix expressions with two stacks

Infix expressions need two more ideas: **precedence** (* and / bind more tightly than + and -) and **parentheses**. A classic method, a simplified form of Edsger Dijkstra's *shunting-yard algorithm*, uses two stacks: one for numbers and one for operators. The rules are:

1. A number is pushed onto the number stack.
2. `(` is pushed onto the operator stack.
3. `)` applies operators until the matching `(` is on top, then discards the `(`.
4. An operator first applies every operator on the stack that has the same or higher precedence (stopping at a `(`), then pushes itself.
5. At the end, apply everything that is left.

"Apply" means: pop an operator and two numbers, compute, and push the result.

Listing 10.15 Method InfixCalculator.java

```
static long evaluate(String expression) {
    Deque<Long> values = new ArrayDeque<>();
    Deque<Character> operators = new ArrayDeque<>();
    int i = 0;
    while (i < expression.length()) {
        char ch = expression.charAt(i);
        if (ch == ' ') {
            i++;
        } else if (ch >= '0' && ch <= '9') {
            long number = 0;
            while (i < expression.length()
                    && expression.charAt(i) >= '0' && expression.charAt(i) <= '9') {
                number = number * 10 + (expression.charAt(i) - '0');
                i++;
            }
            values.push(number);
        } else if (ch == '(') {
            operators.push(ch);
            i++;
        } else if (ch == ')') {
            while (operators.peek() != '(') {
                applyTop(values, operators);
            }
            operators.pop();  // discard the '('
            i++;
        } else {  // one of + - * /
            while (!operators.isEmpty() && operators.peek() != '('
                    && precedence(operators.peek()) >= precedence(ch)) {
                applyTop(values, operators);  // earlier operator binds first
            }
            operators.push(ch);
            i++;
        }
    }
    while (!operators.isEmpty()) {
        applyTop(values, operators);
    }
    return values.pop();
}

```

```
41 private static int precedence(char operator) {
42     return (operator == '*' || operator == '/') ? 2 : 1;
43 }
44
45 private static void applyTop(Deque<Long> values, Deque<Character> operators) {
46     char operator = operators.pop();
47     long right = values.pop();
48     long left = values.pop();
49     long result = switch (operator) {
50         case '+' -> left + right;
51         case '-' -> left - right;
52         case '*' -> left * right;
53         default -> left / right;
54     };
55     values.push(result);
56 }
```

How the code follows the rules. Lines 9–16 read a whole multi-digit number: each new digit multiplies the number so far by 10 and adds the digit's value. The expression `expression.charAt(i) - '0'` turns a digit character into its value, because the characters `'0'` to `'9'` have consecutive codes. Lines 17–19 implement rule 2, lines 20–25 rule 3, lines 26–32 rule 4, and lines 35–37 rule 5. `applyTop` (lines 45–56) pops the right operand before the left one, for the same reason as in the postfix evaluator.

Why "same or higher" precedence? The `>=` on line 28 makes operators of equal precedence apply from left to right. For `10 - 4 - 3`, when the second `-` arrives, the first `-` is applied immediately, giving `(10 - 4) - 3 = 3`. With `>` instead, the code would compute `10 - (4 - 3) = 9`, which is wrong.

Dry run of `8 / (3 - 1) * 5`:

Table 10.10: Evaluating 8 / (3 - 1) * 5 (stacks shown bottom → top)

Read	Numbers	Operators	What happened
8	8		push 8
/	8	/	operator stack empty: push /
(	8	/ (	push (
3	8 3	/ (	push 3
-	8 3	/ (-	top is (: just push -
1	8 3 1	/ (-	push 1
)	8 2	/	apply 3 - 1 = 2, discard (
*	4	*	/ has equal precedence: apply 8 / 2 = 4, then push *
5	4 5	*	push 5
end	20		apply 4 * 5 = 20

Listing 10.16 Output `InfixCalculator`

```
2 + 3 * 4         = 14
(2 + 3) * 4       = 20
10 - 4 - 3        = 3
8 / (3 - 1) * 5   = 20
```

Every character is read once, and every number and operator is pushed and popped at most once, so evaluation is O(n) time and O(n) space.

> **WARNING · Character.isDigit accepts more than 0-9**
> The code tests `ch >= '0' && ch <= '9'` instead of `Character.isDigit(ch)`. The library method returns `true` for *every* Unicode decimal digit, including the Bengali digits ০ to ৯. For such a character, `ch - '0'` is a large meaningless number. When you parse input that should contain only ASCII digits, test the range explicitly.

Limitations. This calculator does not handle unary minus (`-3 + 4`), and malformed input causes exceptions rather than clear error messages. For example, an unmatched `)` makes `operators.peek()` return `null` on

line 21, and comparing `null` with `'('` throws `NullPointerException` during unboxing. A production parser would validate its input; in an interview, state these limits out loud and ask whether the input is guaranteed to be valid.

10.6 Monotonic stacks

This section introduces one of the most useful patterns in coding interviews. It turns a family of $O(n^2)$ problems into $O(n)$ ones.

The problem. For each element of an array, find the *next greater element*: the first value to its right that is strictly larger. If there is none, the answer is −1. For `[2, 7, 3, 5, 4, 6, 8]`, the answers are `[7, 8, 5, 6, 6, 8, -1]`.

Brute force. For each position, scan to the right until a larger value appears.

Listing 10.17 Method NextGreater.java

```
static int[] nextGreaterBrute(int[] values) {
    int[] result = new int[values.length];
    for (int i = 0; i < values.length; i++) {
        result[i] = -1;
        for (int j = i + 1; j < values.length; j++) {
            if (values[j] > values[i]) {
                result[i] = values[j];
                break;
            }
        }
    }
    return result;
}
```

In the worst case, a decreasing array, every inner loop runs to the end: about $n^2/2$ comparisons, so $O(n^2)$.

Finding the fast idea. Scan from left to right and keep a list of positions that are *still waiting* for their answer. Two observations make this list special:

1. When a new value `x` arrives, it is the answer for *every* waiting position whose value is smaller than `x`, because `x` is the first larger value to their right.
2. After those positions are answered and removed, every remaining waiting value is at least `x`. So the waiting values, from oldest to newest, never increase.

Because the waiting values are in non-increasing order, the smaller ones are always the most recent ones. So we only ever need to look at, and remove from, the newest end of the list. That is a stack.

> **DEFINITION · Monotonic stack**
> A monotonic stack is a stack whose elements stay in sorted order from bottom to top: always increasing, or always decreasing. Before pushing a new element, the algorithm pops every element that would break the order. The pops are where the useful work happens.

Listing 10.18 Method NextGreater.java

```
static int[] nextGreater(int[] values) {
    int[] result = new int[values.length];
    Arrays.fill(result, -1);                      // -1 means "no greater value"
    Deque<Integer> waiting = new ArrayDeque<>();  // indices still without an answer
    for (int i = 0; i < values.length; i++) {
        while (!waiting.isEmpty() && values[waiting.peek()] < values[i]) {
            result[waiting.pop()] = values[i];    // values[i] answers that index
        }
        waiting.push(i);
    }
    return result;                                // indices left waiting keep -1
}
```

Line by line. Line 3 fills the result with −1, the answer for any position that never finds a larger value. The stack `waiting` holds *indices*, not values (line 4), because we must know *which* position to answer. For each new index `i`, the `while` loop (lines 6–8) pops every waiting index whose value is smaller than `values[i]` and records `values[i]` as its answer. Then `i` itself starts waiting (line 9). Indices still on the stack at the end keep their −1.

Dry run on `[2, 7, 3, 5, 4, 6, 8]`. The stack column shows `index:value` pairs.

Table 10.11: Next greater element with a monotonic stack

i	values[i]	Popped (and answered)	Stack after push (bottom → top)
0	2	none	0:2
1	7	0 (answer 7)	1:7
2	3	none	1:7, 2:3
3	5	2 (answer 5)	1:7, 3:5
4	4	none	1:7, 3:5, 4:4
5	6	4 (answer 6), 3 (answer 6)	1:7, 5:6
6	8	5 (answer 8), 1 (answer 8)	6:8
end		index 6 keeps −1	

Listing 10.19 Output NextGreater

```
values           [2, 7, 3, 5, 4, 6, 8]
next greater     [7, 8, 5, 6, 6, 8, -1]
brute force      [7, 8, 5, 6, 6, 8, -1]
prev smaller idx [-1, 0, 0, 2, 2, 4, 5]
```

Why it is O(n). The `while` loop is nested inside the `for` loop, so the code *looks* quadratic. But count the work per index instead of per loop: each index is pushed exactly once and popped at most once. So the total number of pops over the whole run is at most n, and the total time is O(n), with O(n) extra space for the stack. This is an amortized argument: a single iteration of the `for` loop can pop many indices, but the total over all iterations is bounded. Interviewers often ask you to explain exactly this.

The four variants. The same skeleton answers four related questions. Only the comparison changes, and whether the answer is recorded for the *popped* index or for the *current* one:

Table 10.12: Monotonic stack variants (strict "greater" and "smaller")

Question, for each position i	Pop while the top's value is...	Record the answer
next greater (to the right)	`< values[i]`	for each popped index: it is `i`
next smaller	`> values[i]`	for each popped index: it is `i`
previous greater (to the left)	`<= values[i]`	for `i`: the top after popping
previous smaller	`>= values[i]`	for `i`: the top after popping

Here is the previous-smaller variant, which the next section needs:

Listing 10.20 Method NextGreater.java

```java
/** Index of the previous strictly smaller value for each position, or -1. */
static int[] previousSmallerIndex(int[] values) {
    int[] result = new int[values.length];
    Deque<Integer> candidates = new ArrayDeque<>();  // values increase toward the top
    for (int i = 0; i < values.length; i++) {
        while (!candidates.isEmpty() && values[candidates.peek()] >= values[i]) {
            candidates.pop();                        // can never be an answer again
        }
        result[i] = candidates.isEmpty() ? -1 : candidates.peek();
        candidates.push(i);
    }
    return result;
}
```

When equal values matter, decide deliberately between < and <=. The table uses *strictly* greater and smaller. If a problem asks for "greater than or equal", flip the equality in the pop condition, and test with repeated values.

10.6.1 Application: the largest rectangle in a histogram

The problem. A histogram has bars of width 1 and heights `heights[0..n-1]`. Find the area of the largest rectangle that fits entirely under the bars. For heights `[2, 1, 5, 6, 2, 3]`, the answer is 10: the bars of height 5 and 6 together hold a rectangle of height 5 and width 2. (This is LeetCode 84, a well-known hard problem.)

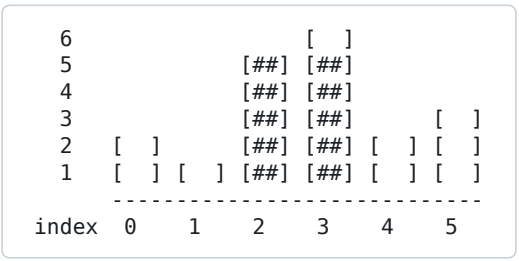


Figure 10.4: Heights [2, 1, 5, 6, 2, 3]; the best rectangle (#) has area 5 × 2 = 10

Brute force. Try every pair `(left, right)` of bar positions; the tallest rectangle spanning them has the height of the shortest bar between them.

Listing 10.21 Method — LargestRectangle.java

```
1  static long largestRectangleBrute(int[] heights) {
2      long best = 0;
3      for (int left = 0; left < heights.length; left++) {
4          int minHeight = Integer.MAX_VALUE;
5          for (int right = left; right < heights.length; right++) {
6              minHeight = Math.min(minHeight, heights[right]);
7              best = Math.max(best, (long) minHeight * (right - left + 1));
8          }
9      }
10     return best;
11 }
```

This is $O(n^2)$: fine for 1,000 bars, too slow for 100,000.

Finding the fast idea. Every optimal rectangle has some bar that is the *shortest* bar inside it, and the rectangle uses that bar's full height. So for each bar `j`, ask: how far can a rectangle of height `heights[j]` stretch? It stretches left until the *previous smaller* bar and right until the *next smaller* bar. Its width is `next - previous - 1`. A monotonic stack with increasing heights gives both limits at once: when bar `j` is popped because a shorter bar `i` arrived, `i` is its next smaller bar, and the element just below `j` on the stack is its previous smaller bar.

Listing 10.22 Method — LargestRectangle.java

```
1  static long largestRectangle(int[] heights) {
2      int n = heights.length;
3      int[] stack = new int[n + 1];  // indices; their heights increase toward the top
4      int top = 0;                   // number of indices on the stack
5      long best = 0;
6      for (int i = 0; i <= n; i++) {
7          int current = (i == n) ? 0 : heights[i];  // a final 0 empties the stack
8          while (top > 0 && heights[stack[top - 1]] >= current) {
9              int height = heights[stack[--top]];  // this bar cannot extend further
10             int leftLimit = (top == 0) ? -1 : stack[top - 1];  // previous smaller bar
11             long width = i - leftLimit - 1;
12             best = Math.max(best, height * width);
13         }
14         stack[top++] = i;
15     }
16     return best;
17 }
```

Line by line. This version uses a plain `int[]` as the stack (line 3) with `top` counting its elements (line 4), which avoids boxing every index into an `Integer`. The loop runs one step *past* the end (line 6); at `i == n` the current height is 0 (line 7), which is shorter than every bar and so pops everything that remains. Inside the `while` loop, `stack[--top]` first decreases `top` and then reads the popped index (line 9). The previous smaller bar is the

new top, or −1 if the stack is now empty (line 10). Line 11 computes the width, and line 12 updates the best area. `height * width` multiplies an `int` by a `long`, so Java computes it as a `long`.

Dry run on `[2, 1, 5, 6, 2, 3]` (stack shows indices):

Table 10.13: Largest rectangle with a monotonic stack

i	current	Popped: height × width	best	Stack after
0	2	none	0	0
1	1	index 0: 2 × (1 − (−1) − 1) = 2	2	1
2	5	none	2	1, 2
3	6	none	2	1, 2, 3
4	2	index 3: 6 × (4 − 2 − 1) = 6; index 2: 5 × (4 − 1 − 1) = 10	10	1, 4
5	3	none	10	1, 4, 5
6	0	index 5: 3 × 1 = 3; index 4: 2 × 4 = 8; index 1: 1 × 6 = 6	10	6

Listing 10.23 Output `LargestRectangle`

```
monotonic stack: 10
brute force:     10
```

The time is O(n), by the same push-once, pop-once argument, and the extra space is O(n).

> **WARNING** · **Overflow in area calculations**
> With up to 10^5 bars of height up to 10^5, an area can reach 10^{10}, which is far larger than the largest `int` (about 2.1×10^9). The code stores `best` and `width` as `long`. Writing `int area = height * (i - leftLimit - 1);` would overflow silently and produce a wrong, possibly negative, answer. The self-check program tests exactly this case.

> **INTERVIEW TIP** · **Recognizing a monotonic-stack problem**
> Look for phrases such as *next greater*, *previous smaller*, *how many days until*, *span*, *visible to the left*, *nearest larger*, or *remove digits to make the smallest number*. If the brute force scans outward from each element until it meets something larger or smaller, a monotonic stack can usually do it in O(n). When you present the solution, state the invariant ("the stack holds indices whose values decrease from bottom to top") and the amortized argument ("each index is pushed and popped at most once").

10.7 Monotonic queues: the sliding-window maximum

The problem. Given an array and a window size `k`, return the maximum of every window of `k` consecutive elements. For `[1, 3, -1, -3, 5, 3, 6, 7]` and `k = 3`, the windows are `[1, 3, -1]`, `[3, -1, -3]`, `[-1, -3, 5]`, `[-3, 5, 3]`, `[5, 3, 6]` and `[3, 6, 7]`, and their maxima are `[3, 3, 5, 5, 6, 7]`. (LeetCode 239.)

Brute force. Scan each window: O(k) per window and O(nk) in total. With $n = 10^5$ and $k = 5 \times 10^4$, that is billions of operations.

Listing 10.24 Method `SlidingWindowMax.java`

```
1  static int[] windowMaximaBrute(int[] values, int k) {
2      int[] result = new int[values.length - k + 1];
3      for (int start = 0; start + k <= values.length; start++) {
4          int max = values[start];
5          for (int i = start + 1; i < start + k; i++) {
6              max = Math.max(max, values[i]);
7          }
8          result[start] = max;
9      }
10     return result;
11 }
```

A heap (Chapter 15) can do it in O(n log k). A deque does it in O(n).

Finding the fast idea. Suppose the window contains an element at index `j` and a later element at index `i > j` with `values[i] >= values[j]`. Then `j` can *never* be the maximum of any future window: every window that contains `j` and is still to come also contains `i`, which is at least as large, and `i` stays in the window longer. So `j` can be thrown away. After throwing away all such elements, the remaining indices have values that strictly *decrease* from oldest to newest, and the maximum is the oldest one.

> **DEFINITION · Monotonic queue**
> A monotonic queue is a deque whose elements stay in sorted order from front to back. New elements enter at the back after removing the elements they dominate; elements leave from the front when they expire. The front is always the best element of the current window.

Listing 10.25 Method — SlidingWindowMax.java

```
1  static int[] windowMaxima(int[] values, int k) {
2      if (k <= 0 || k > values.length) {
3          throw new IllegalArgumentException("need 1 <= k <= values.length");
4      }
5      int[] result = new int[values.length - k + 1];
6      Deque<Integer> window = new ArrayDeque<>();  // indices, values decreasing
7      for (int i = 0; i < values.length; i++) {
8          if (!window.isEmpty() && window.peekFirst() <= i - k) {
9              window.pollFirst();                    // front index left the window
10         }
11         while (!window.isEmpty() && values[window.peekLast()] <= values[i]) {
12             window.pollLast();                     // can never be a maximum
13         }
14         window.offerLast(i);
15         if (i >= k - 1) {
16             result[i - k + 1] = values[window.peekFirst()];
17         }
18     }
19     return result;
20 }
```

Line by line. The method first rejects impossible window sizes (lines 2–4). There are `n - k + 1` windows, so that is the size of the result (line 5). The deque holds indices (line 6). For each new index `i`:

- lines 8–10 remove the front index if it has fallen out of the window. The window ending at `i` covers indices `i - k + 1` to `i`, so an index `<= i - k` is too old. At most one index can expire per step, so an `if` is enough;
- lines 11–13 remove from the *back* every index whose value is at most `values[i]`;
- line 14 adds `i` at the back;
- once the first full window has been seen (`i >= k - 1`, line 15), the front holds the maximum of the window ending at `i` (line 16).

Dry run with `k = 3` (the deque shows `index:value`, front first):

Table 10.14: Sliding-window maximum with a monotonic deque

i	values[i]	Expired from front	Removed from back	Deque after	Output
0	1			0:1	
1	3		0:1	1:3	
2	−1			1:3, 2:−1	3
3	−3			1:3, 2:−1, 3:−3	3
4	5	1:3	3:−3, 2:−1	4:5	5
5	3			4:5, 5:3	5
6	6		5:3, 4:5	6:6	6
7	7		6:6	7:7	7

Listing 10.26 Output SlidingWindowMax

```
values [1, 3, -1, -3, 5, 3, 6, 7], k = 3
maxima [3, 3, 5, 5, 6, 7]
brute  [3, 3, 5, 5, 6, 7]
```

Complexity. Each index enters the deque once and leaves at most once, from either end. So the total work is O(n), and the deque holds at most k indices: O(k) extra space, plus the output array.

Common mistakes

- *Storing values instead of indices.* With values alone, you cannot tell when the front element has left the window.
- *Forgetting the expiry check,* or using `< i - k` instead of `<= i - k`, which keeps an element one step too long.
- *Writing the result at index `i`* instead of `i - k + 1`.
- *Using `<` instead of `<=` on line 11.* Both give correct maxima, because an equal, newer value is as good as an older one. With `<=`, duplicates are removed earlier and the deque stays smaller.

10.8 Queues in breadth-first search (a preview)

Chapter 17 studies graphs in depth. This section shows the reason queues matter there.

The problem. A grid has open cells `.` and walls `#`. Starting at the top-left cell and moving up, down, left or right, what is the smallest number of steps to reach the bottom-right cell? If it cannot be reached, return −1.

The idea. Breadth-first search (BFS) explores the grid in *rings*: first the start, then every cell 1 step away, then every cell 2 steps away, then 3, and onward. A queue produces exactly this order. All cells at distance d enter the queue before any cell at distance d + 1, because each cell at distance d + 1 is discovered while processing a cell at distance d. So the first time BFS reaches a cell, it has found a shortest path to it.

Listing 10.27 Method GridBfs.java

```
static int shortestPath(char[][] grid) {
    int rows = grid.length;
    int cols = grid[0].length;
    if (grid[0][0] == '#' || grid[rows - 1][cols - 1] == '#') {
        return -1;
    }
    int[][] distance = new int[rows][cols];
    for (int[] row : distance) {
        Arrays.fill(row, -1);                       // -1 means "not reached yet"
    }
    Deque<int[]> queue = new ArrayDeque<>();        // cells as {row, col}
    distance[0][0] = 0;
    queue.offer(new int[] {0, 0});
    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        int row = cell[0];
        int col = cell[1];
        if (row == rows - 1 && col == cols - 1) {
            return distance[row][col];              // first arrival is the shortest
        }
        for (int d = 0; d < 4; d++) {
            int nextRow = row + ROW_STEP[d];
            int nextCol = col + COL_STEP[d];
            boolean inside = nextRow >= 0 && nextRow < rows
                    && nextCol >= 0 && nextCol < cols;
            if (inside && grid[nextRow][nextCol] == '.'
                    && distance[nextRow][nextCol] == -1) {
                distance[nextRow][nextCol] = distance[row][col] + 1;  // mark now
                queue.offer(new int[] {nextRow, nextCol});
            }
        }
    }
    return -1;                                      // the target is unreachable
}
```

Key details. The `distance` array (lines 7–10) does two jobs: it stores each cell's distance and, with −1, marks cells that have not been reached. Cells are stored in the queue as two-element arrays `{row, col}` (line 11); a small record class would also work (Chapter 3). A neighbour's distance is set *when it is added to the queue* (line 28), not when it is removed. Marking early guarantees that no cell is added twice. The two arrays `ROW_STEP` and `COL_STEP`, declared at the top of the class, list the four moves, so a single loop (lines 21–31) handles all directions.

```
grid          distances
. . # .        0  1  #  7
. # . .        1  #  5  6
. . . .        2  3  4  5
# . # .        #  4  #  6
```

Figure 10.5: Distance of each cell from the top-left corner (# is a wall)

Listing 10.28 Output GridBfs

```
shortest path length: 6
```

Complexity. Each of the R × C cells enters the queue at most once and is examined with four neighbours, so BFS takes O(R × C) time and O(R × C) space for the `distance` array and the queue.

> **WARNING · Marking cells too late**
> If you mark a cell as visited only when you *remove* it from the queue, the same cell can be added many times by different neighbours before it is processed. The answer is still correct, but the queue can grow far beyond R × C and the program may run out of time or memory on large grids. Mark when you enqueue.

10.9 Building one structure from another

Two classic interview questions ask you to build a queue using only stacks, and a stack using only queues. They test whether you understand the two orders well enough to convert one into the other, and the first one is a beautiful example of amortized analysis.

10.9.1 A queue from two stacks

Use one stack, the *inbox*, for new elements and a second stack, the *outbox*, for elements that are ready to leave. Popping everything from the inbox and pushing it onto the outbox *reverses* the order, which turns the newest-first order of a stack into the oldest-first order of a queue.

Listing 10.29 Fields and methods QueueWithStacks.java

```
1  private final Deque<T> inbox = new ArrayDeque<>();   // receives new elements
2  private final Deque<T> outbox = new ArrayDeque<>();  // serves the oldest elements
3
4  public void enqueue(T value) {
5      inbox.push(value);
6  }
7
8  public T dequeue() {
9      refillOutboxIfEmpty();
10     if (outbox.isEmpty()) {
11         throw new NoSuchElementException("dequeue from an empty queue");
12     }
13     return outbox.pop();
14 }
15
16 public T peek() {
17     refillOutboxIfEmpty();
18     if (outbox.isEmpty()) {
19         throw new NoSuchElementException("peek at an empty queue");
20     }
21     return outbox.peek();
22 }
```

```
23
24 public boolean isEmpty() {
25     return inbox.isEmpty() && outbox.isEmpty();
26 }
27
28 private void refillOutboxIfEmpty() {
29     if (outbox.isEmpty()) {
30         while (!inbox.isEmpty()) {
31             outbox.push(inbox.pop());  // reverses the order: oldest ends on top
32         }
33     }
34 }
```

The important rule is on line 29: refill the outbox **only when it is empty**. If the outbox still holds elements, they are older than everything in the inbox and must leave first.

Listing 10.30 Output QueueWithStacks

```
dequeue -> a
then    -> b c d
```

Here is what happened. After `a`, `b` and `c` were enqueued, the inbox held them with `c` on top. The first `dequeue` found the outbox empty and moved all three, leaving `a` on top of the outbox, and returned `a`. Then `d` went into the inbox. The next two dequeues returned `b` and `c` straight from the outbox. The last dequeue found the outbox empty again, moved `d`, and returned it.

Cost. One `dequeue` can be expensive: if n elements are waiting in the inbox, it moves all n, which is O(n). But follow a single element through its life: it is pushed onto the inbox once, popped from the inbox once, pushed onto the outbox once, and popped from the outbox once. That is four stack operations per element, ever. So any sequence of n queue operations costs O(n) in total, and each operation is O(1) amortized.

10.9.2 A stack from one queue

To make a queue behave like a stack, keep the newest element at the *front*. After adding a new element at the back, move every older element from the front to the back, behind the new one.

Listing 10.31 Fields and methods StackWithQueue.java

```
 1 private final Queue<T> queue = new ArrayDeque<>();  // queue front = stack top
 2
 3 public void push(T value) {
 4     queue.offer(value);
 5     for (int i = 1; i < queue.size(); i++) {
 6         queue.offer(queue.poll());  // rotate the older elements behind the new one
 7     }
 8 }
 9
10 public T pop() {
11     if (queue.isEmpty()) {
12         throw new NoSuchElementException("pop from an empty stack");
13     }
14     return queue.poll();
15 }
16
17 public T peek() {
18     if (queue.isEmpty()) {
19         throw new NoSuchElementException("peek at an empty stack");
20     }
21     return queue.peek();
22 }
23
24 public boolean isEmpty() {
25     return queue.isEmpty();
26 }
```

The loop on lines 5–7 runs `size - 1` times, so `push` is O(n), while `pop` and `peek` are O(1). The opposite choice (cheap push, expensive pop) is also possible; which one is better depends on which operation the program performs more often.

Listing 10.32 Output StackWithQueue

```
pop order: 4 3 2 1
```

10.10 Choosing the right structure

Table 10.15: Which structure to use

You need to...	Structure	In Java
undo the most recent action; match nested things; do DFS without recursion	stack	`Deque<T> s = new ArrayDeque<>()` with `push`, `pop`, `peek`
process items in arrival order; do BFS	queue	`Deque<T> q = new ArrayDeque<>()` with `offer`, `poll`, `peek`
add or remove at both ends; sliding windows	deque	`ArrayDeque` with the `First` and `Last` methods
answer "next/previous greater or smaller" for every element	monotonic stack	`ArrayDeque` of indices, or an `int[]` with a `top` counter
keep the max or min of a moving window	monotonic queue	`ArrayDeque` of indices
always take the smallest or largest item, not the oldest	priority queue	`PriorityQueue` (Chapter 15)
a stack of primitives in performance-critical code	array stack	`int[]` plus a `top` index

Mistakes to avoid in every stack or queue problem

- Calling `pop`, `remove` or `element` on a possibly empty deque. Check `isEmpty()` first, or use `poll`/`peek` and handle `null`.
- Mixing up ends: using `push` (front) to add and `pollLast` (back) to remove turns a stack into a queue by accident.
- Comparing `Integer` values from a deque with `==`.
- Storing values when the algorithm needs indices (to compute distances or detect expiry).
- Assuming a `while` loop inside a `for` loop must be $O(n^2)$; count pushes and pops instead.

10.11 Summary

Stacks, queues and deques decide *which waiting element comes next*: the newest (stack), the oldest (queue), or either end (deque). Each can be built on a growable array with O(1) amortized operations, or on linked nodes with O(1) worst-case operations but more memory. In Java, `ArrayDeque` is the default choice for all three roles; the legacy `Stack` class should be avoided. Stacks solve bracket matching and expression evaluation, and the *monotonic stack* and *monotonic queue* patterns solve "next greater", "previous smaller" and "sliding-window maximum" problems in linear time, because every element enters and leaves the structure at most once. A queue makes breadth-first search find shortest paths in unweighted grids and graphs.

Table 10.16: Key takeaways

Idea	Remember
Stack	LIFO; `push`, `pop`, `peek` on the front of an `ArrayDeque`
Queue	FIFO; `offer` at the back, `poll` and `peek` at the front
Circular array	indices wrap with `% capacity`; keep `size` to tell full from empty
Doubling	makes array growth O(1) amortized per insertion
`ArrayDeque`	no `null`s; `peek`/`poll` return `null` when empty, `pop`/`remove` throw
Monotonic stack	pop what the new element "beats"; each index pushed and popped once: O(n)
Monotonic queue	drop dominated elements at the back and expired ones at the front
BFS	a queue visits cells in order of distance; mark when you enqueue
Queue from two stacks	refill the outbox only when it is empty; O(1) amortized

10.12 Exercises

Exercise 10.1 EASY Predict the output

Without running it, predict what this fragment prints. Then explain each step in terms of the *front* and *back* of the deque.

Listing 10.33 Fragment (not a complete program)

```
Deque<Integer> d = new ArrayDeque<>();
d.push(1);
d.offer(2);
d.push(3);
d.offerFirst(4);
d.addLast(5);
System.out.println(d);
System.out.println(d.pop() + " " + d.pollLast() + " " + d.peek());
```

Solution →

Exercise 10.2 EASY Where is the error?

Write `int firstError(String text)`. It ignores characters other than the six brackets. If some closing bracket has no matching opening bracket, return its index. Otherwise, if some opening bracket is never closed, return the index of the earliest such bracket. If the brackets are balanced, return −1. For example, `"x(y]z"` gives 3 and `"((a)"` gives 0.

Solution →

Exercise 10.3 EASY A fixed-capacity circular queue

Implement a class `CircularQueue` with a constructor taking the capacity, and methods `boolean enqueue(int value)` (returns `false` if the queue is full), `boolean dequeue()` (returns `false` if it is empty), `int front()` and `int rear()` (each returns −1 if the queue is empty), `isEmpty()` and `isFull()`. Every method must be O(1), and the queue must never grow.

Solution →

Exercise 10.4 MEDIUM Min stack

Design a stack of `int` values that supports `push`, `pop`, `top` and `getMin`, where `getMin` returns the smallest value currently on the stack. All four operations must run in O(1) time.

Solution →

Exercise 10.5 MEDIUM Days until a warmer day

Given daily temperatures, return an array where element `i` is the number of days after day `i` until a strictly warmer temperature, or 0 if there is none. For `[30, 32, 31, 29, 33, 33, 28]` the answer is `[1, 3, 2, 1, 0, 0, 0]`. Aim for O(n).

Solution →

Exercise 10.6 MEDIUM Stock span

Write a class with a method `int next(int price)` that is called once per day with that day's stock price. It returns the *span*: the number of consecutive days, ending today, on which the price was less than or equal to today's price. For the prices 100, 80, 60, 70, 60, 75, 85 the spans are 1, 1, 1, 2, 1, 4, 6. Each call should take O(1) amortized time.

Solution →

Exercise 10.7 MEDIUM Simplify a path

Given an absolute Unix-style path, return its simplest form. A single `.` means the current folder, `..` means the parent folder (at the root it stays at the root), and several slashes in a row count as one. The result starts with one `/`, has single slashes between folder names, and has no trailing slash. For example, `"/a/./b/../../c/"` becomes `"/c"`, and `"/..."` stays `"/..."` because `...` is an ordinary folder name.

Solution →

Exercise 10.8 MEDIUM **Decode a string**

An encoded string uses the rule `k[text]`, meaning `text` repeated `k` times; rules can be nested. Decode it. For example, `"3[a2[c]]"` becomes `"accaccacc"` and `"2[abc]3[cd]ef"` becomes `"abcabccdcdcdef"`. Assume the input is valid, counts are positive and may have several digits, and letters are lowercase.

Solution →

Exercise 10.9 MEDIUM **Asteroid collision**

Asteroids move along a line. A positive value moves right and a negative value moves left; the absolute value is the asteroid's size. When two asteroids meet, the smaller one explodes, and if they are the same size, both explode. Asteroids moving in the same direction never meet. Return the asteroids that remain. For example, `[5, 10, -5]` becomes `[5, 10]` and `[10, 2, -5]` becomes `[10]`.

Solution →

Exercise 10.10 MEDIUM **First negative number in every window**

Given an array and a window size `k`, return, for every window of `k` consecutive elements, the first negative number in that window, or 0 if the window has none. For `[12, -1, -7, 8, -15, 30, 16, 28]` and `k = 3`, the answer is `[-1, -1, -7, -15, -15, 0]`. Aim for O(n).

Solution →

Exercise 10.11 HARD **Remove k digits**

A non-negative integer is given as a string of digits. Remove exactly `k` digits so that the remaining number is as small as possible, and return it without leading zeros (return `"0"` if nothing is left). For example, `"1432219"` with `k = 3` gives `"1219"`, and `"10200"` with `k = 1` gives `"200"`. Explain why your greedy choice is correct.

Solution →

Exercise 10.12 HARD **Sum of subarray minimums**

Given an array of n integers with $1 \le n \le 30{,}000$ and each value between 1 and 30,000, return the sum of `min(subarray)` over all contiguous subarrays, modulo 1,000,000,007. For `[3, 1, 2, 4]` the answer is 17. Aim for O(n), and make sure equal values are not counted twice.

Solution →

Exercise 10.13 MEDIUM **Rotting oranges**

In a grid, 0 is an empty cell, 1 is a fresh orange and 2 is a rotten orange. Every minute, each rotten orange makes its fresh neighbours (up, down, left, right) rotten. Return the number of minutes until no fresh orange remains, or −1 if some fresh orange can never rot. A grid with no fresh oranges needs 0 minutes.

Solution →

Exercise 10.14 CHALLENGE **Maximal rectangle in a binary matrix**

Given a matrix of `'0'` and `'1'` characters, find the area of the largest rectangle that contains only `'1'`s. Hint: for each row, how tall is the column of consecutive `'1'`s ending at that row in each column? Reuse an algorithm from this chapter. Aim for O(rows × cols).

Solution →

Exercise 10.15 MEDIUM **Amortized cost, carefully**

For the queue built from two stacks: (a) give a sequence of operations in which a single `dequeue` performs n stack pops and n stack pushes; (b) prove that any sequence of m queue operations, starting from an empty queue, performs at most 4m stack operations in total, not counting the `isEmpty` checks.

Solution →

10.13 Solutions

10.13.1 Warm-up quiz answers

1. **3**, because the last value pushed is the first popped (LIFO).
2. **1**, because the first value added is the first removed (FIFO).

3. `peek()` returns `null`; `pop()` throws `NoSuchElementException`.
4. A single push can cost O(n) when it triggers a resize, but over any sequence of n pushes the total work is O(n), so the *average* per push is O(1). "Amortized" describes this average over a sequence, not the cost of each individual operation.
5. Typical answers: matching brackets or HTML tags, undo, the back button, evaluating nested expressions, and returning from nested method calls.

Solution 10.1

← Exercise

The deque changes as follows (front on the left): `push(1)` adds at the front: `[1]`. `offer(2)` adds at the back: `[1, 2]`. `push(3)` adds at the front: `[3, 1, 2]`. `offerFirst(4)` adds at the front: `[4, 3, 1, 2]`. `addLast(5)` adds at the back: `[4, 3, 1, 2, 5]`. So the first line prints `[4, 3, 1, 2, 5]`. Then `pop()` removes the front, 4; `pollLast()` removes the back, 5; `peek()` looks at the new front, 3. The second line prints `4 5 3`. The file `solutions/ch10/DequeTrace.java` runs the fragment:

Listing 10.34 Output — DequeTrace

```
[4, 3, 1, 2, 5]
4 5 3
```

The lesson: `push` and `offer` work on *different ends*. Mixing them on one deque is legal but almost always a bug.

Solution 10.2

← Exercise

Push *indices* of opening brackets instead of the brackets themselves; the character can always be read back from the string. A plain `int[]` works as the stack because there can never be more than `text.length()` open brackets.

Listing 10.35 Method — FirstBracketError.java

```
static int firstError(String text) {
    int[] openIndex = new int[text.length()];  // stack of indices of open brackets
    int top = 0;
    for (int i = 0; i < text.length(); i++) {
        char ch = text.charAt(i);
        if (ch == '(' || ch == '[' || ch == '{') {
            openIndex[top++] = i;
        } else if (ch == ')' || ch == ']' || ch == '}') {
            if (top == 0 || !matches(text.charAt(openIndex[top - 1]), ch)) {
                return i;                  // a closer with no matching opener
            }
            top--;
        }
    }
    return top == 0 ? -1 : openIndex[0];   // earliest opener never closed
}

private static boolean matches(char open, char close) {
    return (open == '(' && close == ')') || (open == '[' && close == ']')
            || (open == '{' && close == '}');
}
```

A mismatched closing bracket is reported as soon as it is found. If the loop finishes with brackets still open, the earliest unclosed one is at the *bottom* of the stack, `openIndex[0]`, because the stack keeps open brackets in the order they appeared.

Listing 10.36 Output — FirstBracketError

```
"a(b[c]d)e"  -1
"x(y]z"      3
"((a)"       0
"a)b("       1
""           -1
```

Time O(n), extra space O(n).

Solution 10.3

← Exercise

This is `IntQueue` without growth. Because the capacity is fixed, `enqueue` reports a full queue instead of resizing. The back element sits `size - 1` places after `head`.

Listing 10.37 Fields and methods — CircularQueue.java

```
private final int[] items;
private int head;  // index of the front element
private int size;

public CircularQueue(int capacity) {
    if (capacity <= 0) {
        throw new IllegalArgumentException("capacity must be positive");
    }
    items = new int[capacity];
}

/** Adds value at the back; returns false if the queue is full. */
public boolean enqueue(int value) {
    if (isFull()) {
        return false;
    }
    items[(head + size) % items.length] = value;
    size++;
    return true;
}

/** Removes the front value; returns false if the queue is empty. */
public boolean dequeue() {
    if (isEmpty()) {
        return false;
    }
    head = (head + 1) % items.length;
    size--;
    return true;
}

public int front() {
    return isEmpty() ? -1 : items[head];
}

public int rear() {
    return isEmpty() ? -1 : items[(head + size - 1) % items.length];
}

public boolean isEmpty() {
    return size == 0;
}

public boolean isFull() {
    return size == items.length;
}
```

Listing 10.38 Output — CircularQueue

```
true true true false
rear 3, full true
dequeue true, enqueue 4 true
front 2, rear 4
```

Keeping `size` makes `isEmpty` and `isFull` trivial. Without it, `head == tail` would be ambiguous, as explained in the warning in Section 10.2.1.

Solution 10.4

← Exercise

Store, with every value, the minimum of the stack *at the moment that value was pushed*. The minimum of the whole stack is then simply the stored minimum of the top entry, and popping automatically restores the previous minimum.

Listing 10.39 Fields and methods — MinStack.java

```
/** Each entry holds a value and the stack's minimum at the moment it was pushed. */
private final Deque<int[]> entries = new ArrayDeque<>();

public void push(int value) {
    int min = entries.isEmpty() ? value : Math.min(value, entries.peek()[1]);
    entries.push(new int[] {value, min});
}

public int pop() {
    requireNotEmpty();
    return entries.pop()[0];
}

public int top() {
    requireNotEmpty();
    return entries.peek()[0];
}

public int getMin() {
    requireNotEmpty();
    return entries.peek()[1];
}

private void requireNotEmpty() {
    if (entries.isEmpty()) {
        throw new NoSuchElementException("empty stack");
    }
}
```

Listing 10.40 Output — MinStack

```
min 3
after pop, min 3
after two more pops, min 5
```

All four operations are O(1), and the space is O(n). A common alternative keeps two stacks: the values, and a second stack that receives a value only when it is less than or equal to the current minimum. That version must compare the popped value with the top of the minimum stack, and if both are `Integer` objects, comparing them with == is the classic bug from the warning in Section 10.3.1. Use `equals`, or unbox to `int` first.

Solution 10.5

← Exercise

This is “next greater element”, but the answer is a *distance* rather than a value, so the stack must hold indices. The stack keeps days whose temperatures decrease from bottom to top; a warmer day pops and answers every cooler day still waiting.

Listing 10.41 Method — DailyTemperatures.java

```
static int[] daysUntilWarmer(int[] temperatures) {
    int n = temperatures.length;
    int[] answer = new int[n];        // 0 by default: "no warmer day"
    int[] waiting = new int[n];       // stack of day indices, temperatures decreasing
    int top = 0;
    for (int day = 0; day < n; day++) {
        while (top > 0 && temperatures[waiting[top - 1]] < temperatures[day]) {
            int earlier = waiting[--top];
            answer[earlier] = day - earlier;
        }
        waiting[top++] = day;
    }
    return answer;
}
```

Listing 10.42 Output — DailyTemperatures

```
[1, 3, 2, 1, 0, 0, 0]
```

Each day is pushed and popped at most once: O(n) time and O(n) space. Day 4 (33 degrees) and day 5 (also 33) both get 0, because the comparison is strictly *warmer*: equal temperatures do not pop each other.

Solution 10.6

← Exercise

Keep a stack of `{price, span}` pairs with prices decreasing from bottom to top. When a new price arrives, every pair with a price less than or equal to it lies inside the new span, together with all the days *that* pair already covered. So pop those pairs and add their spans.

Listing 10.43 Fields and methods — StockSpan.java

```
private final Deque<int[]> history = new ArrayDeque<>();  // {price, span} pairs

public int next(int price) {
    int span = 1;
    while (!history.isEmpty() && history.peek()[0] <= price) {
        span += history.pop()[1];   // absorb the days that the popped entry covered
    }
    history.push(new int[] {price, span});
    return span;
}
```

Listing 10.44 Output — StockSpan

```
1 1 1 2 1 4 6
```

On day 6 (price 85), the stack holds the pairs (100, 1), (80, 1), (75, 4), with 75 on top. The new price pops (75, 4) and (80, 1), so the span is 1 + 4 + 1 = 6. Each pair is pushed once and popped at most once, so a sequence of n calls takes O(n) time: O(1) amortized per call.

Solution 10.7

← Exercise

Split the path on `/`. The pieces are folder names, `.`, `..`, or empty strings (from `//` and from the leading and trailing slashes). Folder names are pushed; `..` pops if the stack is not empty.

Listing 10.45 Method — SimplifyPath.java

```
static String simplify(String path) {
    Deque<String> folders = new ArrayDeque<>();  // used as a stack of folder names
    for (String part : path.split("/")) {
        if (part.isEmpty() || part.equals(".")) {
            continue;                // "//" and "." do not change the location
        }
        if (part.equals("..")) {
            folders.pollFirst();     // go up one level (nothing happens at the root)
```

```
        } else {
            folders.push(part);
        }
    }
    StringBuilder result = new StringBuilder();
    while (!folders.isEmpty()) {
        result.append('/').append(folders.pollLast());  // bottom of the stack first
    }
    return result.isEmpty() ? "/" : result.toString();
}
```

Listing 10.46 Output — SimplifyPath

```
/home/                  -> /home
/a/./b/../../c/         -> /c
/../                    -> /
/home//user/./docs/..   -> /home/user
/...                    -> /...
```

Two details deserve attention. `pollFirst()` returns `null` on an empty deque instead of throwing, which handles `..` at the root for free. And the result must list folders from the *bottom* of the stack upwards, so the loop takes them with `pollLast()`. Time and space are O(n) for a path of length n.

Solution 10.8

← Exercise

Each `[` starts a new, nested piece of text. Save the text built so far and the repeat count on two stacks, and start a fresh builder. At the matching `]`, the finished inner text is repeated and appended to the saved outer text.

Listing 10.47 Method — DecodeString.java

```
static String decode(String encoded) {
    Deque<Integer> counts = new ArrayDeque<>();        // repeat count per open '['
    Deque<StringBuilder> outer = new ArrayDeque<>();   // text built before each '['
    StringBuilder current = new StringBuilder();
    int number = 0;
    for (int i = 0; i < encoded.length(); i++) {
        char ch = encoded.charAt(i);
        if (ch >= '0' && ch <= '9') {
            number = number * 10 + (ch - '0');         // a count may have many digits
        } else if (ch == '[') {
            counts.push(number);
            outer.push(current);
            current = new StringBuilder();             // start the bracketed part
            number = 0;
        } else if (ch == ']') {
            String inner = current.toString();
            current = outer.pop();
            current.append(inner.repeat(counts.pop()));
        } else {
            current.append(ch);
        }
    }
    return current.toString();
}
```

Listing 10.48 Output — DecodeString

```
3[a]2[bc]       -> aaabcbc
3[a2[c]]        -> accaccacc
2[abc]3[cd]ef   -> abcabccdcdcdef
10[x]           -> xxxxxxxxxx
```

`String.repeat(n)` (Java 11 and later) returns the string repeated `n` times. The running time is proportional to the length of the *output*, which can be much longer than the input: `"9[9[9[a]]]"` is 10 characters but decodes to 729.

Solution 10.9

← Exercise

Only one situation causes a collision: an asteroid moving left meets the nearest surviving asteroid to its left that is moving right. Keep the survivors on a stack. While the incoming asteroid moves left and the top moves right, they collide; the loop ends when the incoming asteroid explodes, or nothing on the stack can hit it.

Listing 10.49 Method — AsteroidCollision.java

```
static int[] survivors(int[] asteroids) {
    int[] stack = new int[asteroids.length];  // survivors so far, left to right
    int top = 0;
    for (int asteroid : asteroids) {
        boolean alive = true;
        while (alive && asteroid < 0 && top > 0 && stack[top - 1] > 0) {
            int right = stack[top - 1];           // moving right, meets this one
            if (right < -asteroid) {
                top--;                            // the right-mover explodes
            } else if (right == -asteroid) {
                top--;                            // both explode
                alive = false;
            } else {
                alive = false;                    // the left-mover explodes
            }
        }
        if (alive) {
            stack[top++] = asteroid;
        }
    }
    return Arrays.copyOf(stack, top);
}
```

Listing 10.50 Output — AsteroidCollision

```
[5, 10, -5]      -> [5, 10]
[8, -8]          -> []
[10, 2, -5]      -> [10]
[-2, -1, 1, 2]   -> [-2, -1, 1, 2]
```

Each asteroid is pushed at most once and popped at most once: O(n) time and O(n) space. The last sample shows the “no collision” case: everything moving left is already to the left of everything moving right.

Solution 10.10

← Exercise

Only negative numbers can be answers, and among those in the window, the *earliest* one is the answer. So keep the indices of negative numbers in a queue, in order. Remove the front when it leaves the window; the front is then the answer.

Listing 10.51 Method — FirstNegativeInWindow.java

```
static int[] firstNegatives(int[] values, int k) {
    int[] result = new int[values.length - k + 1];
    Deque<Integer> negatives = new ArrayDeque<>();  // indices of negatives, in order
    for (int i = 0; i < values.length; i++) {
        if (values[i] < 0) {
            negatives.offerLast(i);
        }
        if (!negatives.isEmpty() && negatives.peekFirst() <= i - k) {
            negatives.pollFirst();                  // it has left the window
        }
        if (i >= k - 1) {
            int first = negatives.isEmpty() ? 0 : values[negatives.peekFirst()];
            result[i - k + 1] = first;
        }
    }
    return result;
}
```

Listing 10.52 Output — FirstNegativeInWindow

```
[-1, -1, -7, -15, -15, 0]
```

O(n) time and O(k) extra space. Unlike the sliding-window maximum, nothing is ever removed from the back: every negative number stays a candidate until it expires.

Solution 10.11

← Exercise

Compare two numbers of the same length digit by digit from the left: the first different digit decides. So if some digit is followed by a *smaller* digit, deleting the larger one lets the smaller digit move one place left, and that makes the result smaller. Removing any digit to the right instead can never make up for it. This greedy rule leads to a monotonic stack whose digits never decrease:

Listing 10.53 Method — RemoveKDigits.java

```
static String removeKDigits(String number, int k) {
    StringBuilder kept = new StringBuilder();  // a stack of digits, never decreasing
    int toRemove = k;
    for (int i = 0; i < number.length(); i++) {
        char digit = number.charAt(i);
        while (toRemove > 0 && !kept.isEmpty()
                && kept.charAt(kept.length() - 1) > digit) {
            kept.deleteCharAt(kept.length() - 1);  // a smaller digit can move left
            toRemove--;
        }
        kept.append(digit);
    }
    kept.setLength(kept.length() - toRemove);     // removals still owed: cut the end
    int start = 0;
    while (start < kept.length() - 1 && kept.charAt(start) == '0') {
        start++;                                  // skip leading zeros
    }
    String result = kept.substring(start);
    return result.isEmpty() ? "0" : result;
}
```

Listing 10.54 Output — RemoveKDigits

```
1219
200
0
11
```

If the digits are already non-decreasing (as in `"112"`), no pops happen and the remaining removals are taken from the end, where the largest digits are. Leading zeros are skipped at the end, and an empty result becomes `"0"`. Each digit is appended once and deleted at most once, so the method is O(n). The self-check program compares it with a brute force that tries every possible set of digits to keep.

Solution 10.12

← Exercise

Instead of looking at every subarray, ask, for each element, *in how many subarrays is it the minimum?* If `values[i]` is the minimum, the subarray can start anywhere after the previous smaller element and end anywhere before the next smaller element. With `left[i]` and `right[i]` as those boundaries, the count is `(i - left[i]) × (right[i] - i)`, and the answer is the sum of `values[i] × count` over all i.

Equal values need care. In `[2, 2]`, the whole array has two equal minimums, but it must be counted once. The fix is to make the boundaries asymmetric: `left` uses the previous *strictly smaller* value and `right` uses the next *smaller or equal* value. Then each subarray is assigned to exactly one of its equal minimums, the leftmost one.

Listing 10.55 Method — SumOfSubarrayMinimums.java

```
static int sumOfMinimums(int[] values) {
    int n = values.length;
```

```
    int[] left = new int[n];    // index of previous value that is STRICTLY smaller
    int[] right = new int[n];   // index of next value that is smaller OR EQUAL
    int[] stack = new int[n];
    int top = 0;
    for (int i = 0; i < n; i++) {
        while (top > 0 && values[stack[top - 1]] >= values[i]) {
            top--;
        }
        left[i] = (top == 0) ? -1 : stack[top - 1];
        stack[top++] = i;
    }
    top = 0;
    for (int i = n - 1; i >= 0; i--) {
        while (top > 0 && values[stack[top - 1]] > values[i]) {
            top--;
        }
        right[i] = (top == 0) ? n : stack[top - 1];
        stack[top++] = i;
    }
    long total = 0;
    for (int i = 0; i < n; i++) {
        // values[i] is the minimum of exactly this many subarrays:
        long subarrays = (long) (i - left[i]) * (right[i] - i);
        total = (total + values[i] % MOD * (subarrays % MOD)) % MOD;
    }
    return (int) total;
}
```

Listing 10.56 Output — SumOfSubarrayMinimums

```
17
444
12
```

The product `values[i] × count` can exceed the range of `int` (here it can reach about $3 \times 10^4 \times 2.25 \times 10^8 \approx 6.75 \times 10^{12}$), so the code computes in `long` and reduces both factors modulo 1,000,000,007 before multiplying. The product of two numbers below $10^9 + 7$ fits safely in a `long`. Two monotonic-stack passes and one summing pass give O(n) time.

Solution 10.13

← Exercise

This is breadth-first search with *many* starting points: put every rotten orange in the queue at the start. Process the queue one level at a time; each level is one minute. The variable `levelSize` records how many oranges rotted during the previous minute.

Listing 10.57 Method — RottingOranges.java

```
static int minutesToRot(int[][] grid) {
    int rows = grid.length;
    int cols = grid[0].length;
    int[][] state = new int[rows][];
    Deque<int[]> queue = new ArrayDeque<>();
    int fresh = 0;
    for (int r = 0; r < rows; r++) {
        state[r] = grid[r].clone();             // do not modify the caller's grid
        for (int c = 0; c < cols; c++) {
            if (state[r][c] == 2) {
                queue.offer(new int[] {r, c});  // every rotten orange starts at once
            } else if (state[r][c] == 1) {
                fresh++;
            }
        }
    }
    int[][] steps = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
    int minutes = 0;
    while (!queue.isEmpty() && fresh > 0) {
        int levelSize = queue.size();           // all oranges that rotted last minute
```

```
        for (int i = 0; i < levelSize; i++) {
            int[] cell = queue.poll();
            for (int[] step : steps) {
                int r = cell[0] + step[0];
                int c = cell[1] + step[1];
                if (r >= 0 && r < rows && c >= 0 && c < cols && state[r][c] == 1) {
                    state[r][c] = 2;
                    fresh--;
                    queue.offer(new int[] {r, c});
                }
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes : -1;
}
```

Listing 10.58 Output — RottingOranges

```
4
-1
0
```

The loop stops as soon as no fresh oranges remain, so it does not count an extra, empty minute at the end. Counting `fresh` in advance makes the final check O(1). Time and space are O(rows × cols). The method copies the grid first, because changing an input array that the caller still owns is a side effect that callers do not expect.

Solution 10.14

← Exercise

Process the matrix row by row. For each column, `heights[c]` is the number of consecutive `'1'`s ending at the current row; a `'0'` resets it to 0. Every all-ones rectangle has a bottom row, and when that row is processed, the rectangle fits under the histogram `heights`. So the answer is the largest histogram rectangle over all rows.

Listing 10.59 Method — MaximalRectangle.java

```
static long maximalRectangle(char[][] matrix) {
    if (matrix.length == 0) {
        return 0;
    }
    int cols = matrix[0].length;
    int[] heights = new int[cols];  // consecutive '1's ending at the current row
    long best = 0;
    for (char[] row : matrix) {
        for (int c = 0; c < cols; c++) {
            heights[c] = (row[c] == '1') ? heights[c] + 1 : 0;
        }
        best = Math.max(best, largestRectangle(heights));
    }
    return best;
}

/** Same algorithm as LargestRectangle in code-examples/ch10. */
static long largestRectangle(int[] heights) {
    int n = heights.length;
    int[] stack = new int[n + 1];
    int top = 0;
    long best = 0;
    for (int i = 0; i <= n; i++) {
        int current = (i == n) ? 0 : heights[i];
        while (top > 0 && heights[stack[top - 1]] >= current) {
            int height = heights[stack[--top]];
            int leftLimit = (top == 0) ? -1 : stack[top - 1];
            best = Math.max(best, (long) height * (i - leftLimit - 1));
        }
        stack[top++] = i;
    }
```

```
32      return best;
33  }
```

Listing 10.60 Output `MaximalRectangle`

```
6
```

Each row costs O(cols) to update the heights and O(cols) for the histogram algorithm, so the total is O(rows × cols) time and O(cols) extra space.

Solution 10.15

← Exercise

(a) Enqueue n elements; they all sit in the inbox. The first `dequeue` finds the outbox empty and moves all n: n pops from the inbox and n pushes onto the outbox, plus one final pop.

(b) Charge every stack operation to the element it moves. An element is pushed onto the inbox once, when it is enqueued. It can be moved to the outbox at most once, because nothing ever moves from the outbox back to the inbox: one pop and one push. It is popped from the outbox at most once, when it is dequeued. So each element causes at most four stack operations in its whole life. A sequence of m queue operations enqueues at most m elements, so it performs at most 4m stack operations. A `peek` that triggers a refill moves elements that are charged in the same way, so it does not change the bound. Hence the total cost is O(m), and each operation is O(1) amortized, even though a single operation can cost O(n).

Glossary

Short definitions of the technical terms used in this book, in alphabetical order. The chapter in parentheses is where the term is explained in full.

Algorithm
A precise, finite sequence of steps that solves a problem. (How to Use This Book)

Amortized cost
The average cost per operation over a worst-case *sequence* of operations. An operation can occasionally be expensive (for example, resizing an array) and still be O(1) amortized if the expensive cases are rare enough. (Chapters 6 and 10)

ArrayDeque
Java's resizable, circular-array implementation of the `Deque` interface; the recommended class for stacks and queues. It does not allow `null` elements. (Chapter 10)

Autoboxing
The automatic conversion of a primitive value, such as an `int` or `char`, into its wrapper object, such as `Integer` or `Character`. The reverse conversion is called *unboxing*. (Chapter 5)

BigDecimal
Java's class for exact decimal numbers, stored as an integer value plus a scale. Create values from strings, and compare them with `compareTo`. (Chapter 2)

BigInteger
Java's class for integers of any size. Every operation returns a new object. (Chapter 2)

Breadth-first search (BFS)
A way to explore a graph or grid level by level, using a queue, so that nodes are visited in order of their distance from the start. In unweighted graphs, it finds shortest paths. (Chapters 10 and 17)

Bytecode
The instruction set of the Java Virtual Machine, stored in `.class` files by the compiler. The same bytecode runs on every operating system. (Chapter 1)

Circular buffer
An array used as a ring: an index that moves past the last slot wraps around to slot 0. Also called a *ring buffer*. It lets a queue add and remove in O(1) without shifting elements. (Chapter 10)

Comparator
A functional interface whose method `compare(a, b)` returns a negative number, zero or a positive number to say whether `a` comes before, together with, or after `b`. (Chapters 2 and 3)

Deep copy
A copy that duplicates an object *and* everything it refers to, so that changing the copy never affects the original. For a 2D array, every row must be copied. Compare *shallow copy*. (Chapter 1)

Deque
A *double-ended queue*: a collection that allows insertion, removal and inspection at both ends. Pronounced "deck". (Chapter 10)

Fast reader
An input reader that reads raw bytes in large blocks and converts digits itself, avoiding the overhead of `Scanner`. (Chapter 2)

FIFO (first in, first out)
The rule of a queue: the element that has waited longest is removed first. (Chapter 10)

Functional interface
An interface with exactly one abstract method, such as `Comparator` or `Runnable`. Lambdas and method references create objects of functional interfaces. (Chapters 2 and 22)

Immutable
Describes an object whose state can never change after it is created, such as a `String` or an `Integer`. (Chapters 1 and 3)

Infix notation
The usual way of writing expressions, with each operator between its operands, as in `3 + 4`. (Chapter 10)

Integer overflow
What happens when the exact result of an `int` or `long` operation does not fit in the type: Java keeps only the lowest bits, so the result silently wraps around. (Chapters 1 and 19)

Invariant
A condition that an algorithm keeps true at a fixed point, for example at the start of every loop iteration. Stating the invariant is often the clearest way to explain why an algorithm is correct. (How to Use This Book)

JDK (Java Development Kit)
The runtime (JVM and class library) plus development tools such as `javac`, `javap` and `jshell`. (Chapter 1)

JIT compiler
The *just-in-time* compiler inside the JVM that translates frequently executed bytecode into native machine code while the program runs. (Chapters 1 and 5)

JVM (Java Virtual Machine)
The program that loads `.class` files, verifies their bytecode and executes it. (Chapters 1 and 5)

Lambda expression
A short way to write an implementation of a functional interface's single method: parameters, an arrow, and a body, as in `(a, b) -> Integer.compare(a, b)`. (Chapters 2 and 22)

LIFO (last in, first out)
The rule of a stack: the element added most recently is removed first. (Chapter 10)

Locale
A setting that describes how a language and region write numbers and dates. `String.format` follows it, so pass `Locale.ROOT` when output must not depend on the computer. (Chapter 2)

LTS (long-term support)
A Java release that vendors support for several years. Java 8, 11, 17, 21 and 25 are LTS releases. (Chapter 1)

Method reference
A shorthand for a lambda that only calls one existing method, such as `Integer::compare`, `String::length` or `ArrayList::new`. (Chapter 2)

Monotonic queue
A deque whose elements are kept in sorted order from front to back. New elements remove the elements they dominate from the back; expired elements leave from the front. Used for sliding-window maximum and minimum. (Chapter 10)

Monotonic stack
A stack whose elements are kept in sorted order from bottom to top. Before a push, the elements that would break the order are popped. Used for "next greater" and "previous smaller" problems in O(n). (Chapter 10)

NullPointerException
The exception thrown when a program uses a `null` reference as if it pointed to an object, including when it unboxes `null`. (Chapter 1)

Pass by value
Java's only parameter-passing rule: a method receives copies of its arguments. For objects and arrays, the copied value is a reference. (Chapter 1)

Postfix notation
A way of writing expressions with each operator after its operands, as in `3 4 +`. Also called *Reverse Polish notation* (RPN). It needs no parentheses or precedence rules. (Chapter 10)

Primitive type
One of Java's eight built-in value types: `byte`, `short`, `int`, `long`, `float`, `double`, `char` and `boolean`. A primitive variable holds the value itself. (Chapter 1)

Queue
A collection where elements are added at the back and removed from the front, in first-in, first-out order. (Chapter 10)

Reference type
Any class, interface, array or enum type. A variable of a reference type holds a reference to an object, or `null`. (Chapter 1)

Shallow copy
A copy of an object that shares the objects it refers to with the original. `clone()` on a 2D array is shallow: the rows are shared. (Chapter 1)

Stable sort
A sort that keeps elements that compare as equal in their original relative order. Java's object sorts are guaranteed to be stable. (Chapters 2 and 12)

Stack
A collection where elements are added and removed at the same end, the top, in last-in, first-out order. (Chapter 10)

String pool
The JVM's table of shared `String` objects. Every string literal, and every compile-time constant string, refers to the single pooled object with that text. (Chapter 1)

Widening and narrowing conversions
Converting a number to a type with a larger range (widening, automatic) or a smaller one (narrowing, which needs a cast and may lose information). (Chapter 1)

References

Every web link below was checked (HTTP 200) on the date shown. Official Java documentation is for Java SE 21 unless a different version is named. Where the book describes how OpenJDK *implements* something, the claim refers to the OpenJDK 21 source code and may change in other versions; the specification and API documentation are the guarantees.

Java language, JVM and tools

- Oracle. *The Java Language Specification, Java SE 21 Edition.* https://docs.oracle.com/javase/specs/jls/se21/html/index.html (checked 2026-10-15)
- Oracle. *The Java Virtual Machine Specification, Java SE 21 Edition.* https://docs.oracle.com/javase/specs/jvms/se21/html/index.html (checked 2026-10-15)
- Oracle. *The javac Command* (JDK 21 tool reference). https://docs.oracle.com/en/java/javase/21/docs/specs/man/javac.html (checked 2026-10-15)
- Oracle. *The java Command* (JDK 21 tool reference). https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html (checked 2026-10-15)
- OpenJDK. *JDK 21* project page (release contents and schedule). https://openjdk.org/projects/jdk/21/ (checked 2026-10-15)
- OpenJDK. *JDK 25* project page (release contents and schedule). https://openjdk.org/projects/jdk/25/ (checked 2026-10-15)
- Oracle. *Oracle Java SE Support Roadmap* (LTS releases and support dates). https://www.oracle.com/java/technologies/java-se-support-roadmap.html (checked 2026-10-15)

Java Enhancement Proposals (JEPs)

- JEP 254: Compact Strings (Java 9). https://openjdk.org/jeps/254 (checked 2026-10-15)
- JEP 280: Indify String Concatenation (Java 9). https://openjdk.org/jeps/280 (checked 2026-10-15)
- JEP 330: Launch Single-File Source-Code Programs (Java 11). https://openjdk.org/jeps/330 (checked 2026-10-15)
- JEP 358: Helpful NullPointerExceptions (Java 14). https://openjdk.org/jeps/358 (checked 2026-10-15)
- JEP 400: UTF-8 by Default (Java 18). https://openjdk.org/jeps/400 (checked 2026-10-15)
- JEP 431: Sequenced Collections (Java 21). https://openjdk.org/jeps/431 (checked 2026-10-15)
- JEP 512: Compact Source Files and Instance Main Methods (Java 25). https://openjdk.org/jeps/512 (checked 2026-10-15)

Java language specification chapters used

- JLS 21, Chapter 3 (Lexical Structure: string literals and interning). https://docs.oracle.com/javase/specs/jls/se21/html/jls-3.html (checked 2026-10-15)
- JLS 21, Chapter 5 (Conversions and Contexts: widening, narrowing, boxing). https://docs.oracle.com/javase/specs/jls/se21/html/jls-5.html (checked 2026-10-15)
- JLS 21, Chapter 15 (Expressions: integer division, remainder, evaluation order). https://docs.oracle.com/javase/specs/jls/se21/html/jls-15.html (checked 2026-10-15)

Java API documentation (Java SE 21)

- `java.lang.String`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/String.html (checked 2026-10-15)

- `java.lang.StringBuilder`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/StringBuilder.html (checked 2026-10-15)
- `java.lang.StringBuffer`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/StringBuffer.html (checked 2026-10-15)
- `java.lang.Integer` (including the `valueOf` caching guarantee): https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Integer.html (checked 2026-10-15)
- `java.lang.Math` (`addExact`, `floorMod`, `round`): https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Math.html (checked 2026-10-15)
- `java.util.Arrays`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Arrays.html (checked 2026-10-15)
- `java.util.Objects`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Objects.html (checked 2026-10-15)
- `java.util.Scanner` (describes its regular-expression parsing): https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Scanner.html (checked 2026-10-15)
- `java.io.BufferedReader`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/BufferedReader.html (checked 2026-10-15)
- `java.util.StringTokenizer` (documented as a legacy class): https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/StringTokenizer.html (checked 2026-10-15)
- `java.io.PrintWriter`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/PrintWriter.html (checked 2026-10-15)
- `java.util.Locale`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Locale.html (checked 2026-10-15)
- `java.math.BigInteger`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/math/BigInteger.html (checked 2026-10-15)
- `java.math.BigDecimal`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/math/BigDecimal.html (checked 2026-10-15)
- `java.util.Collections` (stability of `sort`): https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Collections.html (checked 2026-10-15)
- `java.util.Comparator`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Comparator.html (checked 2026-10-15)
- Package `java.util.function` (functional interfaces): https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/function/package-summary.html (checked 2026-10-15)
- *The javap Command* (JDK 21): https://docs.oracle.com/en/java/javase/21/docs/specs/man/javap.html (checked 2026-10-15)
- `java.util.Deque`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Deque.html (checked 2026-10-15)
- `java.util.ArrayDeque`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayDeque.html (checked 2026-10-15)
- `java.util.Queue`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Queue.html (checked 2026-10-15)
- `java.util.Stack`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Stack.html (checked 2026-10-15)
- `java.util.Vector`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Vector.html (checked 2026-10-15)
- `java.util.LinkedList`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/LinkedList.html (checked 2026-10-15)

- `java.util.PriorityQueue`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html (checked 2026-10-15)
- `java.util.NoSuchElementException`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/NoSuchElementException.html (checked 2026-10-15)
- `java.util.concurrent.ArrayBlockingQueue`: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ArrayBlockingQueue.html (checked 2026-10-15)

OpenJDK source code

- OpenJDK. `ArrayDeque.java` in the JDK 21 updates repository (circular array, growth policy). https://github.com/openjdk/jdk21u/blob/master/src/java.base/share/classes/java/util/ArrayDeque.java (checked 2026-10-15)

Algorithms and data structures

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*, 4th edition. MIT Press, 2022.
- Sedgewick, R., and Wayne, K. *Algorithms*, 4th edition. Addison-Wesley, 2011. Companion site: https://algs4.cs.princeton.edu/home/ (checked 2026-10-15); stacks and queues: https://algs4.cs.princeton.edu/13stacks/ (checked 2026-10-15)

Java practice and style

- Bloch, J. *Effective Java*, 3rd edition. Addison-Wesley, 2018.

Online judges

- HackerRank. *Execution Environment* (languages, versions, time and memory limits). https://hackerrank-knowledge-base.help.usepylon.com/articles/6693750503 (checked 2026-10-15; the page said “Last updated 2 months ago”)

Tools used to build this book

- Pandoc User’s Guide. https://pandoc.org/MANUAL.html (checked 2026-10-15)
- Typst documentation. https://typst.app/docs/ (checked 2026-10-15)

Index

A
amortized cost, 57

B
BigDecimal, 35
BigInteger, 35
breadth-first search (BFS), 75
Bytecode, 3

C
circular buffer, 59
Comparator, 41

D
deep copy, 18
deque (double-ended queue), 61

F
fast reader, 29
FIFO (first in, first out), 59
functional interface, 38

I
immutable, 11
infix notation, 66
Integer overflow, 8

J
JIT compiler, 3
JVM (Java Virtual Machine), 3

L
lambda expression, 38
LIFO (last in, first out), 54
LTS (long-term support), 4

M
method reference, 39
monotonic queue, 74
monotonic stack, 70

N
NullPointerException, 15

P
postfix notation, 66
primitive type, 5

Q
queue, 59

R
reference type, 5

S
stable sort, 40
stack, 54
string pool, 10